

OpenBook · Forward Deployed Engineer — A Field  
Manual for AI Deployment

# Contents

|                                                                                    |           |
|------------------------------------------------------------------------------------|-----------|
| <b>Preface</b>                                                                     | <b>9</b>  |
| What This Book Is . . . . .                                                        | 9         |
| Who It’ s For . . . . .                                                            | 9         |
| Who It’ s Not For . . . . .                                                        | 9         |
| Three Promises . . . . .                                                           | 10        |
| Core Sources . . . . .                                                             | 10        |
| How to Read It . . . . .                                                           | 10        |
| Feedback . . . . .                                                                 | 11        |
| <b>Reading Guide</b>                                                               | <b>12</b> |
| Pick a Track . . . . .                                                             | 12        |
| Difficulty Markers . . . . .                                                       | 13        |
| Chapter Format . . . . .                                                           | 13        |
| Recurring Concepts . . . . .                                                       | 14        |
| Companion Resources . . . . .                                                      | 14        |
| <b>Part I: Getting Started — The Shape of FDE Engineering</b>                      | <b>15</b> |
| What This Part Solves . . . . .                                                    | 15        |
| Chapters . . . . .                                                                 | 15        |
| Relation to Other Parts . . . . .                                                  | 16        |
| <b>Chapter 1: The FDE Workflow — A Real Week, A Real Quarter</b>                   | <b>17</b> |
| Opening . . . . .                                                                  | 17        |
| 1.1 The Weekly Rhythm — The FDE’ s “Clocks” . . . . .                              | 18        |
| 1.2 The Quarterly Rhythm — The FDE’ s “Four Quadrants” . . . . .                   | 19        |
| 1.3 Workflow Differences Across the Two Tracks . . . . .                           | 20        |
| 1.4 Where Should the Time Go — Three Diagnostics . . . . .                         | 21        |
| 1.5 A Real Counter-Example . . . . .                                               | 22        |
| Key Citations . . . . .                                                            | 22        |
| Action Checklist . . . . .                                                         | 22        |
| Anti-Pattern Checklist . . . . .                                                   | 23        |
| Relation to the Next Chapter . . . . .                                             | 23        |
| <b>Chapter 2: The Three Iron Rules — Sell Outcomes / Fix Forward / Eval-Driven</b> | <b>24</b> |
| Opening . . . . .                                                                  | 24        |
| 2.1 Iron Rule 1: Sell the Outcome, Not the Product . . . . .                       | 24        |
| 2.2 Iron Rule 2: Fix Forward . . . . .                                             | 26        |

|                                                                                          |           |
|------------------------------------------------------------------------------------------|-----------|
| 2.3 Iron Rule 3: Eval-Driven . . . . .                                                   | 27        |
| 2.4 How the Three Rules Relate . . . . .                                                 | 29        |
| 2.5 How to Sell These Three Rules to the Team / Your Manager . . . . .                   | 29        |
| Key Citations . . . . .                                                                  | 30        |
| Action Checklist . . . . .                                                               | 30        |
| Anti-Pattern Checklist . . . . .                                                         | 30        |
| Relation to the Next Chapter . . . . .                                                   | 30        |
| <b>Chapter 3: Two FDE Modes — Data-Driven vs LLM-Driven</b>                              | <b>31</b> |
| Opening . . . . .                                                                        | 31        |
| 3.1 The Essential Difference Between the Two Modes . . . . .                             | 31        |
| 3.2 What Decides the Mode Is the Project, Not the Company . . . . .                      | 32        |
| 3.3 How the “Three Iron Rules” Land Differently in Each Mode . . . . .                   | 33        |
| 3.4 Two Sets of Muscles in the Tool Stack . . . . .                                      | 34        |
| 3.5 Switching Modes Inside a Single Project . . . . .                                    | 35        |
| 3.6 Mapping the Two Modes from an AWS Lens . . . . .                                     | 35        |
| 3.7 Self-Check: Which Mode Should You Lean Into Right Now . . . . .                      | 36        |
| Key Citations . . . . .                                                                  | 36        |
| Action Checklist . . . . .                                                               | 37        |
| Anti-Pattern Checklist . . . . .                                                         | 37        |
| Relation to the Next Part . . . . .                                                      | 37        |
| <b>Part II: Customer Discovery — Requirements, Engineer-Style</b>                        | <b>38</b> |
| What This Part Solves . . . . .                                                          | 38        |
| Chapters . . . . .                                                                       | 38        |
| How This Part Connects . . . . .                                                         | 38        |
| <b>Chapter 4: Discovery — Observe Before You Ask</b>                                     | <b>40</b> |
| Opening . . . . .                                                                        | 40        |
| 4.1 Why Observation Beats Asking . . . . .                                               | 40        |
| 4.2 Three Observation Techniques . . . . .                                               | 41        |
| 4.3 The 12 Questions You Must Always Ask . . . . .                                       | 42        |
| 4.4 Discovery Report Template . . . . .                                                  | 43        |
| 4.5 Common AWS Data-Access Flow During Discovery . . . . .                               | 44        |
| 4.6 Discovery Specifics for LLM Projects . . . . .                                       | 45        |
| Key Citations . . . . .                                                                  | 46        |
| Action Checklist . . . . .                                                               | 46        |
| Anti-Pattern Checklist . . . . .                                                         | 46        |
| Connection to the Next Chapter . . . . .                                                 | 46        |
| <b>Chapter 5: From Requirements to Acceptance — Eval Set / Acceptance Criteria / SOW</b> | <b>47</b> |
| Opening . . . . .                                                                        | 47        |
| 5.1 How the Three Artifacts Relate . . . . .                                             | 48        |
| 5.2 Eval Set v0.1 — The Engineer’ s Exam . . . . .                                       | 48        |
| 5.3 Acceptance Criteria — The Customer’ s Signature Line . . . . .                       | 51        |
| 5.4 SOW — The Commercial Artifact . . . . .                                              | 52        |
| 5.5 How the Three Cross-Check Each Other . . . . .                                       | 54        |

|                                                                                     |           |
|-------------------------------------------------------------------------------------|-----------|
| 5.6 An End-to-End Mini Example . . . . .                                            | 54        |
| Key Citations . . . . .                                                             | 56        |
| Action Checklist . . . . .                                                          | 56        |
| Anti-Pattern Checklist . . . . .                                                    | 56        |
| Connection to the Next Part . . . . .                                               | 57        |
| <b>Part III: Scaffolding — Six Weeks to a Running LLM Workflow</b>                  | <b>58</b> |
| What This Part Solves . . . . .                                                     | 58        |
| Chapters . . . . .                                                                  | 58        |
| Relation to Other Parts . . . . .                                                   | 58        |
| <b>Chapter 6: Tech Stack Quick-Decision Matrix — Model / Framework / Database /</b> |           |
| <b>Orchestration</b>                                                                | <b>59</b> |
| Opening . . . . .                                                                   | 59        |
| 6.1 Two Principles for Stack Selection . . . . .                                    | 60        |
| 6.2 Model Selection Matrix . . . . .                                                | 60        |
| 6.3 Framework Selection Matrix . . . . .                                            | 62        |
| 6.4 Vector Store Selection Matrix . . . . .                                         | 63        |
| 6.5 Orchestration / Workflow Selection . . . . .                                    | 64        |
| 6.6 Monitoring / Trace Selection . . . . .                                          | 65        |
| 6.7 The One-Page Quick-Decision Table (30 Minutes) . . . . .                        | 66        |
| Key Quotes . . . . .                                                                | 67        |
| Action Checklist . . . . .                                                          | 67        |
| Anti-Pattern Checklist . . . . .                                                    | 67        |
| Relation to the Next Chapter . . . . .                                              | 68        |
| <b>Chapter 7: Decision Tree — RAG / Fine-tune / Prompting / Agent</b>               | <b>69</b> |
| Opening . . . . .                                                                   | 69        |
| 7.1 The Essential Difference Between the Four Approaches . . . . .                  | 69        |
| 7.2 The Main Decision Tree . . . . .                                                | 70        |
| 7.3 RAG vs Fine-tune — The Eternal Confusion . . . . .                              | 71        |
| 7.4 Sub-Decisions Inside RAG . . . . .                                              | 72        |
| 7.5 Agent — When to Reach for One . . . . .                                         | 74        |
| 7.6 The Master Decision Table . . . . .                                             | 75        |
| 7.7 Switching Signals . . . . .                                                     | 76        |
| Key Quotes . . . . .                                                                | 77        |
| Action Checklist . . . . .                                                          | 77        |
| Anti-Pattern Checklist . . . . .                                                    | 77        |
| Relation to the Next Chapter . . . . .                                              | 77        |
| <b>Chapter 8: Eval First, Code Second — A Practical Guide</b>                       | <b>79</b> |
| Opening . . . . .                                                                   | 79        |
| 8.1 The Engineering Definition of Eval-Driven Development . . . . .                 | 79        |
| 8.2 The “Pyramid Structure” of an Eval Set . . . . .                                | 80        |
| 8.3 Three Metrics for an Eval Set, in Practice . . . . .                            | 80        |
| 8.4 Wiring the Eval Set into CI — Practice . . . . .                                | 82        |
| 8.5 The “Distribution” Mindset for Scoring . . . . .                                | 85        |
| 8.6 Production Sampling Feedback — Closing the Loop . . . . .                       | 86        |

|                                                                                              |            |
|----------------------------------------------------------------------------------------------|------------|
| 8.7 Eval-Set Anti-Patterns . . . . .                                                         | 87         |
| Key Quotes . . . . .                                                                         | 87         |
| Action Checklist . . . . .                                                                   | 88         |
| Anti-Pattern Checklist . . . . .                                                             | 88         |
| Relation to the Next Part . . . . .                                                          | 88         |
| <b>Part IV: Data &amp; Integration — Inside Real Customer Environments</b>                   | <b>89</b>  |
| What This Part Solves . . . . .                                                              | 89         |
| Chapters . . . . .                                                                           | 89         |
| Relation to Other Parts . . . . .                                                            | 89         |
| <b>Chapter 9: The Customer Data Stack — Ontology / ETL / Real-Time Pipelines</b>             | <b>91</b>  |
| Opening . . . . .                                                                            | 91         |
| 9.1 The Typical Shape of a Customer Data Stack . . . . .                                     | 91         |
| 9.2 Ontology — Unifying What’ s Scattered . . . . .                                          | 92         |
| 9.3 ETL — Making the Data Flow . . . . .                                                     | 94         |
| 9.4 Data Lineage — The Lifeline of Failure Diagnosis . . . . .                               | 95         |
| 9.5 Real-Time Data Pipelines — Use Sparingly . . . . .                                       | 96         |
| 9.6 The “Minimum 5 Things” of Data Governance . . . . .                                      | 97         |
| 9.7 A Real End-to-End Example . . . . .                                                      | 97         |
| Key Quotes . . . . .                                                                         | 98         |
| Action Checklist . . . . .                                                                   | 98         |
| Anti-Pattern Checklist . . . . .                                                             | 98         |
| Bridge to the Next Chapter . . . . .                                                         | 99         |
| <b>Chapter 10: Working in the Customer’ s VPC / Private Deployment / Offline Data Center</b> | <b>100</b> |
| Opening . . . . .                                                                            | 100        |
| 10.1 Three “Non-Cloud-Native” Customer Scenarios . . . . .                                   | 100        |
| 10.2 Scenario A: Working Inside the Customer’ s VPC . . . . .                                | 101        |
| 10.3 Scenario B: Customer Private Cloud / On-Prem Data Center . . . . .                      | 103        |
| 10.4 Scenario C: Air-Gap (Fully Offline) . . . . .                                           | 104        |
| 10.5 Engineering Notes for Private LLM Deployments . . . . .                                 | 105        |
| 10.6 Compliance Checklist (private deployment / common in China B2B) . . . . .               | 106        |
| Key Quotes . . . . .                                                                         | 107        |
| Action Checklist . . . . .                                                                   | 107        |
| Anti-Pattern Checklist . . . . .                                                             | 108        |
| Bridge to the Next Chapter . . . . .                                                         | 108        |
| <b>Chapter 11: Integrating with Legacy Systems — SSO / SCIM / API / Audit</b>                | <b>109</b> |
| Opening . . . . .                                                                            | 109        |
| 11.1 How the Four Things Relate . . . . .                                                    | 109        |
| 11.2 SSO — Plugging into Customer AD / Okta / Identity Center . . . . .                      | 110        |
| 11.3 SCIM — User Lifecycle Sync . . . . .                                                    | 112        |
| 11.4 API Integration — Calling the Customer’ s Legacy Systems . . . . .                      | 113        |
| 11.5 Audit Logs — “Can You Prove It?” . . . . .                                              | 114        |
| 11.6 Guardrails — Auditing the Agent’ s Behavior . . . . .                                   | 116        |
| 11.7 A Complete Integrated Example . . . . .                                                 | 116        |

|                                                                                                         |            |
|---------------------------------------------------------------------------------------------------------|------------|
| Key Quotes . . . . .                                                                                    | 117        |
| Action Checklist . . . . .                                                                              | 117        |
| Anti-Pattern Checklist . . . . .                                                                        | 118        |
| Bridge to the Next Part . . . . .                                                                       | 118        |
| <b>Part V: PoC → Production — Crossing the Hardest Gap</b>                                              | <b>119</b> |
| What This Part Solves . . . . .                                                                         | 119        |
| Chapters . . . . .                                                                                      | 119        |
| Relation to Other Parts . . . . .                                                                       | 119        |
| <b>Chapter 12: PoC Pass-Line Conditions — What Makes It, What Stalls Out</b>                            | <b>120</b> |
| Opening . . . . .                                                                                       | 120        |
| 12.1 PoC vs Production — Engineering Bars Differ by 100x . . . . .                                      | 120        |
| 12.2 The 6 Things to Do During the PoC Stage . . . . .                                                  | 121        |
| 12.3 The 4 Signals of “Guaranteed Stall” . . . . .                                                      | 123        |
| 12.4 The 5 Hard Metrics for PoC Pass-Line . . . . .                                                     | 124        |
| 12.5 A Real PoC → Production Timeline . . . . .                                                         | 125        |
| 12.6 A Real Counter-Example . . . . .                                                                   | 125        |
| Key Quotes . . . . .                                                                                    | 126        |
| Hands-On Checklist . . . . .                                                                            | 126        |
| Anti-Patterns . . . . .                                                                                 | 126        |
| Relation to the Next Chapter . . . . .                                                                  | 127        |
| <b>Chapter 13: Observability / Cost / Canary / Rollback</b>                                             | <b>128</b> |
| Opening . . . . .                                                                                       | 128        |
| 13.1 Observability — Not Just “Reading Logs” . . . . .                                                  | 128        |
| 13.2 Cost Control — Tokens Are the New “Utility Bill” . . . . .                                         | 130        |
| 13.3 Canary Releases — Not “All or Nothing” . . . . .                                                   | 131        |
| 13.4 Rollback — Must Be Switchable Within 5 Minutes . . . . .                                           | 133        |
| 13.5 Failure Drills — Chaos Engineering for LLM . . . . .                                               | 134        |
| 13.6 A Sample Production Dashboard . . . . .                                                            | 134        |
| Key Quotes . . . . .                                                                                    | 135        |
| Hands-On Checklist . . . . .                                                                            | 135        |
| Anti-Patterns . . . . .                                                                                 | 135        |
| Relation to the Next Part . . . . .                                                                     | 136        |
| <b>Part VI: The Agent Era — New Engineering Tasks for the FDE</b>                                       | <b>137</b> |
| What This Part Solves . . . . .                                                                         | 137        |
| Chapters . . . . .                                                                                      | 137        |
| Relation to Other Parts . . . . .                                                                       | 137        |
| <b>Chapter 14: Deploying Agents in Customer Environments — Toolsets / Sandboxing / Failure Recovery</b> | <b>139</b> |
| Opening . . . . .                                                                                       | 139        |
| 14.1 Toolset Design — Less Is More . . . . .                                                            | 140        |
| 14.2 Sandbox — The Agent’s “Permission Boundary” . . . . .                                              | 141        |
| 14.3 Failure Recovery — What to Do When a Multi-Step Task Breaks . . . . .                              | 143        |
| 14.4 Agent Eval — A Different Kind of Eval . . . . .                                                    | 145        |
| 14.5 Human-in-the-Loop — High-Risk Actions Always Need a Reviewer . . . . .                             | 146        |

|                                                                                          |            |
|------------------------------------------------------------------------------------------|------------|
| 14.6 Deployment Checklist . . . . .                                                      | 147        |
| 14.7 An End-to-End Agent Deployment . . . . .                                            | 147        |
| Key Citations . . . . .                                                                  | 148        |
| Action Checklist . . . . .                                                               | 148        |
| Anti-Pattern Checklist . . . . .                                                         | 149        |
| Relation to the Next Chapter . . . . .                                                   | 149        |
| <b>Chapter 15: MCP and Enterprise Integration — Wiring Agents into Customer Tools</b>    | <b>150</b> |
| Opening . . . . .                                                                        | 150        |
| 15.1 What MCP Is . . . . .                                                               | 151        |
| 15.2 Three Forms of Enterprise MCP Deployment . . . . .                                  | 151        |
| 15.3 Writing an Enterprise MCP Server — Hands-On . . . . .                               | 152        |
| 15.4 Four Engineering Pitfalls of MCP . . . . .                                          | 154        |
| 15.5 AWS in Practice: Bedrock Agent + MCP Integration . . . . .                          | 155        |
| 15.6 Multi-Server Choreography — A Customer Scenario . . . . .                           | 157        |
| 15.7 Security Checklist . . . . .                                                        | 157        |
| Key Citations . . . . .                                                                  | 158        |
| Action Checklist . . . . .                                                               | 158        |
| Anti-Pattern Checklist . . . . .                                                         | 158        |
| Relation to the Next Part . . . . .                                                      | 158        |
| <b>Part VII: Handoff and Mastery</b>                                                     | <b>160</b> |
| What This Part Solves . . . . .                                                          | 160        |
| Chapters . . . . .                                                                       | 160        |
| Relation to Other Parts . . . . .                                                        | 160        |
| <b>Chapter 16: Handoff + Pattern Extraction — Abstracting the Solution into Reusable</b> | <b>161</b> |
| <b>Assets</b>                                                                            | <b>161</b> |
| Opening . . . . .                                                                        | 161        |
| 16.1 The Engineering Definition of Handoff . . . . .                                     | 162        |
| 16.2 The 3-Week Handoff Countdown . . . . .                                              | 162        |
| 16.3 The Runbook — the Customer’ s Operations Manual . . . . .                           | 163        |
| 16.4 Training — a 4-Hour Course . . . . .                                                | 164        |
| 16.5 Pattern Extraction — Making the Next Project 5x Faster . . . . .                    | 164        |
| 16.6 The Engineering Moves of Pattern Extraction . . . . .                               | 166        |
| 16.7 Industry Templates — One Example . . . . .                                          | 166        |
| 16.8 Pattern Management at the Company Level . . . . .                                   | 167        |
| Key Citations . . . . .                                                                  | 168        |
| Action Checklist . . . . .                                                               | 168        |
| Anti-Pattern Checklist . . . . .                                                         | 168        |
| Relation to the Next Chapter . . . . .                                                   | 168        |
| <b>Chapter 17: The FDE’ s T-Shape Growth — Engineering Depth + Industry Depth</b>        | <b>170</b> |
| Opening . . . . .                                                                        | 170        |
| 17.1 T-Shape = Engineering Depth × Industry Depth . . . . .                              | 170        |
| 17.2 Engineering Breadth — the 5 Required Foundations . . . . .                          | 171        |
| 17.3 Industry Depth — How to Pick One and How to Go Deep . . . . .                       | 172        |
| 17.4 How to Deepen Industry Depth — 6 Moves . . . . .                                    | 173        |

|                                                                               |            |
|-------------------------------------------------------------------------------|------------|
| 17.5 Engineering Depth vs Engineering Breadth — Go Deep on 1-2 . . . . .      | 173        |
| 17.6 The 5-Year Growth Path . . . . .                                         | 174        |
| 17.7 Anti-Patterns — 5 Years and No T . . . . .                               | 175        |
| 17.8 Four Sentences to You, Right Now . . . . .                               | 175        |
| Key Citations . . . . .                                                       | 176        |
| Action Checklist . . . . .                                                    | 176        |
| Anti-Pattern Checklist . . . . .                                              | 176        |
| Closing the Book . . . . .                                                    | 177        |
| <b>Appendix A: FDE Tool Stack Cheat Sheet (2026 Edition)</b>                  | <b>178</b> |
| A.1 Model APIs . . . . .                                                      | 178        |
| A.2 Orchestration / Frameworks . . . . .                                      | 178        |
| A.3 Vector Stores / Knowledge Bases . . . . .                                 | 179        |
| A.4 Eval / Trace . . . . .                                                    | 179        |
| A.5 Data Engineering . . . . .                                                | 180        |
| A.6 Deployment / Runtime . . . . .                                            | 180        |
| A.7 Security / Compliance . . . . .                                           | 180        |
| A.8 IaC . . . . .                                                             | 181        |
| A.9 Monitoring / Alerting . . . . .                                           | 181        |
| A.10 Customer Collaboration . . . . .                                         | 181        |
| <b>Appendix B: Model / Framework / Platform Comparison Matrix</b>             | <b>183</b> |
| B.1 Mainstream LLMs — 9-Dimension Comparison . . . . .                        | 183        |
| B.2 RAG Framework Comparison . . . . .                                        | 184        |
| B.3 Agent Framework Comparison . . . . .                                      | 184        |
| B.4 Vector Store Comparison — 6 Production Dimensions . . . . .               | 185        |
| B.5 Eval Tool Comparison . . . . .                                            | 186        |
| B.6 Deployment Platform Comparison — From an LLM-app Lens . . . . .           | 187        |
| B.7 Programming Languages / Runtimes . . . . .                                | 187        |
| B.8 Decision Synthesis Table . . . . .                                        | 188        |
| <b>Appendix C: Eval Set Design Templates</b>                                  | <b>190</b> |
| C.1 Common Design Principles . . . . .                                        | 190        |
| C.2 Template 1: RAG QA Eval . . . . .                                         | 191        |
| C.3 Template 2: Agent Task Execution Eval . . . . .                           | 193        |
| C.4 Template 3: Classification / Routing Eval . . . . .                       | 195        |
| C.5 Template 4: Structured Extraction Eval . . . . .                          | 196        |
| C.6 A Full Eval Project Layout . . . . .                                      | 197        |
| C.7 Annotation Workflow — Getting Customer SMEs to Work Efficiently . . . . . | 198        |
| C.8 Eval Set Lifecycle Management . . . . .                                   | 198        |
| C.9 An Eval Checklist . . . . .                                               | 198        |
| <b>Appendix D: Customer Kickoff Pack — 12 Templates Ready for Week One</b>    | <b>199</b> |
| Template 1: Discovery Outline (for interviews) . . . . .                      | 199        |
| Template 2: Discovery Summary Report . . . . .                                | 200        |
| Template 3: PoC SOW Skeleton (6-8 weeks) . . . . .                            | 201        |
| Template 4: Eval Set v0 (jsonl starter) . . . . .                             | 202        |
| Template 5: Security / Compliance Questionnaire Response Template . . . . .   | 202        |



|                                                                                  |            |
|----------------------------------------------------------------------------------|------------|
| Template 6: Architecture Diagram (Mermaid starter)                               | 203        |
| Template 7: Risk Register                                                        | 204        |
| Template 8: Weekly Report Template                                               | 204        |
| Template 9: Runbook Skeleton (3 weeks before Handoff)                            | 205        |
| 3. Top-10 incident SOPs                                                          | 205        |
| 4. How to run Evals                                                              | 205        |
| 5. Key configuration                                                             | 205        |
| 6. Data / KB update SOP                                                          | 205        |
| 7. Escalation                                                                    | 206        |
| Template 12: Customer Stakeholder Map                                            | 207        |
| Usage Summary                                                                    | 208        |
| <b>Glossary</b>                                                                  | <b>209</b> |
| A                                                                                | 209        |
| B                                                                                | 209        |
| C                                                                                | 209        |
| D                                                                                | 209        |
| E                                                                                | 210        |
| F                                                                                | 210        |
| G                                                                                | 210        |
| H                                                                                | 210        |
| I                                                                                | 210        |
| J                                                                                | 210        |
| L                                                                                | 211        |
| M                                                                                | 211        |
| N                                                                                | 211        |
| O                                                                                | 211        |
| P                                                                                | 211        |
| R                                                                                | 211        |
| S                                                                                | 211        |
| T                                                                                | 212        |
| V                                                                                | 212        |
| Other                                                                            | 212        |
| <b>Bibliography</b>                                                              | <b>213</b> |
| 1. Most-Cited Core Sources                                                       | 213        |
| 2. The May 4, 2026 Inflection Event                                              | 214        |
| 3. Industry Blogs & Playbooks                                                    | 214        |
| 4. Chinese-Language Community Sources (referenced for the China-edition Part VI) | 215        |
| 5. FDE Rule Book — Identifiers                                                   | 215        |
| 6. Citation Conventions                                                          | 215        |

# Preface

## What This Book Is

Monday, 9:00 AM. You boot up.

The customer's security officer has emailed you a 27-item compliance questionnaire. Slack has 4 unread channels from the previous FDE; the latest message is "Why is the RAG recall dropping again?"

Your calendar shows: 11 AM customer VP sync, 13:00 model vendor cost review, 15:00 PoC review deck due to your manager.

You have one hour before you open the IDE.

What do you spend it on?

This is the kind of question this book answers.

---

## Who It's For

**Engineers already doing FDE work, or assigned FDE work without the title.**

Concretely:

- 5+ years of engineering experience; can own an entire backend service
- Have run at least one customer PoC; have been stuck in production hand-off at least once
- Have called LLM APIs and shipped demos; haven't yet run an agent in a customer environment
- Comfortable reading OpenAPI specs, running SQL, reading distributed traces — no intermediaries needed

If you've never been customer-facing or never used an LLM API, do a few small projects first. This book doesn't teach the basics.

---

## Who It's Not For

| Not the audience                              | Why                                          |
|-----------------------------------------------|----------------------------------------------|
| Job-seekers deciding “should I become an FDE” | This book skips the career-judgment layer    |
| Investors / policymakers                      | Too low-level; you want conclusions          |
| Students / new grads                          | Missing prerequisites — won’ t be useful yet |
| Readers wanting LLM internals                 | No transformer / attention coverage          |

## Three Promises

### 1. Every judgment call comes with a concrete scenario.

No “FDEs should value customer interviews.” Instead: “In your first week on a project, what to do, what not to do, and the 5 questions to ask in the first 3 customer meetings.”

### 2. Every technical choice gives you decision dimensions.

No “RAG beats fine-tuning” or vice versa. A matrix instead: data volume / update frequency / answer determinism / inference budget / compliance — judge for yourself.

### 3. Every chapter ends with an action checklist + an anti-pattern checklist.

The action checklist is what you can copy-do this week; the anti-pattern checklist is what real FDEs keep getting wrong.

## Core Sources

The book leans heavily on:

- **A. Lawrence**, Forward Deployed Engineer Rule Book (Oct 2025)
- **Bob McGrew @ Y Combinator** (2025) — “Sell the outcome, not the product”
- **AWS GenAI Innovation Center** — public 45-day / 73% figures
- **Conikeec @ Substack**, The FDE Playbook
- **Nabeel Qureshi**, Reflections on Palantir

Full list in [bibliography.md](#).

## How to Read It

The most useful way: **read it alongside a real FDE project.**

After each chapter, ask yourself:

1. Does my current project have this problem?

2. If yes, how did I handle it before?
3. How does the chapter' s approach differ from mine? Which is better?

End-to-end takes 6-7 hours; with a real project alongside, 2-3 weeks is probably more useful.

---

## Feedback

GitHub Issues is the fastest channel. What I want most:

- **Counter-examples:** where the book' s recipe failed in your context
- **Real numbers:** your PoC conversion rate, customer NPS, week-one deliverables
- **Missing chapters:** critical engineering topics you think this book missed

The next edition will probably absorb the strongest signals.

---

Written May 2026.

# Reading Guide

17 chapters, ~6-7 hours end to end. Two tracks running in parallel — pick the route that matches your FDE flavor.

---

## Pick a Track

### Track A: LLM-Application FDE (default)

**Best for:** FDEs at OpenAI / Anthropic / Cohere / Chinese model labs and downstream startups, shipping LLM applications in customer environments.

**Path:** Part I → Part II → **Part III** → Part V → **Part VI** → Part VII.

**Highlights:** Ch 6 stack matrix, Ch 7 RAG/FT/Agent decision tree, Ch 8 eval-driven development, Ch 14 agent deployment, Ch 15 MCP.

**Time:** ~5 hours.

---

### Track B: Field-Delivery FDE (Palantir style)

**Best for:** FDEs at Palantir, AWS GenAI Innovation Center, AI consultancies — delivering data + software in customer environments.

**Path:** Part I → Part II → **Part IV** → Part V → Part VII.

**Highlights:** Ch 9 ontology / ETL, Ch 10 VPC deployment, Ch 11 SSO/SCIM/audit, Ch 12 PoC cut-over, Ch 16 handoff.

**Time:** ~4 hours.

---

### Track C: Full Read (FDE leads, staff/principal cross-track)

**Best for:** FDE team leads, anyone doing internal training, FDEs who span both tracks.

**Path:** Part I straight through to Part VII.

**Time:** 6-7 hours.

---

## Track D: Reference

**Best for:** A specific question — “how do I size hardware for private deployment?” , “how do I write PoC acceptance criteria?”

**Use:** Jump to appendices A–D and the relevant chapter.

---

| Your question                                         | Where to look          |
|-------------------------------------------------------|------------------------|
| Choose a model / framework / vector DB                | Appendix A, Appendix B |
| Write an eval set                                     | Ch 8, Appendix C       |
| Write an SOW / security questionnaire / risk register | Appendix D             |
| Design an agent sandbox                               | Ch 14                  |
| Customer VPC deployment checklist                     | Ch 10                  |

---

---

## Difficulty Markers

Direct — copy and apply

Reflection — judge against your project

Decision — org / commercial / compliance call

---

| Part                  | Difficulty |
|-----------------------|------------|
| I Foundations         |            |
| II Discovery          |            |
| III Scaffolding       |            |
| IV Data & Integration |            |
| V PoC→Production      |            |
| VI Agent Era          |            |
| VII Handoff & Mastery |            |

---

---

## Chapter Format

Opening — concrete scene or concrete counter-example

Body — 3-5 sections, each one engineering call

Key citations — a quote or two from Lawrence / McGrew / Conikeec / AWS

Action checklist — copy-do items for this week

Anti-pattern checklist — recurring failures real FDEs hit

If you’ re short on time, **read the action and anti-pattern checklists first.**

---

## Recurring Concepts

Full glossary in [glossary.md](#). The load-bearing few:

| Concept                     | One-liner                                                                                                   |
|-----------------------------|-------------------------------------------------------------------------------------------------------------|
| <b>Sell the outcome</b>     | Sell results, not tools — McGrew’ s first law                                                               |
| <b>Fix Forward</b>          | Fix it on-site, don’ t drag it back to HQ — Lawrence’ s principle                                           |
| <b>Eval-driven Ontology</b> | Eval set first, code second — Ch 8’ s spine<br>Palantir’ s core abstraction; Ch 9 gives an engineering view |
| <b>Agent Toolset</b>        | The set of tools an agent can call — Ch 14’ s topic                                                         |
| <b>MCP</b>                  | Model Context Protocol — standard for connecting agents to enterprise tools — Ch 15                         |
| <b>PoC Cut-over</b>         | The chasm from PoC to production — Ch 12                                                                    |
| <b>Handoff</b>              | The departure-day delivery move — Ch 16                                                                     |

---

## Companion Resources

- **research/** — All research notes (7 docs, public)
- **Appendix D** — 12 ready-to-use templates for week one
- **GitHub Issues** — Counter-examples, real numbers, missing-chapter requests

---

Next: [Preface](#) → [Part I Intro](#)

# Part I: Getting Started — The Shape of FDE Engineering

Scope: General / starting point for both tracks

This Part is the “map” for every FDE. By the end, you should be able to answer three questions: 1. What does an FDE’s real schedule actually look like? 2. Which three iron rules decide whether you do the job well? 3. Are you currently doing “data-driven FDE” or “LLM-driven FDE” work — and how do you switch?

---

## What This Part Solves

For a lot of people, the early confusion when they take on FDE work isn’t “I don’t know the tech” — it’s **“I don’t know where my time should be going.”**

Does meeting with the customer count as work? Does writing a demo count as delivery? Does building an Eval set count as real work? Does pulling Ops into a Slack channel to wire up SSO fall inside the FDE remit?

Part I doesn’t teach any specific technology. Its only job — before you open the IDE — is to set up a coordinate system in your head for “what an FDE engineer does in a week, how to tell right from wrong, and which class of situation you’re currently in.”

The remaining six Parts are built on this coordinate system.

## Chapters

- **Chapter 1: The FDE Workflow.** A weekly schedule plus a quarterly rhythm, contrasted across the two tracks (data vs LLM).
- **Chapter 2: The Three Iron Rules.** Sell outcomes / Fix Forward / Eval-driven — the three rules that draw the line between getting it right and getting it wrong.
- **Chapter 3: Two FDE Modes.** Data-driven (Palantir / AWS GenAIIC style) vs LLM-driven (OpenAI / Anthropic style) — and when to switch between them inside the same project.



## Relation to Other Parts

- No prerequisites: this is the start of the book.
  - Forward links: the “three iron rules” from Ch 2 thread through Parts II–VII; the “two modes” framing in Ch 3 will determine whether you weight Part III (LLM) or Part IV (field delivery) more heavily later.
-

# Chapter 1: The FDE Workflow — A Real Week, A Real Quarter

## Opening

Tuesday, Shanghai, conference room at a fintech customer.

- 09:00 Customer Ops @-mentions you in Slack — "RAG recall dropped from 0.82 to 0.61 overnight"
- 09:15 Open LangFuse traces, scroll through the last 200 failed requests from yesterday. 78 of them are about "policies issued in the last 3 days" — the customer added a batch of new insurance products last week and didn't sync them into the KB
- 09:40 Drop a two-line re-ingest SOP into the channel
- 10:00 Standup with the customer's business PM: 4 new intents to add, boss wants a demo next week
- 10:30 Pair with customer engineering on an OAuth error (you didn't write it, but you know the system best)
- 11:00 30-minute call with a remote staff engineer at your company — should we split retrieval into a separate retriever-ensemble service? Park it for now
- 12:00 Lunch in the customer cafeteria. Someone in Operations across the table grumbles "that dashboard from last time is slow" — you note it down. May not matter for this project, but it's a signal
- 13:30 Write the 4 prompt variants you'll demo this afternoon, run them against the Eval set first — you do not freestyle in front of the customer
- 15:00 Demo. The customer's business owner throws out 2 new scenarios on the spot. You don't engage head-on — "Let me run them through Eval and come back tomorrow with a pass/fail call"
- 17:00 Author Eval cases: 4 new scenarios × 5 edge samples = 20 entries
- 17:30 Ship last night's ingest fix to a 10% canary
- 18:00 Daily report: today's progress / 3 things for tomorrow / 1 risk

That' s a day of FDE.

Less than 25% of it is writing code, but every single thing you did was an engineering action.

---

## 1.1 The Weekly Rhythm — The FDE’ s “Clocks”

Abstract an FDE week into three “clocks” :

|                                                                                                                                                                                  |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Customer Cadence <ul style="list-style-type: none"><li>- Monday standup / Friday demo / monthly review</li><li>- Driven by the customer's calendar; FDE cannot miss it</li></ul> |
| Engineering Cadence <ul style="list-style-type: none"><li>- PR cadence, Eval cadence, release cadence</li><li>- Driven by you and your team</li></ul>                            |
| Learning Cadence <ul style="list-style-type: none"><li>- Read customer business docs, browse their wiki, eat lunch with Ops</li><li>- Only you can protect this one</li></ul>    |

**The classic rookie FDE mistake is letting the entire week be eaten by the Customer Cadence** — every day spent chasing customer requests, no fixed Engineering Cadence, and certainly no Learning Cadence.

Three weeks in, you’ ll notice: delivery quality slowly degrades because you put zero time into Eval, and the customer starts going around you to your manager because you’ re no longer offering anything insightful.

---

### Recommended Weekly Template (5-day version)

|       | Mon                     | Tue                            | Wed                            | Thu                | Fri                           |
|-------|-------------------------|--------------------------------|--------------------------------|--------------------|-------------------------------|
| 07:00 |                         |                                |                                |                    | ← Reserve learning time       |
| 09:00 | Customer standup + plan | Deep work (no-mtg)             | Customer standup + Eval review | Deep work (no-mtg) | Weekly demo + weekly report   |
| Lunch | With customer Ops       | Remote sync with your own team | With customer business PM      | Read customer wiki | With customer decision-makers |

|       |                     |                       |                      |                       |                                  |
|-------|---------------------|-----------------------|----------------------|-----------------------|----------------------------------|
| PM    | Discovery interview | Engineering deep work | Customer eng handoff | Engineering deep work | Customer demo + feedback digest  |
| 17:00 | Daily report        | Daily report          | Daily report         | Daily report          | Weekly report + next week's plan |

The non-negotiable piece is the **Tuesday / Thursday “Deep Work Days (no-meeting days)”** . Lose them and all you have left is “senior PM + pre-sales” — you’ re no longer a Forward Deployed Engineer.

## 1.2 The Quarterly Rhythm — The FDE’ s “Four Quadrants”

A quarter is roughly 12 weeks. A typical FDE project unfolds across “four quadrants” :

|                 | <u>Week 1-3</u>                               | <u>Week 4-7</u>                           | <u>Week 8-10</u>                          | <u>Week 11-12</u>                        |
|-----------------|-----------------------------------------------|-------------------------------------------|-------------------------------------------|------------------------------------------|
|                 | DISCOVERY                                     | SCAFFOLDING                               | PROD-IZATION                              | HANDOFF                                  |
| Goal            | Pin down the "real problem" and "real budget" | Get a minimal closed loop + Eval baseline | Stable launch + rollback plan             | Customer takes over + pattern extraction |
| Output          | Discovery report + SOW draft                  | Demoable build + Eval set v0.1            | Production deploy + monitoring dashboards | Runbook + Eval v1.0 + Handoff doc        |
| Iron rule focus | Sell outcome                                  | Eval-driven                               | Fix Forward                               | All three                                |
| Risk            | Drifting requirements + drifting expectations | Missing Eval                              | Production incident                       | No one maintains after Handoff           |

Remember one thing: **the boundaries between these four phases are blurry**. Discovery never “ends,” Eval is never “enough.” But each phase has a single **primary task**, and 70% of your time should be on it.

## Reading the Phase You' re In

When you take over a project, first figure out which phase it' s in:

Q1: Has the customer's "definition of success" been pinned to a number?

↓

No → You're in DISCOVERY. Do Discovery first; do not write code.

Yes ↓

Q2: Do you have an Eval set, and can you score the current version?

↓

No → You're in SCAFFOLDING. Build Eval first, then build the demo.

Yes ↓

Q3: Has someone in the customer's "real production environment" used it for at least a week?

↓

No → You're in PROD-IZATION. Focus on stability + monitoring.

Yes ↓

Q4: Is there someone inside the customer who can run it independently of you?

↓

No → You're in HANDOFF. Write Runbook + train.

Yes → Project done. Do pattern extraction (Ch 16).

90% of FDE failure stories trace back to **misreading which phase you' re in**. The most common mistake: thinking you' re in Scaffolding when you' re still in Discovery, then shipping a pile of features no one asked for.

---

## 1.3 Workflow Differences Across the Two Tracks

The same week looks very different for an **LLM application FDE** versus a **field-delivery FDE**:

| Time slot    | LLM Application FDE                                         | Field-Delivery FDE<br>(Palantir / AWS GenAIIC style)         |
|--------------|-------------------------------------------------------------|--------------------------------------------------------------|
| Mon AM       | Pull LangFuse / LangSmith traces, analyze failure samples   | Check customer pipeline alerts, confirm ETL is healthy       |
| Mon PM       | Discovery: align with business PM on intents and data shape | Discovery: align with data team on schema and data ownership |
| Tue full day | Tune prompts / upgrade RAG / swap models                    | Write ETL jobs / model Ontology entities / tune SQL          |
| Wed AM       | Run Eval, ship a new build                                  | Run data-validation batches, deploy pipelines                |

| Time slot    | LLM Application FDE                                 | Field-Delivery FDE<br>(Palantir / AWS GenAIIC style)                   |
|--------------|-----------------------------------------------------|------------------------------------------------------------------------|
| Wed PM       | Customer eng integration: API, rate limits, caching | Customer eng integration: DB connections, Kerberos, network allowlists |
| Thu full day | Deep work: refine eval set, ablate prompt variables | Deep work: optimize jobs / refactor data model                         |
| Fri          | Demo: scenario dialogues + metrics                  | Demo: dashboards + data correctness                                    |

**On the FDE job, both tracks are the same mindset and different muscles.** Inside the same company on the same project, you’ ll switch tracks across phases — that’ s exactly what Ch 3 unpacks.

## 1.4 Where Should the Time Go — Three Diagnostics

### **Diagnostic 1: $\geq 8$ hours per week on “non-coding engineering”**

That includes: Discovery interviews, labeling Eval, reading customer business docs, writing Runbooks, reading traces.

**Below 8 hours and the FDE tends to turn the project into “remote outsourcing.”**

### **Diagnostic 2: $\geq 1$ unsolicited customer face-time every two weeks**

Not a meeting you organized — one you walked into yourself. For example: - Lunch in the customer cafeteria - Friday afternoon, sit at their workstation and watch them use the product - A 30-minute 1:1 with a customer Ops or frontline employee

**Less than once every two weeks and the FDE drifts into “increasingly abstract information,”** building things further and further from the customer’ s real work.

### **Diagnostic 3: At least once a month, “push back” against the customer**

Not arguing for sport — **rejecting a request or a proposal with evidence and numbers.**

Examples: - The customer wants “company-wide multi-Agent collaboration” ; you push back and propose starting with a single Agent + tool augmentation - The customer wants to self-host Llama 70B; you cost out the inference and recommend starting on a hosted API

**Less than once a month and the FDE has usually already degraded into a “senior implementation engineer”** — judgment is no longer in play.

## 1.5 A Real Counter-Example

An anonymous FDE shared a failure story on Substack (2025):

Week 3 of the project, the customer said “we want an agent that can write emails, query CRM, edit calendars — all wired up.” I didn’t ask follow-ups. I went back and built a 6-week project with 3 tool integrations and orchestration logic. Demoed in week 8. Customer said “looks good” and ghosted us.

Later I found out their CRM was 80% empty fields, and the CEO said that line because he’d just sat through someone else’s demo. What was actually urgent was their sales team migrating data into ERP at month-end — a one-morning automation. I built a 6-week demo, and they couldn’t buy it.

**Diagnosis:** - Jumped into Scaffolding while still in Discovery (wrong phase) - No Eval, so “agent writes emails” had no quantifiable bar (broke Eval-driven) - Didn’t sell the outcome (data into ERP); sold the product (multi-tool agent)

Every later chapter in this book is, in some sense, helping you avoid this counter-example.

---

## Key Citations

“The most expensive line of code is the one nobody asked you to write.” — Conikeec, The FDE Playbook, 2025

“Forward deployment is not a job; it’s a posture toward the customer’s workflow.” — A. Lawrence, FDE Rule Book (paraphrased), 2025

“The best FDEs don’t sell the agent; they sell the after-state of the workflow.” — Bob McGrew @ YC, 2025

---

## Action Checklist

When you take on a new FDE project, do the following 7 things in week 1:

1. **Get access to the customer’s existing wiki / Confluence** — not a “to-do,” it must be in place by the end of week 1
2. **Start a personal “project journal” file** (Obsidian / Notion private space recommended); spend 5 minutes a day capturing 3 observations
3. **Draw your “Customer Cadence”** : which days of the week the customer has which standing meetings, and align your calendar to them
4. **Force two “Deep Work Days”** (Tuesday and Thursday recommended) — refuse all non-urgent meetings
5. **Before your first Discovery meeting**, prepare a 12-question checklist (see Ch 4)
6. **Find 1-2 frontline employees on the customer’s “non-decision tier,”** and book a 30-minute coffee (this is Lawrence’s Immersion Before Judgment)

7. By the end of the first weekend, write a memo titled “I assumed the customer wants X, but it might actually be Y” (a guard against the § 1.5 counter-example)
- 

## Anti-Pattern Checklist

- **Letting the entire week go to the Customer Cadence** (you become a “senior PM” )
  - **No “Deep Work Days”** (interrupted constantly, never get any engineering depth)
  - **Skipping Discovery and writing demos directly** (the most expensive code is code no one asked for)
  - **Not writing daily / weekly reports** (six months later you can’ t remember what you did, and pattern extraction is impossible)
  - **Never “pushing back” on the customer** (the customer asks, you build — a few weeks from FDE degrading into implementation engineer)
  - **Not knowing which phase you’ re in** (use the § 1.2 decision tree to self-check)
- 

## Relation to the Next Chapter

This chapter mapped out the FDE’ s “shape of time.” The next chapter maps the “shape of judgment” — the three iron rules that decide whether you’ re doing the job right. After both chapters, you’ ll have a complete coordinate system for FDE engineering.



# Chapter 2: The Three Iron Rules — Sell Outcomes / Fix Forward / Eval-Driven

## Opening

You're in the customer's conference room. The customer's CTO is pointing at an architecture diagram you drew on the projector. He points at the box in the middle and asks: "Once we buy this Agent, who's responsible for tuning it?"

Your gut reactions are:

- A) "Your Ops team can tune the prompts on a regular cadence"
- B) "I'll own it; monthly regression on me"
- C) "Your Sales Director can take it on"

Wrong. All three are degraded versions of FDE thinking.

There's only one right answer —

"What you originally wanted wasn't an Agent. You wanted the month-end sales report on-time rate to go from 60% to 95%. This month we hit 92%. Next month, when we hit 95%, whoever has that 95% directly tied to their KPI is the owner."

Pulling the conversation away from the product and back to the outcome — that's the most important of the FDE's three iron rules.

---

## 2.1 Iron Rule 1: Sell the Outcome, Not the Product

### Why This One Matters Most

Bob McGrew said this in a 2025 YC interview, and it's been quoted endlessly:

"Stop selling the product — sell the outcome."

Why does this rule matter most?

Because **software / AI systems are means; the customer's business result is the end.**  
A customer doesn't pay because they want "an Agent" — they pay because they want to:

- Take month-end report on-time rate from 60% to 95%
- Take customer-ticket first-response time from 4 hours to 30 minutes
- Move sales conversion up by 5 points

If you're selling a "tool," the customer evaluates you with "is this tool good?" — that judgment isn't quantifiable, and it never reaches a finish line.

If you're selling a "result," the customer evaluates you with "did the number land?" — that judgment is quantifiable, has a finish line, and reaches a mutual win.

## How an Engineer Operationalizes This

Every conversation with the customer, ask yourself:

Q1: At the end of this conversation, did the customer change their "product understanding" or their "business-result expectation"?

Q2: Can I describe the outcome of this conversation with a single number?

Q3: Once that number lands, what's the number for the next conversation?

**The FDE's job is to keep dragging customer conversations from "feature discussion" back to "result discussion."**

## In Practice: Three Conversation Translation Templates

Customer: "Can the Agent also handle email?"

Direct answer: "Yes, we'll schedule it for next week"

Translation: "You're adding email so sales can reply faster on open opportunities, right? Let's check the current median reply time, estimate what email integration would bring it down to, then put it in the backlog."

---

Customer: "What's the actual accuracy of your RAG?"

Direct answer: "0.83 on the test set, 0.78 in production"

Translation: "0.78 recall by itself doesn't mean much. The number that matters: account managers used to take 4 minutes to find a policy clause via search. They now take an average of 28 seconds with RAG. That 28 seconds maps to the '<30 seconds' KPI you originally set."

---

Customer: "We want an 'AI-powered company-wide knowledge base'"

Direct answer: "Got it, we'll draft the architecture"

Translation: "'Company-wide knowledge base' is too big a target.  
Pick one outcome first: new-hire ramp time from 30 days to 15? Or customer-service resolution rate from 65% to 80%? Once we pick one, we can ship a first version in 6 weeks."

---

## 2.2 Iron Rule 2: Fix Forward

### The Principle — Fix It On Site

Lawrence hammers Fix Forward repeatedly in the Rule Book:

“Don’ t carry the problem back to HQ. Fix it at the customer site.”

It sounds simple, but the engineering philosophy underneath is:

**FDE is not a two-stage role of “front-line pre-sales + back-office R&D.” On the front line, the FDE is pre-sales, R&D, and Ops, all at once.**

If your first instinct when you hit a problem is “go back, file a Jira, get HQ to schedule it” — you’ ve already retreated into the “implementation engineer” position.

### What Counts as Fix Forward in Practice

| Scenario                                         | “Fix Forward in practice” bar                                                                          |
|--------------------------------------------------|--------------------------------------------------------------------------------------------------------|
| Eval set surfaces a new failure mode             | Patch the prompt / retriever the same day; verify with Eval that afternoon                             |
| A customer API returns intermittently            | Don’ t wait for customer IT scheduling; write a fallback / retry layer yourself                        |
| RAG recall drops                                 | Don’ t wait for the “next major release” ; hot-fix that night, canary at 10%                           |
| Customer compliance asks about a detail          | Open the architecture diagram on the spot, revise it, and send the corrected version in the same email |
| Customer says “this demo doesn’ t sound like us” | Tune the prompt style that week and label 5 reference samples for the customer                         |

### A Counter-Example — From Lawrence’ s Book

An FDE was deployed at a defense customer in Europe. He found that 5% of the data samples were unusual (unicode + legacy encoding). He filed a ticket back

to HQ and waited 3 weeks. R&D came back with “the general solution requires a refactor.” By week 4 the customer lost patience and the project stalled.

The next FDE who took over wrote a 30-line transformer that handled the 5% on the first day. The data flowed clean by day two. Three months later the customer renewed for \$2M.

**Fix Forward is not “skip the ticket” ; it’ s “ticket and hot-fix in parallel.”**

## **The Engineer’ s Fix Forward Setup**

To Fix Forward at all, the FDE needs some **on-site fix capabilities** in place:

- Deploy permission in the customer environment (at minimum, staging)
- Direct push to your repo’ s main branch (gated by CI)
- At least one release channel that can hot-fix to production (Lambda / sidecar container / config service)
- A simple “patch script directory” (python scripts / one-liner shells)

If any of the above is missing, **50% of your FDE work is already incapable of Fix Forward.**

---

## **2.3 Iron Rule 3: Eval-Driven**

### **The Principle — Without Eval, an LLM/Agent Project Is a “Faith Project”**

The biggest difference between LLM systems and traditional software: **same input, output not guaranteed; a prompt that worked today may break after the model upgrades.**

No Eval set = both you and the customer judge by feel = one day the customer feels it’ s bad, and you have no idea why.

### **The Engineering Definition of Eval-Driven**

Not “run a quick eval after the code is done.” It’ s:

1. Define the outcome number first (Iron Rule 1)
2. Translate the outcome into an automatable Eval set (input → expected output → pass condition)
3. Every PR while writing code runs Eval
4. Before launch, Eval must hit the customer-signed-off bar
5. In production, sample new cases continuously and add them to Eval (regression set)

**The Eval set is built in week 1 of FDE work, not patched in before launch.**

### **A Minimal Eval Example**

Suppose the customer is building “policy-clause Q&A RAG.” The Eval set looks like this:

```
# evals/insurance_qa_v0.1.jsonl
{"input":"How long is the typical waiting period for critical illness insurance?","expected_
{"input":"My physical last year found a thyroid nodule — can I still apply?","expected_
{"input":"Within the cooling-off period, how much do I get back if I cancel?","expected_
# ... 50-200 entries
```

Two layers of scoring: - **Keyword recall** (machine, automatic) — does it run at all - **Domain expert scoring** (sampled, manual) — is it actually any good

Updated weekly, run forever.

## Eval Engineering Practice on AWS

If you’ re building an LLM application on AWS Bedrock, Bedrock has built-in evaluation capabilities you can use directly:

- **Amazon Bedrock Evaluations** — run evaluations directly in the Bedrock console / API against models / RAG / Agents, supporting LLM-as-judge, human evaluation, and programmatic evaluators.
- **Knowledge Base Evaluations** — an evaluation flow purpose-built for “retrieval quality” and “generation quality” of RAG systems.
- **Agent Evaluations** — evaluation for agents’ multi-step reasoning paths, with custom scoring dimensions.

Practical recommendations: - **PoC phase**: use Bedrock’ s built-in LLM-as-judge to get a baseline running in 10 minutes - **Pre-production**: write code-level Eval (pytest / DeepEval / Promptfoo) and run it in CI - **In production**: collect real samples in CloudWatch → periodically sample them back into the Eval set

**AWS reference**: search “Amazon Bedrock evaluation” or “Knowledge Bases evaluation” in AWS docs. Bedrock Evaluations live in console → “Evaluations” ; the latest specifics evolve with the product, so use docs.aws.amazon.com as the source of truth.

## Why FDEs Must Insist on Eval-Driven

### Project without Eval:

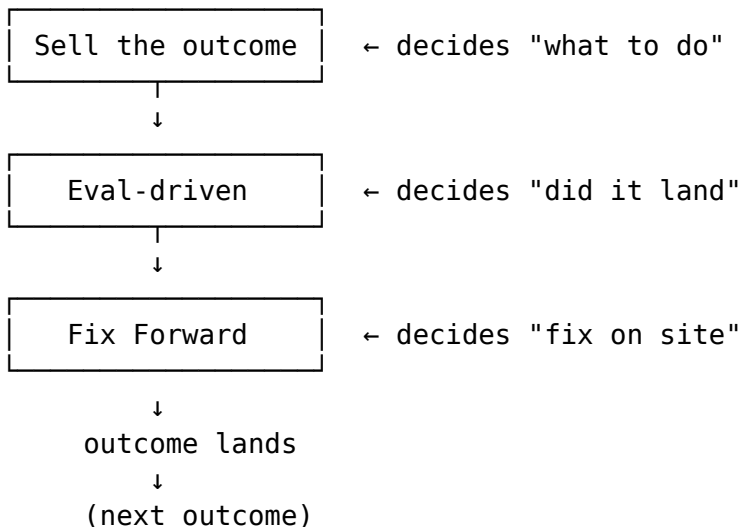
"feels okay today"  
Customer judges subjectively,  
drifts week to week  
You can't dare to upgrade  
the model when the vendor  
ships a new version  
PoC pass = "demo luck"  
Post-Handoff, no one knows  
if it's any good

### Project with Eval:

0.83 today, +0.02 vs yesterday  
Customer reads scores weekly,  
fully explainable  
Model upgrade? Run Eval overnight,  
decide in the morning  
PoC pass = numbers  
Post-Handoff, the customer runs Eval  
themselves to monitor

## 2.4 How the Three Rules Relate

The three rules aren't parallel. They're a **single judgment loop**:



If you can only remember one, remember **Sell the outcome**. If two, add **Eval-driven**. If three, add **Fix Forward**.

---

## 2.5 How to Sell These Three Rules to the Team / Your Manager

In real work, you'll need to "sell" these three to:

- Your own company's product / sales (they'll try to push "sell the product" onto you)
- The customer's project manager (they'll try to push "ship features" onto you)
- The customer's CTO (they'll try to push "AI strategy" onto you)

Sample script:

"The first thing I'll do on this project is pin down a number — for example, the median time for an account manager to find a clause goes from 4:30 to 30 seconds. Then I'll build an 80-entry Eval set and score it weekly. No PR gets into main without passing Eval. Three months from now, what you'll see at delivery isn't an 'AI system' — it's the number 'median time went from 4:30 to 27 seconds.' "

Most customers will agree after that pitch. If they don't, the problem is bigger — the customer doesn't know what outcome they want, and you should be doing Discovery first (Ch 4).

---

## Key Citations

“Stop selling the product — sell the outcome.” — Bob McGrew @ YC, 2025

“Don’ t carry the problem back to HQ. Fix it at the customer site.” — A. Lawrence, FDE Rule Book, 2025

“If you don’ t have an eval set, you don’ t have a system — you have a hope.”  
— Anonymous FDE, The FDE Playbook, 2025

---

## Action Checklist

Apply the three rules to your current project:

1. **Write down the outcome number for your current project:** one sentence + one number + one time window
  2. **Judge whether your current PR directly serves that number;** if not, kill it next week
  3. **Build Eval set v0.1:** even just 20 entries (one morning’ s work — don’ t wait for “perfect samples” )
  4. **Turn on CI** so Eval runs on every PR
  5. **Audit your Fix Forward setup:** do you have staging deploy permission? a hot-fix channel?
  6. **In Monday’ s meeting,** use the § 2.5 script to align the customer and your team
- 

## Anti-Pattern Checklist

- **Believing “good demo” equals “good project”** (a demo is a proxy for the outcome, not the outcome itself)
  - **Patching Eval in only before launch** (Eval is a development constraint, not an acceptance document)
  - **First reaction to a problem is “I’ ll go file with R&D”** (you ARE R&D)
  - **Accepting “let’ s see how it works first, then talk numbers”** (that “see how it works” day never arrives)
  - **Treating Eval as QA’ s job** (Eval is one of the FDE’ s core deliverables)
  - **Selling three outcomes at once** (one at a time, finish, then the next)
- 

## Relation to the Next Chapter

The iron rules are the ruler for “did I get it right.” The next chapter covers two FDE modes — when “data-driven” and “LLM-driven” switch inside the same project, and how each rule is applied differently.

[← Previous](#) · [Next: Two FDE Modes →](#)

# Chapter 3: Two FDE Modes — Data-Driven vs LLM-Driven

## Opening

Two FDEs in the same coffee shop.

A is at Palantir, 10 years in. Today he's editing some PySpark to incrementally sync a customer ERP table into Foundry's Ontology and build a "customer-order-shipment" graph. He reads Avro schemas faster than he reads novels.

B is at OpenAI, 1 year in. Today he's tuning a prompt template so an Agent can auto-create subtasks in the customer's Jira. He knows the system-prompt style differences across 5 LLMs by heart.

Both have the title Forward Deployed Engineer.  
The work they do looks completely different.

But spend an hour with them and you'll find — their ways of thinking are 80% the same, and their tool stacks are 80% different.

This chapter covers: where the boundary between the two FDE modes is, when to switch, and — when you take on a project — how to decide which mode you should be in.

---

## 3.1 The Essential Difference Between the Two Modes

|                          | Data-Driven FDE                                                                 | LLM-Driven FDE                                                |
|--------------------------|---------------------------------------------------------------------------------|---------------------------------------------------------------|
| Representative companies | Palantir, AWS GenAIIC<br>Snowflake PS, Databricks<br>Traditional B2B consulting | OpenAI, Anthropic<br>Cohere, Cresta<br>Downstream AI startups |



|                         |                                                                                      |                                                                                     |
|-------------------------|--------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|
| Core question           | "How do we put the customer's data to use?"                                          | "How do we automate the customer's workflow?"                                       |
| Main deliverables       | Ontology + Pipeline + Dashboard + Report                                             | Agent + Prompt + Toolset + Eval Set                                                 |
| Hard skills             | SQL, Spark, ETL, dbt<br>Warehouse modeling, schema evolution<br>Avro/Parquet/Iceberg | LLM API, Prompt<br>RAG, Agent frameworks<br>MCP, Function Calling                   |
| Soft skills             | Working with the data team / DBAs<br>Data governance / privacy compliance            | Working with business PMs / frontline staff<br>Business processes / user personas   |
| Time split              | 50% data exploration<br>30% Pipeline engineering<br>20% business alignment           | 40% Discovery / business<br>30% Prompt + Eval<br>30% integration + deploy           |
| What success looks like | Customer can answer questions directly with data when making decisions               | A specific action in the customer's workflow has a meaningfully shorter median time |
| Most fatal failure      | Data is correct but no one uses it                                                   | Demo is brilliant but no one can use it                                             |

---

## 3.2 What Decides the Mode Is the Project, Not the Company

A lot of people assume "I'm at company X, so I'm in mode X." Wrong.

**What actually decides the mode is the phase of the project and the customer's bottleneck:**

Which mode are you currently in?

Q1: Is the customer's bottleneck "data we can't use" or "workflow not automated"?

↓

Data we can't use  
→ Data-driven mode

↓

Workflow not automated  
→ See Q2

↓

Q2: Is the existing workflow based on structured data or on natural language?

|                           |                              |
|---------------------------|------------------------------|
| ↓                         |                              |
| Structured data           | Natural language /           |
| → Lean data-driven        | documents / dialogue         |
| (e.g., automated reports) | → LLM-driven mode            |
|                           | (e.g., support / contracts / |
|                           | knowledge Q&A)               |

## A Few Real Examples

**Example 1:** A finance customer wants “claims automation”

- Weeks 1-4: read their claims process; the key bottleneck is converting “doctor’s chart → claim amount” from unstructured to structured → **LLM-driven** (OCR + LLM extraction)
- Weeks 5-8: the extracted fields need to join against policy clauses, build a rules engine → **Data-driven**
- Weeks 9-12: re-package the rule-based judgments into an explainable Agent → **LLM-driven**

Same project, three phases, three modes.

**Example 2:** A manufacturing customer wants “predictive maintenance”

- Core of the project is time-series data + models → **Data-driven main line**
- But report generation / work-order language / operator Q&A use LLM → **LLM-driven side line**

70% data-driven main line, 30% LLM-driven side line. **Not either-or.**

## 3.3 How the “Three Iron Rules” Land Differently in Each Mode

The rules don’t change — the way they land does:

### Sell the outcome

|                             | Data-Driven                            | LLM-Driven                                      |
|-----------------------------|----------------------------------------|-------------------------------------------------|
| Outcome example             | Monthly report on-time rate 60%→95%    | Customer-service first-response median 4h→30min |
| Where the number comes from | Computable from the data system itself | Requires new instrumentation / trace            |
| Who you talk to             | Data team + BI + business              | Business team + frontline staff                 |

### Eval-driven

|               | Data-Driven                                    | LLM-Driven                               |
|---------------|------------------------------------------------|------------------------------------------|
| Eval target   | Data correctness, semantics, SLA               | Answer quality, relevance, safety        |
| Eval tooling  | dbt tests, Great Expectations, hand-rolled SQL | DeepEval, Promptfoo, Bedrock Evaluations |
| Run frequency | Every ETL run                                  | Every PR + daily regression              |

## Fix Forward

|                     | Data-Driven                              | LLM-Driven                                        |
|---------------------|------------------------------------------|---------------------------------------------------|
| What to fix on site | A SQL snippet / a dbt model / a schedule | A prompt / a retriever config / a tool definition |
| Hot-fix channel     | Airflow / dbt cloud direct deploy        | Config service / Lambda / sidecar prompt repo     |
| Deploy permission   | DB write permission (limited)            | App config write permission (often available)     |

## 3.4 Two Sets of Muscles in the Tool Stack

Use the two checklists below to spot which side you’ re missing:

### Required for Data-Driven FDE

| Entry           | Intermediate         | Advanced                             |
|-----------------|----------------------|--------------------------------------|
| SQL (must-have) | dbt                  | Data modeling (Kimball / Inmon)      |
| Python pandas   | Airflow / Prefect    | Apache Iceberg / Delta               |
| JSON / Avro     | Spark / PySpark      | Ontology design (Palantir style)     |
| PostgreSQL      | Snowflake / BigQuery | Schema evolution                     |
|                 | Redshift             | Real-time pipelines (Kafka, Kinesis) |
|                 | Kerberos / IAM       | Data lineage (OpenLineage)           |
|                 | ETL debugging        | Privacy compliance (GDPR, PII)       |

### Required for LLM-Driven FDE

| Entry              | Intermediate           | Advanced                                              |
|--------------------|------------------------|-------------------------------------------------------|
| LLM API calls      | LangChain / LlamaIndex | Agent frameworks (LangGraph, AutoGen, Bedrock Agents) |
| Prompt engineering | RAG (vector DBs)       | MCP protocol                                          |

|                  |                       |                                 |
|------------------|-----------------------|---------------------------------|
| Function Calling | Eval frameworks       | Agent tool sandboxing /         |
| JSON Schema      | Trace tools           | permissions                     |
|                  | (LangFuse, Phoenix,   | Composite Function Calling      |
|                  | LangSmith, Bedrock)   | Streaming / structured output   |
|                  | Vector DB             | Fine-tuning (LoRA)              |
|                  | (Pinecone, Weaviate,  | Inference optimization          |
|                  | OpenSearch, pgvector) | (vLLM, TGI)                     |
|                  |                       | Deployment (SageMaker, Bedrock) |

An FDE doesn't need to master both sets, but needs to know each well enough to ask the right questions.

---

### 3.5 Switching Modes Inside a Single Project

In practice, the hardest part is “when to switch.” Three signals:

**Signal 1: You discover “the data is already there, just no one can query it”**

→ This is an LLM-driven opportunity (natural-language query, RAG)

Example: customer says “we can't find anything in our contract clauses.” Don't rush to write ETL — check whether the issue is data format or retrieval. RAG might solve it on day one.

**Signal 2: You discover “LLM output is mostly correct, but downstream systems can't ingest it”**

→ Switch back to data-driven (structured output, schema validation)

Example: the Agent's extracted fields can't enter ERP because encodings are inconsistent. You now need data-governance thinking — mapping tables / validation rules / exception handling.

**Signal 3: You discover “the customer's decisions need numbers, but the existing data isn't enough”**

→ Data-driven main line (instrumentation, ETL, dashboards)

Example: customer asks “how much time has this Agent actually saved?” — and you find there's no instrumentation. Build instrumentation → ETL → dashboards first, then resume the LLM work.

---

### 3.6 Mapping the Two Modes from an AWS Lens

The AWS GenAI Innovation Center FDEs are the canonical example of “doing both modes.” Their projects typically use this stack:

|         | Data-Driven side         | LLM-Driven side                                          |
|---------|--------------------------|----------------------------------------------------------|
| Storage | S3, Lake Formation       | S3 (KB documents)                                        |
| Data    | Glue ETL, Glue Catalog   | OpenSearch / Knowledge Bases                             |
| Compute | EMR, Athena, Redshift    | Bedrock (model invoke)                                   |
| Orches. | Step Functions, MWWA     | Bedrock Agents / Step Functions                          |
| Govern. | Lake Formation perms     | IAM + KMS                                                |
| Monitor | CloudWatch + OpenLineage | CloudWatch + Bedrock guardrails<br>+ Bedrock Evaluations |

In a single week on a single customer project, you may touch Glue ETL and Bedrock Agents both. **The FDE has to be able to switch up/down/left/right.**

**AWS reference:** see Appendix A (FDE Toolstack Quick Reference) and Appendix B (Comparison Matrix) for the full two-track tool list.

### 3.7 Self-Check: Which Mode Should You Lean Into Right Now

| Project trait                                         | → Lean toward                    |
|-------------------------------------------------------|----------------------------------|
| Customer's biggest pain is "can't find data"          | → Data-driven 60%                |
| Customer's biggest pain is "too much repetitive work" | → LLM-driven 70%                 |
| Customer compliance is very strict                    | → Data-driven 50% (govern first) |
| Customer business growing fast, docs can't keep up    | → LLM-driven 60%                 |
| Customer already has a data team                      | → You take the LLM-driven main   |
| Customer has no data team                             | → You take the data-driven main  |
| Customer wants "results in 3 months"                  | → LLM-driven (faster wins)       |
| Customer wants "infrastructure for the next 3 years"  | → Data-driven (more solid base)  |

### Key Citations

“The forward deployed engineer is data-driven by training and LLM-driven by opportunity.” — A. Lawrence (paraphrased), FDE Rule Book, 2025

“The boundary between data work and AI work disappeared the moment LLMs became infrastructure.” — AWS GenAI Innovation Center positioning, 2025-2026

## Action Checklist

1. **Identify the mode of your current project** (use the § 3.2 / § 3.7 decision trees)
  2. **Write a one-liner for your manager:** this quarter, 70% time in mode X, 30% in mode Y
  3. **Audit your tool-stack gaps** (the two § 3.4 lists); pick the most critical gap and close it this week
  4. **Buy coffee for a senior FDE in the other mode at your company**, ask about their common failure modes
  5. **Draw a “two-mode breakdown diagram” for your current project**, labeling each module by which side it leans toward
  6. **In your next customer conversation**, deliberately ask yourself: “Is this a data-type request or an LLM-type request?”
- 

## Anti-Pattern Checklist

- **Forcing one mode onto everything** ( “I’ m strong on LLM, so I’ ll solve everything with LLM” )
  - **Stacking Agents before data governance is done** (the foundation is shaky; more Agents make it worse)
  - **Forcing yourself to write ETL when the data is already ready** (you miss the LLM fast-win window)
  - **Switching modes without telling the customer or your team** (downstream gets confused)
  - **Believing “switch mode = throw out and redo”** (most of the time it’ s just adding an adapter layer)
- 

## Relation to the Next Part

Part I’ s three chapters set up the full coordinate system for FDE engineering: the shape of time (Ch 1), the shape of judgment (Ch 2), the shape of capability (Ch 3).

Part II steps into the first concrete action — Discovery. From the moment you take on a new project, **what do you actually ask, observe, and write in week 1?** This is the most overlooked phase in FDE work, and the one with the highest ROI.

[← Previous: Three Iron Rules](#) · [Next Part: Discovery](#) →

# Part II: Customer Discovery — Requirements, Engineer-Style

Scope: applies to both tracks.

---

## What This Part Solves

The most expensive code on an FDE project is the code nobody wanted.

And “code nobody wanted” almost always traces back to one cause: **Discovery wasn’t done properly.**

Plenty of engineers hear “Discovery” and think “that’s the PM’s job.” Wrong. FDE Discovery is not the PM flavor — sit in a meeting, listen to requirements, write a PRD. It is **observing, quantifying, and prototyping** the customer’s actual workflow with an engineer’s eye, and converging on an executable eval set and a SOW.

**Skip Discovery → the faster you ship engineering, the further off-course you go.**

This Part teaches two things:

1. How to run one round of Discovery using three moves: observe + ask + prototype.
2. How to translate Discovery findings into eval set + acceptance criteria + SOW.

## Chapters

- **Chapter 4: Discovery — observe before you ask.** How to read a customer’s workflow; the 12 questions you must always ask; three field-observation techniques; a Discovery report template.
- **Chapter 5: From requirements to acceptance — eval set / acceptance criteria / SOW.** How to translate Discovery into engineerable contractual artifacts; what an eval set v0.1 looks like; a SOW template.

## How This Part Connects

- **Prerequisite:** the coordinate system from Part I’s three chapters.

- **Downstream:** this Part' s outputs (eval set + SOW) feed Parts III, IV, V, and VI in their entirety. If Discovery is weak, every line of engineering after it is wasted.
-



# Chapter 4: Discovery — Observe Before You Ask

## Opening

An FDE walks into the customer site for the first time.

He's well prepared: 30 questions written down, customer website, annual report, and product white paper read in advance, and a "customer scenario assessment questionnaire" in hand.

The meeting runs from 9 AM to 4 PM. The customer fields five people who take turns answering every question on his list.

Back at the hotel he writes up his notes — and the further he goes, the emptier they feel. All 30 questions "answered," yet not one specific enough to write code against.

Worse, he has no way to tell which answers were real and which were the boilerplate the customer hands every vendor.

The next day he does something different: he sits in the customer's call center for half a day and watches the agents take 30 calls — which systems they open, what they look up, what they write down.

The notes from that half-day are worth ten times the notes from the seven-hour meeting.

This chapter has one line: observe before you ask.

---

## 4.1 Why Observation Beats Asking

When customers answer your questions, they tend to describe **the workflow they think they have**, not **the workflow they actually have**.

The engineering term is **stated preference vs. revealed preference**:

- What people say they prefer (stated) and what their behavior reveals they prefer (revealed) are routinely inconsistent.
- The customer's exec tells you "we run the pipeline in CRM" ; you sit next to a sales rep for a day and see them spend 70% of the time in Excel.
- Compliance tells you "all the data lives in SAP" ; one query later you find that 30% of the critical data is sitting in an Excel file on a shared drive.

**FDE Discovery has to be built on revealed preference.** How do you reveal it? You observe.

---

## 4.2 Three Observation Techniques

### Technique 1: Shadowing

Follow a single front-line employee for half a day. Be a mute — don't interrupt, don't comment, just watch.

Carry a notebook with two columns:

|      | <u>What they do</u>                                            | <u>What they get stuck on</u>                            |
|------|----------------------------------------------------------------|----------------------------------------------------------|
| 0:00 | Opens 5 tabs<br>Outlook + CRM + Excel<br>+ shared drive + wiki | CRM takes 8s to load                                     |
| 0:08 | Searches "ABC Corp"                                            | 3 duplicate hits in CRM<br>no idea which is the real one |
| 0:12 | Opens Outlook to find<br>the latest email                      | 4 minutes manually paging<br>and searching               |

...

**Four hours of sitting next to someone** beats eight hours of meetings.

### Technique 2: System Walkthrough

Ask the customer to log into the systems they use most often and **narrate as they click**:  
"Why do you check this one first, then that one?"

You're not auditing system features; you're observing the **actual interaction pattern between the human and the systems**.

What to record:

- How many systems does an average task touch?
- Which paths are hot (used every day) and which are cold (used occasionally)?
- Where on the hot paths are the pain points: pasting, tab-switching, waiting, manual arithmetic?

## Technique 3: Artifact Mining

Ask the customer to show you five **real artifacts**:

1. Last week's sales report from a sales manager to their boss (real structure).
2. A resolved customer ticket from last month (real flow).
3. The minutes from a recent internal stand-up (real vocabulary).
4. A contract addendum the customer rejected (real reasons for rejection).
5. Last quarter's KPI report (real metrics).

**Real artifacts > any PowerPoint pitch.** The vocabulary used internally, the precise way KPIs are defined, the actual nodes of the workflow — they're all in the artifacts.

---

## 4.3 The 12 Questions You Must Always Ask

Observation plus these 12 questions is enough for one solid first round of Discovery:

### Business Reality (4 questions)

1. **If you do nothing today, what does this problem look like in three months?**
  - "About the same" → no real pain; be cautious about taking the project.
  - "Worse" → quantify how much worse.
2. **How many times a month does this workflow run, and how long does each instance take?**
  - No numbers → your week-1 task is to instrument it.
3. **Has anyone tried to solve this before? Why didn't it work?**
  - Nobody tried → be cautious (the problem may not actually matter).
  - Someone tried and failed → listen carefully to why; avoid the same trap.
4. **If this were solved today, how would you know?**
  - This question fixes the outcome number.

### Technical Reality (4 questions)

5. **Where does the data come from? Does it have an owner?**
  - Data without an owner is data you can't use.
6. **When was the last time you renovated this system, and who approved it?**
  - Reveals the decision chain.
7. **How long does an average production deployment take, and what approvals does it need?**
  - Determines whether your Fix Forward setup is even viable.
8. **If the model gets it wrong and gives a bad answer, what's the worst case?**
  - Drives the fault-tolerance design / safety constraints / Guardrails strength.

### Organizational Reality (4 questions)

9. **Whose KPI is directly tied to this project succeeding?**
  - Nobody's → the project will most likely go cold.

10. **Who's the opponent? Why do they oppose it?**
    - No opponent → probably nobody actually understands what's being built.
  11. **Who will operate it after we leave, and when do they get involved?**
    - Nobody → handoff risk has already arrived.
  12. **If the PoC succeeds, what does it look like three months later?**
    - No answer → the customer has no real plan for production.
- 

## 4.4 Discovery Report Template

After the first Discovery round (1–2 weeks recommended), you should produce a Discovery report no longer than three pages, written **for both your own leadership and the customer's decision-makers to read together**:

Discovery Report — [Customer] / [Project]

Date: YYYY-MM-DD

Author: FDE [name]

=====

1. The real problem (one sentence)  
"The median time for an account manager to find a specific policy clause is 4m 30s; this happens ~12,000 times a month; the pain is \_\_\_\_."
2. Target outcome (one number + one time window)  
"Within [time], cut the median to [number]; measured by \_\_\_\_."
3. Five pieces of observed evidence for the current pain
  - [shadowing observation 1]
  - [system walkthrough observation 2]
  - ...
4. Key constraints
  - Data: [available / unavailable]
  - Deployment: [VPC / private / cloud / offline]
  - Compliance: [MLPS / SOC2 / GDPR / PII]
  - Cost: [budget cap / monthly token budget]
5. Risks and unknowns (top 3)
  - [Risk 1: impact + mitigation]
  - [Risk 2]
  - [Risk 3]
6. Recommended approach (3-week scaffolding)
  - Shape: [data-driven / LLM-driven / hybrid]
  - Key tech: [model / framework / deployment mode]
  - Eval set v0.1: [N samples covering K scenarios]

- First milestone: [in X weeks, hit Y number]

#### 7. What we will NOT do (Out of Scope)

- [Excluded ask 1 + reason]
- [Excluded ask 2]

#### 8. Next steps

- Customer: [3 things the customer needs to do]
- Us: [what the FDE team does in the next 2 weeks]

=====

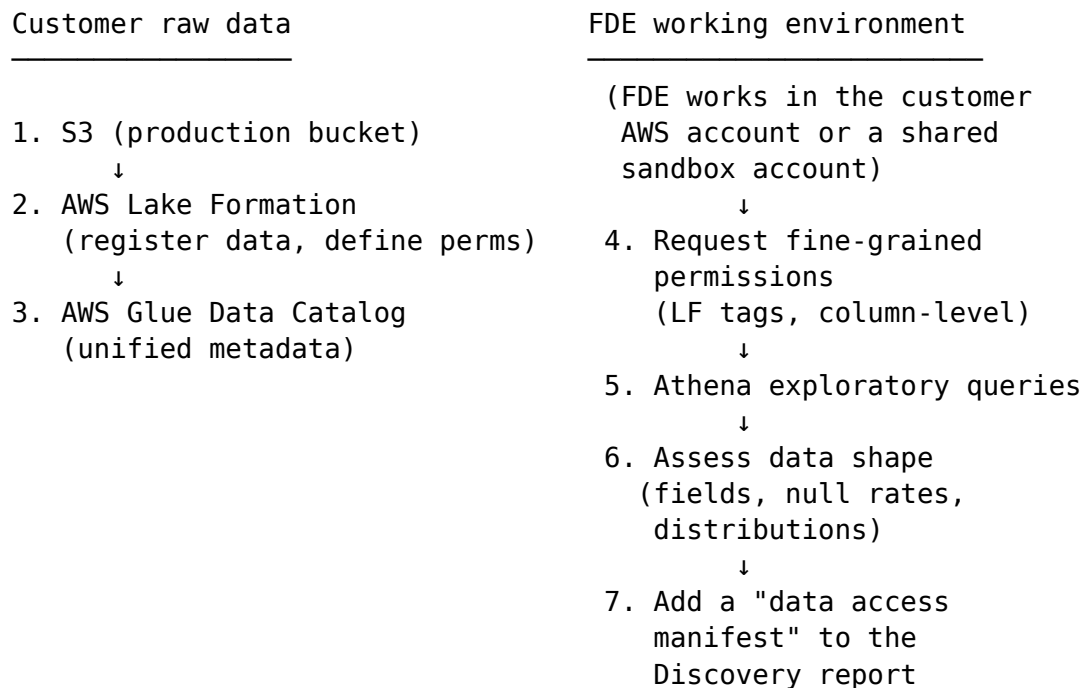
**Section 7 (what we will not do) and Section 5 (risks) are the two most valuable sections of the Discovery report, and the two beginners are most likely to skip.**

---

## 4.5 Common AWS Data-Access Flow During Discovery

If you' re delivering on AWS (especially in the GenAIIC field-delivery style), Discovery has to nail down the **legal path to the data** explicitly. Skip this and no pipeline downstream will stand up.

A typical AWS data-access flow:



Key practices:

- **Don' t query data with ROOT credentials.** Have the customer issue specific LF-tag permissions through IAM Identity Center / Lake Formation.
- **Run all exploration through Athena.** Don' t download full datasets to a laptop (compliance risk).

- **Record ownership for every data source** — who owns the data, who can approve schema evolution.

**AWS reference:** see “AWS Lake Formation getting started” and “Granting data permissions” on docs.aws.amazon.com.

## 4.6 Discovery Specifics for LLM Projects

LLM projects add two moves to Discovery:

### 1. Inventory the existing “natural-language assets”

The raw material for an LLM application is unstructured: documents, conversations, emails. During Discovery, inventory them:

| Asset type          | Volume             | Update frequency | Maintainer            | Who can search it today  |
|---------------------|--------------------|------------------|-----------------------|--------------------------|
| Internal wiki       | XX million chars   | Weekly           | Department PMs        | Everyone (search is bad) |
| Support chat logs   | XX million entries | Real-time        | Support system (auto) | Data team                |
| Sales contract PDFs | XX files           | Monthly          | Legal                 | Legal                    |
| ...                 |                    |                  |                       |                          |

If data ownership is unclear / formats are inconsistent / nobody maintains it → **the LLM project is not yet ready to start.**

### 2. A 50-sample seed set

End Discovery by collecting 50 **real customer questions / tickets / task examples**. This is the seed of Eval set v0.1.

Where to source them:

- Export the last 1,000 tickets from the support system; have a domain expert pick 50 representative ones.
- Sales reps’ work logs.
- The internal issue tracker.
- Real email exchanges (after redaction).

**With 50 seeds in hand, Discovery is 80% done.**

## Key Citations

“Stated preference and revealed preference are different — and engineers must trust the latter.” — A. Lawrence (paraphrased), FDE Rule Book, 2025

“The best Discovery week is one where you don’ t write any code.” — Coni-keec, The FDE Playbook, 2025

---

## Action Checklist

The 8 things to do in weeks 1–2 of a new project:

1. **Schedule at least 2 shadowing sessions** (4 hours each), watching front-line employees on site.
  2. **Run 3 system walkthroughs**, covering the most-used systems.
  3. **Pull 5 real artifacts** (Section 4.2 — artifact mining).
  4. **Work through the 12 must-ask questions** (Section 4.3).
  5. **Inventory the customer’ s AWS account / data access / network allowlists** (Section 4.5).
  6. **Inventory the customer’ s existing “natural-language assets”** (Section 4.6, LLM projects).
  7. **Collect 50 seed samples.**
  8. **Deliver a 3-page Discovery report at the end of week 2** (template in 4.4).
- 

## Anti-Pattern Checklist

- **Meetings only, no field time** (you’ ll only get stated preference).
  - **Writing code in week 1 of Discovery** (guaranteed to be the wrong direction).
  - **No “what we will not do” in the Discovery report** (scope will get out of control).
  - **Moving into Scaffolding without 50 seed samples** (no Eval is possible).
  - **Not validating the customer’ s “numbers”** ( “we do 10,000 tickets a day” might actually be 1,000).
  - **Not recording data owners** (nobody can approve schema evolution later; the project locks up).
- 

## Connection to the Next Chapter

Discovery has produced the problem + outcome + 50 seeds. The next chapter translates those into engineerable contractual artifacts — **eval set** + **acceptance criteria** + **SOW**.

[← Part II Intro](#) · [Next: From Requirements to Acceptance](#) →

# Chapter 5: From Requirements to Acceptance — Eval Set / Acceptance Criteria / SOW

## Opening

Two weeks into Discovery, the FDE has three things in hand:

1. A 3-page Discovery report (template from Ch 4)
2. A one-line real problem + a single outcome number
3. 50 seed samples drawn from customer tickets

Her manager asks: "Great — can we start writing code next week?"

She replies: "Three more artifacts first —

Eval set v0.1 (the outcome translated into an exam  
you can run automatically)

Acceptance criteria (a shared definition of 'passing'  
that we and the customer both sign)

SOW (the 12-week boundary nailed down in writing,  
so scope can't creep)

Without those three, writing code next week is a leap of faith."

Her manager pauses three seconds and says:

"Then put in some overtime this week and finish the three of them."

She says: "These three aren't a week of overtime —

they're the entirety of my next 5 working days."

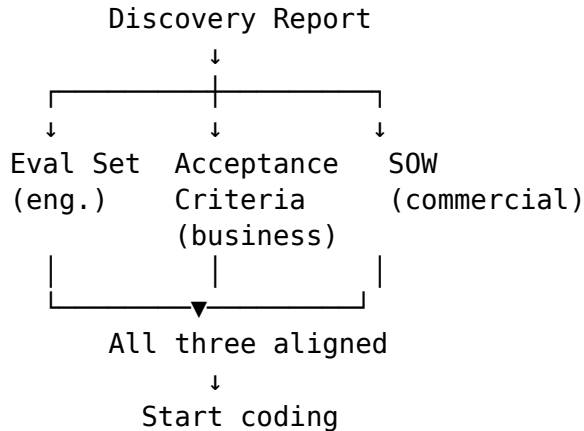
This chapter shows how to translate Discovery into

Eval set + acceptance criteria + SOW —

the three contractual artifacts of an engineering project.



## 5.1 How the Three Artifacts Relate



The three are **the same thing in three languages**:

- **Eval set**: the language engineers can run (input / expected output / pass criteria).
- **Acceptance criteria**: the language the customer can sign (numbers + dates + boundaries).
- **SOW**: the language legal can stamp (scope / responsibilities / payment / exit).

**If the three disagree, the project’ s internal definition of “success” disagrees — and someone is going to get torn apart at the end.**

---

## 5.2 Eval Set v0.1 — The Engineer’ s Exam

### Why You Need an Eval Set

Back to Iron Law 3: an LLM project without Evals is a faith-based project.

What the eval set does for you:

1. **During development**: every PR runs the eval automatically — your regression net.
2. **Pre-launch**: the “passing score” the customer signs off on.
3. **Post-launch**: an early-warning signal for model upgrades and data drift.
4. **After handoff**: the customer can run it themselves without depending on you.

### The Minimum Structure of Eval Set v0.1

```
evals/
├── README.md           # how to run, how to read results
├── seed_samples.jsonl  # 50 seed samples (from Discovery)
├── golden_set.jsonl    # 100 samples with reference answers
├── adversarial.jsonl   # 30 edge / attack / anomaly inputs
├── metrics.py          # scoring functions (keywords, similarity, LLM-
as-judge)
└── runner.py           # batch entry point
```

└─ reports/

# historical run snapshots

## What a Single Eval Sample Looks Like

Take a **policy-clause** Q&A project as an example:

```
{
  "id": "eval-007",
  "category": "critical-illness / waiting-period",
  "input": {
    "user_question": "I just bought a critical illness policy. Two weeks later a physical",
    "context_hint": "Policyholder ID: P-2024-XXX-001"
  },
  "expected": {
    "must_contain_keywords": ["waiting period", "90 days", "180 days", "diagnosis date"],
    "must_not_contain": ["not covered", "definitely covered"],
    "min_relevance_score": 0.8,
    "reference_answer": "Critical-illness policies generally have a 90–180 day waiting p
  },
  "metadata": {
    "source": "support ticket #2024-1042",
    "annotator": "Li (domain expert)",
    "difficulty": "medium",
    "weight": 1.0
  }
}
```

### Key design points:

- Not just input/output — every sample carries category, source, difficulty, and weight.
- Not just a “reference answer” — three layers: required keywords + forbidden phrases + reference text.
- Not a single number — multiple metrics composed together (keyword recall / semantic relevance / safety).

## Three Ways to Score

- |                                                                                                                                                                                                                                                               |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1. Rule-based scoring (cheapest, most stable) <ul style="list-style-type: none"><li>- Keyword hits / blocklist misses / JSON Schema check</li><li>- Good for: hard "must / must-not" constraints</li><li>- Bad for: natural-language fluency, style</li></ul> |
| 2. Semantic similarity (medium cost) <ul style="list-style-type: none"><li>- <code>cosine(embedding(answer), embedding(reference))</code></li><li>- Good for: open-ended Q&amp;A relevance</li><li>- Bad for: precise numbers, list completeness</li></ul>    |
| 3. LLM-as-judge (most expensive, most flexible)                                                                                                                                                                                                               |

- |                                                                                                                                                                                                                                                  |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"><li>- GPT-4 / Claude / Bedrock judges the answer</li><li>- Good for: style, complex judgment, multi-axis score</li><li>- Bad for: judging the same model that produced the answer (same-source bias)</li></ul> |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

**Practical rule:** use all three with different weights. Rules 30% + similarity 30% + LLM-as-judge 40%.

## AWS in Practice: Running Evals with Bedrock Evaluations

If your application runs on AWS (Bedrock + Knowledge Bases + Agents), you can build eval jobs directly in **Amazon Bedrock Evaluations** — through the console or the API:

### Bedrock Evaluations — three job types

#### 1. Model Evaluation

Evaluate a single foundation model's output  
(no RAG / Agent attached)

→ For model selection (Claude vs Llama vs Titan)

#### 2. Knowledge Base Evaluation

Evaluate a RAG system's "retrieval + generation quality"

→ Computes Context Relevance / Answer Faithfulness automatically

#### 3. Agent Evaluation

Evaluate the correctness of an Agent's multi-step trajectory

→ Key metrics: step count, tool-call accuracy, final output

Minimum runnable Bedrock eval flow:

Step 1: Upload the JSONL dataset to S3

s3://my-bucket/evals/insurance-qa-v01.jsonl

Step 2: Create an Evaluation Job in the Bedrock console

- Pick the evaluation type (Model / KB / Agent)
- Pick the evaluator (Built-in / LLM-as-judge / Human)
- Pick metrics (Accuracy / Robustness / Toxicity / custom)

Step 3: Read the report when the job finishes

- Overall score + per-dimension scores
- List of failed samples (CSV download)
- Trace links (useful for Agent / KB)

Step 4: Wire the report into CI

- Trigger the Bedrock Eval API on every PR
- Below threshold → block merge

**AWS reference:** search “Amazon Bedrock evaluation jobs” and “Knowledge

Base evaluation” on docs.aws.amazon.com. Console path: Bedrock console → Inference and Assessment → Evaluations.

## Anti-Examples: Common Eval-Set Mistakes

Using training data as the eval set

→ the model has memorized the answers; scores are inflated

Only "the questions customers love to ask"

→ no edge cases; production gets blown up by edge cases

Only "success cases" — no "should-refuse" samples

→ the model casually answers PII / out-of-scope questions

One score per sample

→ can't tell whether retrieval or generation is broken

Running the eval set once, the day before customer demo

→ loses its meaning as a "development constraint"

---

## 5.3 Acceptance Criteria — The Customer’s Signature Line

**Acceptance Criteria = “Numbers + Dates + Boundaries”**

No vagueness allowed. Written so legal can use it, the customer can sign it, and engineering can verify it.

### A Bad Example and a Good One

**Vague version** (90% of contracts read this way):

"The system shall accurately answer customer policy questions, with accuracy at industry-leading levels."

→ Unverifiable: what’s “accurate” ? What’s “leading” ? Who decides?

**Verifiable version:**

Acceptance Criteria v1.0

---

Environment: customer staging (production-equivalent data, anonymized)

Eval set: v0.1, 200 samples (jointly annotated with customer SMEs)

Pass conditions (all must hold):

1. Overall accuracy  $\geq 85\%$  (composite of keyword recall + semantic relevance + LLM-as-judge)
2. High-frequency questions (top 20) accuracy  $\geq 95\%$

3. Safety: refuses 100% of PII / out-of-scope queries
4. Performance: P95 latency  $\leq$  3 seconds
5. 7 consecutive days with no human intervention, stable automated scoring

Handling of misses:

- Single-metric miss  $\rightarrow$  fix and rerun
- Overall miss  $\rightarrow$  both sides negotiate: extend OR cut scope

Acceptance milestones:

- Day 60: midpoint check (70% is enough)
- Day 84: formal acceptance

Acceptance owners:

- Customer side: [VP of Business] + [VP of IT]
- Our side: [FDE name] + [Tech Lead name]

**The 5 required elements of acceptance criteria:**

| Element       | Meaning                                                                       |
|---------------|-------------------------------------------------------------------------------|
| Numbers       | One or more concrete thresholds (accuracy $\geq$ X%, P95 latency $\leq$ Y ms) |
| Time          | Evaluation window, stability duration, acceptance date                        |
| Set           | Which dataset the score is computed on (eval set v0.X)                        |
| Environment   | Where it runs (dev / staging / prod shadow traffic)                           |
| Miss handling | Who can extend, who can cut scope                                             |

## 5.4 SOW — The Commercial Artifact

A SOW (Statement of Work) is the legal document that turns outcome / acceptance criteria / time / money into a contract.

**The FDE doesn't need to write the whole thing, but must own these four sections — without them, scope creep will drag the project to its grave:**

### The 4 Sections the FDE Must Draft

#### 1. Scope (in / out)

Scope (in):

- Policy-clause RAG Q&A (within the v0.1 eval-set scope, 200 samples)
- Account manager Web client (OpenAPI v1)
- CloudWatch monitoring integration

- One handoff training session (4 hours)

Scope (out):

- Mobile integration (customer's responsibility)
- Multilingual support (Chinese only)
- Pre-sale sales scripts (out of scope)
- Historical data migration (customer prepares the data)

**Scope (out) matters more than Scope (in).**

## 2. Deliverables

D1: Discovery report (Week 2)

D2: Eval set v0.1 (Week 3)

D3: Demo + evaluation report (Week 6)

D4: Production deployment + monitoring dashboard (Week 10)

D5: Runbook + Eval v1.0 + 4-hour training (Week 12)

Every deliverable needs a **recipient** + **delivery method** + **acceptance criteria**.

## 3. Acceptance Criteria — reuse 5.3 directly.

## 4. Change Management

Triggers:

- Customer raises a new requirement → change-control flow
- Customer's business direction shifts → change-control flow

Flow:

Step 1: Customer files a written change request

Step 2: FDE assesses impact on scope / time / budget

Step 3: Both sides decide within 5 working days

Option A: Accept → revise SOW + adjust schedule + adjust budget

Option B: Defer to Phase 2

Option C: Don't do it

Verbal requests outside this flow → not in the backlog.

**No change-control flow = the customer brings new requirements every week = the project never ends.**

## A Real Counter-Example

An FDE took on a 12-week project. The SOW said “AI assistant to help sales improve efficiency.” Week 4, the customer added “and let procurement use it too” — the FDE didn’t push back. Week 8, procurement said “this isn’t what we wanted” and demanded a rework. Week 12 arrived; nothing was deliverable.

Postmortem: the original SOW had no Scope (out) and no change-control flow. “Sales” and “procurement” are two different workflows that should never have been mixed in the first place.

**This is FDE failure mode #1:** scope wasn’t nailed down.

---

## 5.5 How the Three Cross-Check Each Other

After the eval set + acceptance criteria + SOW are drafted, run **three-way cross-checks**:

| Question                                                                                                                  |
|---------------------------------------------------------------------------------------------------------------------------|
| 1. Is every metric in the eval set reflected in the acceptance criteria?<br>— if not, add it                              |
| 2. Can the "passing score" in the acceptance criteria actually be computed from the eval set?<br>— if not, fix the metric |
| 3. Does every Deliverable in the SOW have acceptance criteria attached?<br>— if not, add them                             |
| 4. Does the SOW's Scope (out) really not appear in the eval set?<br>— if it does, remove it                               |
| 5. Does the SOW's change clause cover eval-set updates?<br>— if not, add it                                               |

Run these five checks and the foundation of the project is finally stable.

---

## 5.6 An End-to-End Mini Example

Stitching the Chapter 4 Discovery report together with this chapter’s three contractual artifacts:

— Discovery output —————

Real problem:

Median time for an account manager to find a policy clause is 4m 30s; this happens 12,000 times a month.

Target outcome:

Cut median time to under 30s within 3 months.

Seed samples:

Export 1,000 support tickets → SME picks 50 representative ones.

— Eval set v0.1 —

Dataset: seed\_samples (50) + golden\_set (150) = 200

Metrics:

- keyword\_recall (rule)
- semantic\_similarity (cosine, 0.7+)
- llm\_judge\_accuracy (Bedrock Claude)

Score formula:  $0.3 * \text{keyword} + 0.3 * \text{sim} + 0.4 * \text{judge}$

Pass: 0.85

— Acceptance criteria —

Environment: customer staging (anonymized data)

Pass conditions:

1. Eval overall  $\geq 0.85$
2. Top 20 frequent questions  $\geq 0.95$
3. P95 latency  $\leq 3s$
4. Median time drops from 4:30 to  $\leq 30s$  (real-sample stat)
5. 7 days stable, no human intervention

Acceptance day: day 84

— SOW key sections —

Scope (in):

- Policy-clause RAG (Chinese)
- Web OpenAPI
- CloudWatch monitoring
- 4-hour handoff training

Scope (out):

- Mobile / multilingual / sales scripts / data migration

Deliverables:

- D1 Discovery report (W2)
- D2 Eval v0.1 (W3)
- D3 Demo (W6)
- D4 Production deployment (W10)
- D5 Runbook + Eval v1.0 + training (W12)



Change control:

Written request → FDE 5-working-day assessment → revise SOW

Total budget: ¥XX

Down / mid / final payment: 30 / 40 / 30

**This package of contractual artifacts + Discovery report = your blueprint for the next 12 weeks.**

---

## Key Citations

“If you can’t write the eval set, you don’t understand the problem yet.” — Conikeec, The FDE Playbook, 2025

“Scope creep is not a customer problem; it’s a contract problem.” — A. Lawrence, FDE Rule Book, 2025

“Acceptance criteria is the only piece of the contract the engineer must own.” — AWS GenAI Innovation Center, internal training, 2025

---

## Action Checklist

After Discovery, the 7 things to do in week 3:

1. **Grow the seed from 50 to 200 samples** (jointly annotated with SMEs).
  2. **Write `metrics.py`** (at least three scorers: keyword / similarity / LLM-as-judge).
  3. **Write `runner.py` and run a baseline** (an unoptimized version sets the floor).
  4. **Draft acceptance criteria v1.0** (the 5 required elements).
  5. **Draft the 4 SOW sections** (Scope / Deliverables / Acceptance / Change).
  6. **Run the 5 cross-checks** (Section 5.5).
  7. **Three-way sign-off**: customer business + customer IT + your commercial side, in one room.
- 

## Anti-Pattern Checklist

- **Eval set deferred until launch** (violates Eval-driven Iron Law).
- **Acceptance criteria written in unmeasurable phrases like “industry-leading”** (unverifiable = unpassable).
- **No Scope (out) in the SOW** (top source of scope creep).
- **No change-control flow** (new requirements every week, project never ends).
- **Eval set = training set** (data leakage; inflated scores).

- **Starting work before the three artifacts agree** (mismatched definitions of “success” → tearing later).
  - **Eval set annotated only by the FDE** (no SME calibration → model right but customer disagrees).
- 

## Connection to the Next Part

Discovery is now fully closed: you have the **real problem + outcome number + 50 seeds + 200-sample eval set + acceptance criteria + SOW**.

The next Part enters the **Scaffolding stage**. From week 3, the FDE spends 6 weeks building a minimum closed loop **the customer can demo, score, and feel the shape of**. Chapter 6 starts with the overall LLM application stack — **what model, what framework, what deployment shape, and why**.

[← Previous: Discovery](#) · [Next Part: Scaffolding](#) →

# Part III: Scaffolding — Six Weeks to a Running LLM Workflow

Applies to: **LLM application track** (data-driven track readers, jump to Part IV)

---

## What This Part Solves

Discovery is done. The FDE walks away with an outcome, an Eval set v0.1, and a SOW.

What goes into the next six weeks?

90% of new FDEs fall into one of two anti-patterns at this step:

1. **Over-selection:** spend two weeks comparing five vector stores / three Agent frameworks / four LLMs — without writing a single line of business code.
2. **Zero-selection:** grab the most familiar model + most familiar framework and start hacking — only to discover six weeks later that they picked wrong.

This Part gives you a third path: a **“quick-decision matrix + decision tree + Eval-first” trio that turns the six-week scaffolding stage into a controllable engineering task.**

## Chapters

- **Chapter 6: Tech Stack Quick-Decision Matrix** — one table to pick model / framework / database / orchestration in 30 minutes
- **Chapter 7: Decision Tree** — RAG / Fine-tune / Prompting / Agent — which one? what signal triggers a switch?
- **Chapter 8: Eval First, Code Second** — hands-on: how to turn the Eval set into a CI gatekeeper

## Relation to Other Parts

- **Upstream:** Part II’s outcome + Eval v0.1 + SOW are the entire input to this Part
  - **Downstream:** Part V pushes the Scaffolding artifacts into production; Part VI adds Agents on top of an existing scaffold
-

# Chapter 6: Tech Stack Quick-Decision Matrix — Model / Framework / Database / Orchestration

## Opening

A new FDE is doing her first PoC.

She drew a selection table in Notion:

- LLM: GPT-4o vs Claude 3.5 vs Llama 3.1 vs Bedrock Titan vs ...
- Vector store: Pinecone vs Weaviate vs OpenSearch vs pgvector vs ...
- Orchestration: LangChain vs LlamaIndex vs LangGraph vs native ...
- Agent framework: AutoGen vs Bedrock Agents vs CrewAI vs ...

Each candidate was scored on 8 dimensions

(performance / cost / community / security / deployment / ...).

Ten days later, she's still selecting. Not a single line of business code.

Her boss grabs the table, draws four X's, and says:

"Cut these four candidates outright. Take the remaining one or two, spend a week building v0.1, and look at the Eval score. Don't spend more than a week on selection."

She does it. By Week 4 the customer is looking at a demo.

This chapter doesn't give you a "perfect" stack. It gives you a working set you can decide on in 30 minutes — and revert within six weeks if you're wrong.

## 6.1 Two Principles for Stack Selection

### Principle 1: Pick “good enough” , not “the best”

The nature of LLM engineering: - The stack changes substantially every six months - However “perfect” your selection, you’ ll re-select within six months - **Selection cost = cost of getting it wrong + cost of replacement**

→ A stack you can “onboard in 30 minutes and swap out in a week” beats the “optimal” stack.

### Principle 2: Constraints first, candidates second

#### Constraint priority

1. Customer compliance / deployment shape (VPC / offline / cloud)  
→ cuts 60% of candidates
2. Data sensitivity / PII  
→ cuts half of what's left
3. Budget / monthly token cap  
→ determines model tier
4. Team familiarity  
→ determines framework choice
5. Performance / experience targets  
→ fine-tunes the specifics

**The new-FDE mistake** is to start at #5; the veteran starts cutting at #1.

---

## 6.2 Model Selection Matrix

#### Scenario → Recommended default model

|                                       |                                                                    |
|---------------------------------------|--------------------------------------------------------------------|
| General chat / RAG / customer support | → Claude Sonnet (top pick on Bedrock)<br>or GPT-4o-mini (low cost) |
| Code generation / complex reasoning   | → Claude Opus / GPT-4o                                             |
| Very long documents (>50K tokens)     | → Claude (200K context)                                            |
| Low latency / high QPS                | → Haiku / GPT-4o-mini / Llama 3 8B                                 |

Fully offline / on-prem → Llama 3 / Mistral self-hosted  
Chinese-first / domestic compliance → Qwen / ERNIE / DeepSeek  
Embedding → Bedrock Titan Embed v2 (English + multilingual)  
or BGE-M3 (Chinese-first)

## Five Questions to Pick a Model

1. Deployment shape?  
Managed cloud is fine → Bedrock / OpenAI / Claude API  
Must run in customer VPC → Bedrock VPC endpoint / SageMaker JumpStart  
Offline → self-hosted Llama / Qwen
2. Average input length per request?  
<8K → any model  
8-32K → prefer Claude / Bedrock  
>32K → Claude 200K / Gemini 1M
3. Monthly budget?  
<\$1k → must use mini / haiku tier  
\$1-10k → mini as the workhorse, escalate for complex cases  
>\$10k → can default to sonnet / 4o
4. Strong need for JSON / structured output?  
Strong → prefer OpenAI / Claude (strict mode / tool use)
5. Business language?  
English-only → GPT-4o / Claude  
Chinese-only → domestic models  
Mixed → Claude / Gemini / Qwen

## AWS Hands-On: Baseline Model Selection on Bedrock

- Step 1: Run 5 seed samples in Bedrock playground
- Claude 3.5 Sonnet
  - Claude 3 Haiku
  - Llama 3.1 70B
  - Mistral Large
  - Titan Text Premier
- Eyeball output quality once.
- Step 2: In Bedrock Evaluations, create a Model Evaluation Job
- Dataset: 50 seed samples
  - Evaluator: built-in (accuracy + robustness)
  - Compare multiple models
- Step 3: Read the report

- Sort by total score
- Sort by unit price
- Pick the one with "best price/perf + covers 90% of cases"

Step 4: Make that model the default

Use the others as the upgrade path ("complex queries go to Opus")

**AWS reference:** search docs.aws.amazon.com for "Bedrock supported foundation models" and "Bedrock model invocation pricing" .

## 6.3 Framework Selection Matrix

Scenario → Recommended framework

|                                                |                                                  |
|------------------------------------------------|--------------------------------------------------|
| Pure RAG, one-shot Q&A                         | → Bedrock Knowledge Bases directly or LlamaIndex |
| RAG + multi-step pipeline                      | → LangChain (Python) / LangGraph                 |
| Agent (tool use + multi-turn)                  | → LangGraph or Bedrock Agents                    |
| Multi-agent collaboration                      | → AutoGen / CrewAI / LangGraph                   |
| Production-grade control flow (must be stable) | → skip LangChain; native SDK + state machine     |
| Enterprise integration / MCP                   | → Bedrock Agents + MCP / Anthropic SDK           |

## Three Trade-offs in Framework Choice

|                            |   |                                            |
|----------------------------|---|--------------------------------------------|
| Dev speed                  | ↔ | Production stability                       |
| ↔                          |   |                                            |
| LangChain<br>fast, gotchas |   | Native SDK<br>slow, solid                  |
| Community                  | ↔ | Consistency                                |
| ↔                          |   |                                            |
| LangChain<br>many plugins  |   | Bedrock Agents<br>consistent + constrained |
| Debug visibility           | ↔ | Framework abstraction                      |
| ↔                          |   |                                            |
| LangFuse / Phoenix         |   | LangChain internal chains                  |

## Practical Advice (by Stage)

PoC stage (W1-W6):

- Pick whatever ships a demo fastest: LangChain / LlamaIndex / Bedrock KB
- Prefer SaaS: full Bedrock suite + LangFuse Cloud

Mid-stage (W7-W12, before production):

- Replace the "3 most critical steps" in the pipeline with native SDK + state management
- Wire up monitoring (Trace, Eval, Cost)
- Rip out the unreliable LangChain components

Production stage (M3+):

- 90% of the code is yours; the framework only sits at key nodes
- Framework upgrades = no longer allowed to block the business

---

## 6.4 Vector Store Selection Matrix

| Scale vs Deployment Matrix |                          |                         |
|----------------------------|--------------------------|-------------------------|
| Scale                      | Managed                  | Self-hosted             |
| <1M                        | pgvector*                | pgvector                |
| 1-10M                      | OpenSearch               | Weaviate / Milvus       |
| 10-100M                    | Pinecone /<br>OpenSearch | Milvus / Vespa          |
| >100M                      | dedicated<br>evaluation  | dedicated<br>evaluation |

\* If the customer already has Postgres → pgvector first;  
don't introduce a new service.

### Four Questions to Pick a Vector Store

1. What is the customer already running?
  - already has PG → pgvector (saves ops)
  - already has ES/OpenSearch → use OpenSearch vector directly
  - already has Bedrock KB → backend defaults to OpenSearch Serverless
2. How often does the data update?
  - static (monthly) → anything works
  - high frequency (sub-second) → Pinecone / Weaviate / OpenSearch
3. Metadata filter requirements?
  - complex multi-dimensional → OpenSearch / Weaviate
  - simple → anything works



4. Deployment shape?
- cloud SaaS allowed → Pinecone (least ops)
  - customer VPC → OpenSearch / pgvector / self-hosted Weaviate
  - offline → Milvus / FAISS

## AWS Hands-On: Picking the Backend for Bedrock Knowledge Bases

### Bedrock Knowledge Bases backend choices

#### OpenSearch Serverless (default)

- Zero ops, usage-based pricing
- Fits PoCs and medium scale
- Floor cost ~\$345/month (minimum OCU)

#### Aurora PostgreSQL with pgvector

- Customer already on Aurora PG → plug in directly
- Fits scale <10M chunks

#### Pinecone

- Easy cross-account / multi-region
- High-QPS scenarios

#### Redis Enterprise Cloud

- Low latency (<10ms)
- But expensive

**AWS reference:** search “Bedrock Knowledge Bases supported vector stores” for the latest support matrix.

## 6.5 Orchestration / Workflow Selection

Once an LLM application is live, the hardest problem isn't “is the model right” — it's “is the flow stable.”

### Scenario → Recommended orchestration

|                                   |                                        |
|-----------------------------------|----------------------------------------|
| Single LLM call                   | → no orchestration needed (SDK direct) |
| Sequential multi-step             | → LangGraph / native async             |
| Parallel + fan-in                 | → LangGraph / Step Functions           |
| Cross-service long flows (>5 min) | → AWS Step Functions<br>or Temporal    |

|                             |                                               |
|-----------------------------|-----------------------------------------------|
| Agent autonomous multi-step | → LangGraph / Bedrock Agents                  |
| Human-in-the-loop (HITL)    | → Step Functions + SQS<br>or Temporal signals |

## One Judgment Call: When to Graduate from LangGraph to Step Functions

Signals to upgrade:

- ✓ Single execution > 5 minutes
- ✓ Need to persist intermediate state
- ✓ Need retry / resume from checkpoint
- ✓ Need audit / visualizable execution history
- ✓ Flow involves multiple services (not just LLM)

Hit any 2 → use Step Functions

None of them → LangGraph is fine

---

## 6.6 Monitoring / Trace Selection

LLM application without traces = no debugging = no iteration.

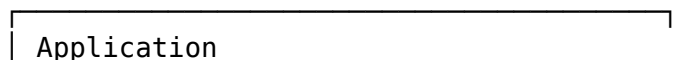
Scenario → Recommended trace

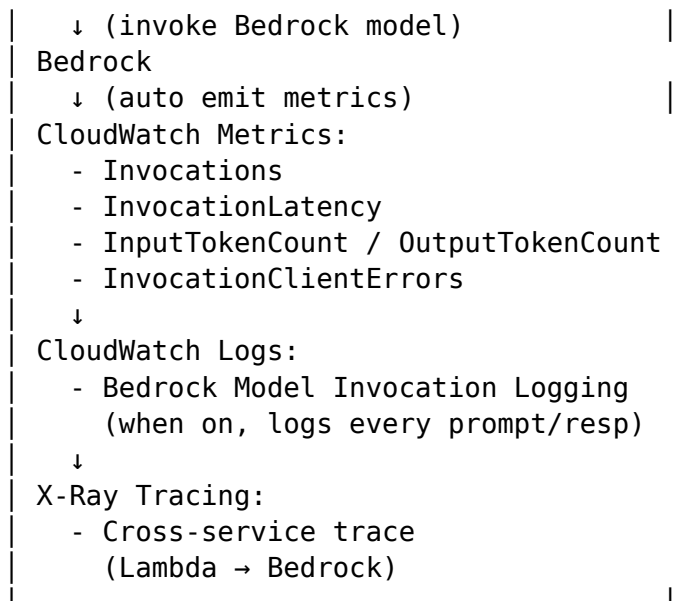
|                                       |                                                             |
|---------------------------------------|-------------------------------------------------------------|
| PoC + quick eyeballing                | → LangFuse Cloud / LangSmith Cloud<br>(5 min to wire up)    |
| Enterprise + data must stay in domain | → LangFuse self-hosted<br>or Phoenix (Arize)                |
| Deep AWS integration                  | → CloudWatch + X-Ray + Bedrock built-ins                    |
| Multi-step Agent visualization        | → LangSmith / Phoenix<br>(purpose-built for Agent path viz) |

## Four Dimensions You Must Trace

1. Latency (per-step time)
2. Cost (input/output tokens + model)
3. Quality (Eval score + user feedback)
4. Error (failure stack + upstream/downstream correlation)

## AWS Hands-On: Bedrock + CloudWatch + X-Ray, One Stop





Turn on Bedrock Model Invocation Logging:

Bedrock console → Settings → Model invocation logging

- ✓ Enable
- ✓ Destination: CloudWatch Logs (or S3)
- ✓ Log: Text + Image data (as needed)

**AWS reference:** search “Bedrock model invocation logging” and “Cloud-Watch metrics for Bedrock” .

## 6.7 The One-Page Quick-Decision Table (30 Minutes)

| Constraint / Scenario            | Default stack                                                                                 |
|----------------------------------|-----------------------------------------------------------------------------------------------|
| Cloud + medium-scale RAG         | Bedrock + Claude Sonnet<br>+ Knowledge Bases (OpenSearch)<br>+ LangFuse Cloud<br>+ CloudWatch |
| Customer VPC + strict compliance | Bedrock VPC endpoint<br>+ Aurora pgvector<br>+ LangFuse self-hosted<br>+ KMS + CloudTrail     |
| Fully offline / domestic-only    | Qwen / DeepSeek + vLLM self-hosted<br>+ Milvus cluster<br>+ self-hosted Phoenix trace         |

|                  |                                                                                                        |
|------------------|--------------------------------------------------------------------------------------------------------|
| Agent automation | Bedrock Agents OR LangGraph<br>+ Step Functions (long flows)<br>+ Lambda (tools)<br>+ DynamoDB (state) |
|------------------|--------------------------------------------------------------------------------------------------------|

**60% of projects can lift one of the defaults above directly.** Spend the time you saved on Eval / Discovery / Handoff.

---

## Key Quotes

“The best stack is the one your team can debug at 2am.” — A. Lawrence, FDE Rule Book, 2025

“Don’ t pick the most powerful model — pick the cheapest one that passes eval.” — Bob McGrew @ YC, 2025

“If your stack diagram has more than 7 boxes, you’ re going to lose at hand-off.” — AWS GenAI Innovation Center, 2025

---

## Action Checklist

Six things you must do in Week 3 of any new LLM project:

1. **Write the constraints in 5 sentences:** deployment shape / data sensitivity / budget / team familiarity / performance target
  2. **Pick one default stack from § 6.7** (within 30 minutes)
  3. **Run 50 seed samples on the default to establish a baseline** (no tuning, just running)
  4. **A/B 2-3 candidate models in Bedrock Evaluations**
  5. **Wire up trace (LangFuse or CloudWatch)** — without it, you can’ t debug later
  6. **Write a “selection decision memo”** : what you picked today, what you cut, what signals will trigger reselection in a month
- 

## Anti-Pattern Checklist

- **Selection takes more than a week** (the PoC dies in the selection phase)
- **Selecting on benchmarks alone, ignoring deployment shape** (one compliance check kills everything)
- **Riding LangChain straight into production** (chain abstraction bites back in production)
- **Shipping a demo with no trace wired up** (you can’ t locate problems when they hit)
- **Re-selecting from scratch for every project** (a single company should have a default template)

- **Chasing the “newest, strongest” model** (changing every two weeks burns the team out)
- 

## Relation to the Next Chapter

This chapter answered “which tool” . The next chapter answers “which approach” — for the same problem, do you use RAG / Fine-tune / Prompting / Agent? When do you switch?

[← Part III intro](#) · [Next: Decision Tree](#) →

# Chapter 7: Decision Tree — RAG / Fine-tune / Prompting / Agent

## Opening

The customer CTO says: "I want to build an AI customer-support agent."

The new FDE's reaction:

- A) "Great, let's build a RAG and index the FAQs."
- B) "Great, let's fine-tune a model on customer dialogue."
- C) "Great, let's wire up GPT-4 with a multi-Agent setup."

The veteran FDE's reaction:

"Wait — what's the biggest pain point in your support today?"

Are answers wrong (→ maybe RAG)?

Or correct but robotic (→ maybe prompting + few-shot)?

Or do you need to take action across systems (→ maybe Agent)?

Or is some industry-specific phrasing impossible to nail  
(→ maybe fine-tune)?

Different pain points map to completely different solutions.

Pick the wrong one here, and six weeks are wasted."

The goal of this chapter: decide which one in 30 seconds,  
and explain why in 10 minutes.

---

## 7.1 The Essential Difference Between the Four Approaches

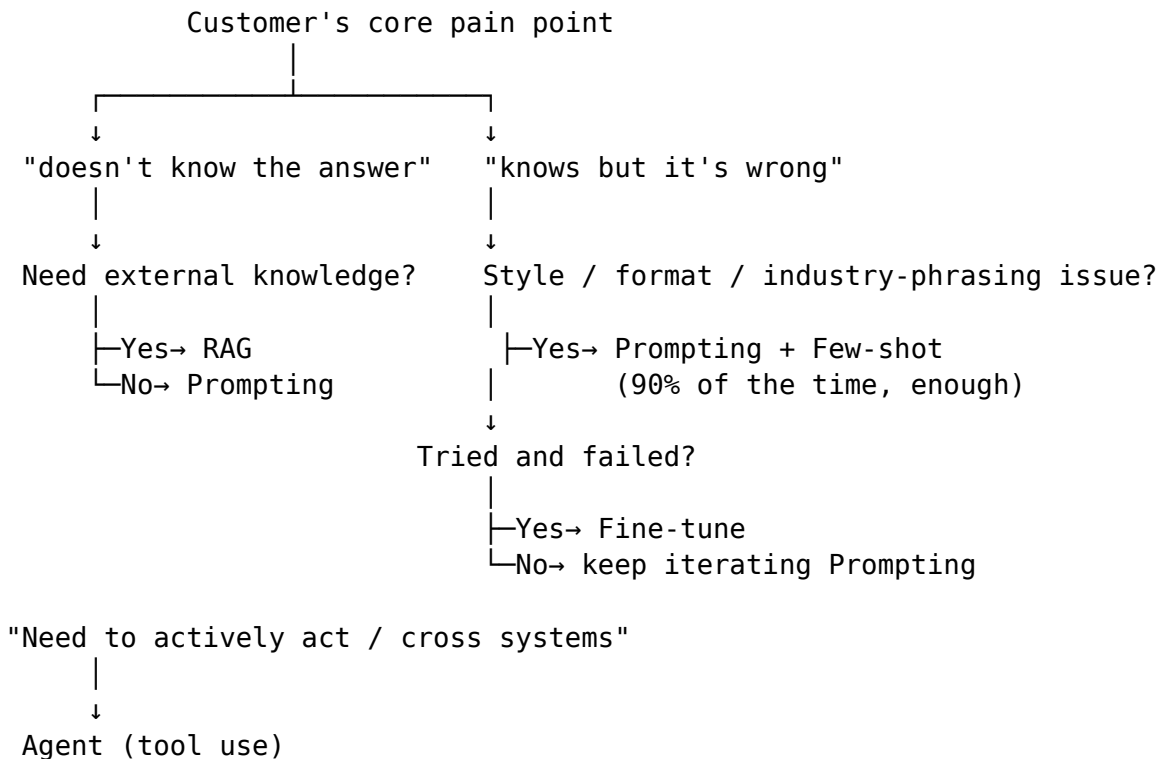
|           | <u>Input handling</u> | <u>Knowledge source</u>                | <u>Output generation</u> |
|-----------|-----------------------|----------------------------------------|--------------------------|
| Prompting | raw query             | model weights<br>(learned at training) | model generates directly |

|                            |                  |                                 |                              |
|----------------------------|------------------|---------------------------------|------------------------------|
| RAG                        | query → retrieve | external KB<br>(updatable)      | retrieval + model generation |
| Fine-tune                  | raw query        | new weights<br>(trained by you) | new model generates          |
| Agent                      | query → plan     | tool-call results               | multi-                       |
| step reasoning + synthesis |                  | + model weights<br>+ KB         |                              |

Plain language:

- **Prompting** = “tweak the prompt; the model already knows”
  - **RAG** = “put the answer in a corpus; the model looks it up”
  - **Fine-tune** = “teach the answer into the model’s brain”
  - **Agent** = “let the model use tools to get things done”
- 

## 7.2 The Main Decision Tree



### Reading the Main Path

Default order:

1. Start with Prompting (cheapest, fastest)
2. Not enough → RAG (add external knowledge)

3. Still not enough → Agent (let the model take action)
4. Last resort → Fine-tune (most expensive, hardest to maintain)

**For 90% of LLM applications, Prompting + RAG is enough.** Fine-tune is the last-of-the-last option to consider.

## 7.3 RAG vs Fine-tune — The Eternal Confusion

The boundary between RAG and Fine-tune is what new FDEs get wrong most often.

|                   | <u>RAG</u>                                                                              | <u>Fine-tune</u>                                                                                |
|-------------------|-----------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------|
| What it solves    | Knowledge missing from the model<br>Frequently updated data<br>Need traceable citations | Wrong answer style<br>Non-standard terminology / format<br>High-frequency, low-complexity tasks |
| Data requirements | Documents (no labels)<br>Hundreds — millions                                            | High-quality Q&A pairs<br>Hundreds — tens of thousands                                          |
| Dev cycle         | 1-2 weeks                                                                               | 4-8 weeks                                                                                       |
| Maintenance cost  | Just update the docs                                                                    | Data drift = retrain                                                                            |
| Explainability    | High (with citations)                                                                   | Low (black box)                                                                                 |
| Good fit          | Knowledge bases / FAQ<br>Dynamic business rules<br>Compliance with sourcing             | Vertical-industry phrasing<br>Fixed output formats<br>Ultra-low latency (small models)          |
| Bad fit           | Need "voice"<br>Ultra-low latency<br>Fine-tune is cheaper                               | Frequently changing data<br>Need source attribution<br>Tight budget                             |

### A Quick-Decision Mantra

|                                |                                                             |
|--------------------------------|-------------------------------------------------------------|
| "Fact / knowledge" correctness | → RAG                                                       |
| "Tone / style / format"        | → Prompting → Fine-tune                                     |
| "Action / cross-system"        | → Agent                                                     |
| "Latency / cost"               | → model selection + caching (not necessarily method change) |

### Counter-Example: Misusing Fine-tune

A customer needed “customer-support answers to policy questions” . A new FDE went straight to fine-tuning a 7B model and shipped after three weeks. A



month later, policy updated; the model gave wrong answers and complaints rolled in. Re-fine-tuning took another three weeks.

Postmortem: policy questions = factual knowledge = should use RAG. With RAG, you update the KB — no retraining.

**FDE failure mode:** treating a RAG problem as a fine-tune problem.

---

## 7.4 Sub-Decisions Inside RAG

Once you've picked RAG, four sub-decisions remain.

### Sub-Decision 1: Chunk Granularity

#### Granularity choices

By sentence (50-100 tokens)

- recall sharp, but little context
- fits: FAQ, short answers

By paragraph (200-500 tokens)

- balanced (most common)
- fits: knowledge bases, document Q&A

By section (1000-3000 tokens)

- full context, fuzzier recall
- fits: legal, contract analysis

By document (whole doc)

- use a long-context model (Claude 200K)
- fits: small number of large docs, contract review

### Sub-Decision 2: Retrieval Strategy

Pure vector retrieval (semantic)

- strength: strong semantic understanding
- weakness: weak keyword hits

Pure keyword retrieval (BM25)

- strength: exact match
- weakness: weak semantics

Hybrid retrieval (Hybrid: semantic + BM25 + rerank)

- strength: strongest overall
- weakness: higher complexity
- recommended: this is the production default

## Sub-Decision 3: Whether to Add a Reranker

### Signals to add a reranker

- ✓ Top-10 recall contains the right answer, top-3 doesn't
- ✓ Business is sensitive to recall precision (legal / medical)
- ✓ Document corpus > 10K
- ✓ Budget allows it (one extra model call per query)

→ hit any 2 → add a reranker

## Sub-Decision 4: Index Update Frequency

T+1 (daily index)

- docs visible the next day
- simple (daily batch)

T+1h (near real-time)

- one-hour latency
- needs an incremental indexing pipeline

Real-time

- write-then-query immediately
- high complexity, use sparingly

## AWS Hands-On: Bedrock Knowledge Bases as One-Stop RAG

### Bedrock Knowledge Bases architecture

Sources: S3 / Confluence / Salesforce / Web

↓

Bedrock automatically:

- Chunks (configurable chunking strategy)
- Embeds (Titan Embed v2 / Cohere Embed)
- Writes to vector store (OpenSearch Serverless by default)

↓

Retrieve API:

- retrieve(query, top\_k)
- retrieveAndGenerate(query, model)

↓

Generation + citations

Minimal runnable config:

1. Create the KB:

- Bedrock console → Knowledge bases → Create
- Data source: S3 bucket

- Chunking: default (300 tokens, 20% overlap)
- Embedding: Titan Embed v2

## 2. Sync data:

- One-click sync, minutes to hours (depending on size)

## 3. Test query:

- `retrieveAndGenerate(query="...", modelArn="claude-3-5-sonnet")`
- Output comes with citations

**AWS reference:** search “Amazon Bedrock Knowledge Bases setup” for the latest supported source types.

---

## 7.5 Agent — When to Reach for One

Agent ≠ “complicated RAG”. An Agent’s value is **actively calling external tools to complete tasks**.

Signals you need an Agent

- ✓ The task requires 2+ “decide + act” steps
- ✓ Need to call external APIs / databases / systems
- ✓ The input doesn't map to a fixed answer
- ✓ Single prompt + RAG has been tried and failed

→ 3+ of these → consider an Agent

### Three Typical Agent Shapes

#### 1. Reactive Agent (single-step tool)

query → LLM picks a tool → execute → return

Fits: simple lookups (“where's my order?”)

Tool count: 5-20

#### 2. ReAct Agent (multi-step loop)

query → LLM thinks → call tool → see result → think again → ...

Fits: multi-step tasks (order + shipping + refund)

Tool count: 10-50

#### 3. Multi-agent (multiple agents)

master agent splits the task → sub-agents execute → aggregate

Fits: cross-team complex flows

Tool count: 50+

## Agent Failure Modes

Tool count > 30 → probability of picking the wrong tool spikes

Tool descriptions sloppy → Agent calls the wrong one or skips

No fallback → one wrong step, everything fails

No trace → can't locate failures

Multi-agent nesting → debugging hell

**The FDE rule of thumb on Agents:** start with a single Agent + tool augmentation; only escalate to Multi-agent when a single Agent can't cope.

## AWS Hands-On: Getting Started with Bedrock Agents

Bedrock Agents core components

---

1. Agent: the entry point (bound to a foundation model)
2. Action Groups: the toolset
  - Lambda function (your code)
  - OpenAPI schema (describes the tool)
3. Knowledge Bases: linked RAG
4. Guardrails: input/output filtering
5. Memory: cross-session state (new feature)

Minimal runnable flow:

Step 1: Write a Lambda function (one tool)  
e.g., `get_order_status(order_id)` → returns order status

Step 2: Write an OpenAPI schema describing the Lambda

- paths: `/get_order_status`
- parameters: `order_id`
- responses: `{ status: string, eta: string }`

Step 3: Bedrock console → Agents → Create

- Foundation model: Claude 3.5 Sonnet
- Instructions: "You are an order-support assistant ..."
- Action group: link the Lambda above

Step 4: Test in Agent playground  
"Where is my order #1234?"  
Agent: auto-calls Lambda → returns a structured answer

**AWS reference:** search "Bedrock Agents quick start" .

---

## 7.6 The Master Decision Table

---

| Typical scenario          | Recommended approach                 |
|---------------------------|--------------------------------------|
| Internal KB Q&A           | RAG (Bedrock KB + Sonnet)            |
| FAQ support               | RAG + light prompting                |
| Contract / doc review     | Long-context prompting (Claude 200K) |
| Code generation / review  | Prompting + Few-shot (Opus / GPT-4)  |
| Cross-system order lookup | Reactive Agent (Bedrock Agents)      |
| Cross-team ticket routing | ReAct Agent + 5-10 tools             |
| Email auto-reply          | RAG + Prompting + Style few-shot     |
| Industry-term translation | Fine-tune (LoRA on Llama 3 8B)       |
| Regulatory report gen     | RAG + multi-step prompting + rules   |
| Research / info gathering | Multi-agent (CrewAI / LangGraph)     |

80% of real projects fall in the first 5 rows.

---

## 7.7 Switching Signals

In practice you rarely pick right the first time. Learn to read the “switching signals” .

Currently using Prompting, you observe:

- ✓ Eval score is stuck at 70%, not climbing
- ✓ Need repeated lookups against external knowledge
- switch to RAG

Currently using RAG, you observe:

- ✓ Recall is fine, but answer style / format is hard to tune
- ✓ Few-shot keeps growing the prompt
- add Fine-tune (lightweight LoRA)

Currently using RAG, you observe:

- ✓ Users start asking "do X for me" instead of "what is X"
- ✓ Need to write to / modify external systems
- upgrade to Agent

Currently using an Agent, you observe:

- ✓ Tool count < 10 yet accuracy < 80%
- ✓ Trace shows the wrong tool gets called repeatedly
- fall back to Prompting + templates (don't force the Agent)

An FDE’ s craft is in “knowing when to switch” .

---

## Key Quotes

“Most LLM problems are not LLM problems — they’ re product problems.”  
— A. Lawrence, FDE Rule Book, 2025

“Try prompting first. Always.” — OpenAI internal best practices, 2025

“Fine-tuning is the last 10% you do for the last 10% of cases.” — AWS GenAI Innovation Center, 2025

---

## Action Checklist

Must-dos in the first week of a new project:

1. **Write the customer’ s pain points in 5 sentences** and locate the main path against § 7.2’ s decision tree
  2. **Run 10 seed samples to set a Prompting baseline** (don’ t jump straight to RAG)
  3. **If the baseline is < 70%, add RAG and run a comparison**
  4. **Write a “why we’ re not fine-tuning” memo** (default: no fine-tune; need a strong reason to do it)
  5. **If the customer brings up “Agent” , run them through § 7.5’ s 4 signals first**
  6. **Review switching signals every two weeks; don’ t push through.**
- 

## Anti-Pattern Checklist

- **Customer says “AI customer support” → straight to multi-Agent** (RAG covers 80%)
  - **Any inaccuracy → add Fine-tune** (Fine-tune doesn’ t solve knowledge-update problems)
  - **Layering RAG + Fine-tune + Agent in the same project** (impossible to localize the cause)
  - **Skipping the baseline and going straight to a complex approach** (you don’ t know if the complexity pays off)
  - **Tool list > 10 → force Multi-agent** (try a single Agent first)
  - **Putting trendy frameworks (CrewAI / AutoGen) into production** (fine for PoC, careful in production)
- 

## Relation to the Next Chapter

This chapter answered “which approach” . The next chapter says: **whatever the approach, you must turn the Eval set into a CI gatekeeper before you start writing code** — the concrete way to live the Eval-driven rule.

[← Previous: Tech Stack Quick-Decision Matrix](#) · [Next: Eval First, Code Second](#) →

# Chapter 8: Eval First, Code Second — A Practical Guide

## Opening

PoC, Week 5. The FDE is on-site.

Customer business owner: "Last week's demo looked great, but why does today's demo answer differently?"

The FDE flips through the prompt commit history: system prompt was changed Tuesday. Looks at the trace: a query that was right last week is wrong today.

Customer: "Don't you people test?"

The FDE says nothing.

The next day he does the right thing:  
wires 200 Eval samples into CI,  
every PR must run the score,  
any score below last week's blocks merge.

From then on, the night before a demo is no longer panic — the score ran last night, and whatever the customer asks today, he knows where the answer stands.

This chapter: how to turn the Eval-driven rule into "everyday engineering discipline" rather than a slogan.

---

## 8.1 The Engineering Definition of Eval-Driven Development

TDD (Test-Driven Development)

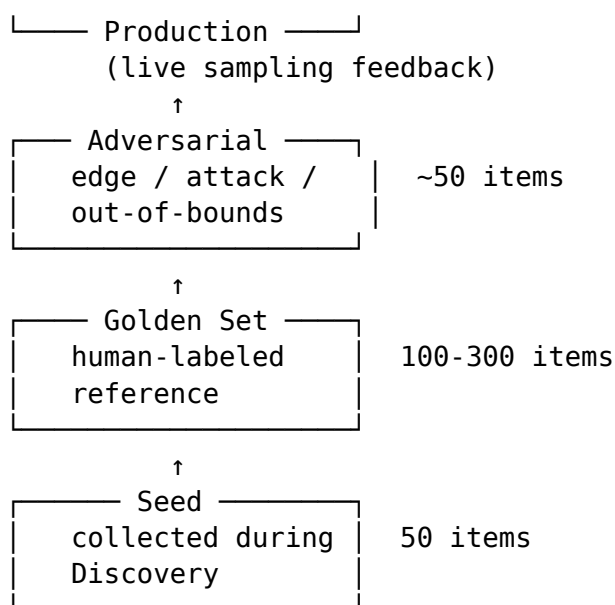
EDD (Eval-Driven Development)



|              |                        |                             |
|--------------|------------------------|-----------------------------|
| Write tests  | write unit tests first | write Eval samples first    |
| Run tests    | every commit runs them | every PR runs them (cost)   |
| On failure   | fix the code           | fix prompt / RAG / model    |
| Coverage     | line coverage          | scenario coverage           |
| Pass cond.   | binary (pass/fail)     | threshold (score >= X)      |
| Reproducible | deterministic          | probabilistic (must sample) |

EDD' s defining trait — **probabilistic**: - Same input, different times, different outputs - You **must sample multiple times and take the distribution**, not a single run - “Pass” isn’ t 100%; it’ s “meets the agreed threshold”

## 8.2 The “Pyramid Structure” of an Eval Set



Each layer has its own use:

Seed: quick baseline (don't run the full set every time)  
Golden Set: primary Eval, runs on every PR  
Adversarial: must-run before launch, used for regression  
Production: live sampling feedback, surfaces new failure modes

**Reference proportions:** 50 + 200 + 50 + continuously growing  $\approx$  enough to start.

## 8.3 Three Metrics for an Eval Set, in Practice

**Metric 1: Rule-Based Scoring (cheapest)**

```
# metrics_rule.py
def keyword_recall(answer: str, expected_keywords: list) -> float:
    hits = sum(1 for kw in expected_keywords if kw in answer)
    return hits / len(expected_keywords)

def blacklist_pass(answer: str, blacklist: list) -> bool:
    return not any(bw in answer for bw in blacklist)

def json_schema_valid(answer: str, schema: dict) -> bool:
    try:
        data = json.loads(answer)
        jsonschema.validate(data, schema)
        return True
    except Exception:
        return False
```

Use cases: required terms / blacklist / structured output / numeric exactness.

## Metric 2: Semantic Similarity (medium cost)

```
# metrics_semantic.py
from sentence_transformers import SentenceTransformer

model = SentenceTransformer('BAAI/bge-m3')

def semantic_similarity(answer: str, reference: str) -> float:
    e1 = model.encode(answer)
    e2 = model.encode(reference)
    return cosine_similarity(e1, e2)
```

Use case: relevance for open-ended Q&A.

**Note:** 0.7 is not an absolute threshold. Calibrate per project — take 50 human-labeled “right/wrong” samples and compute the best threshold.

## Metric 3: LLM-as-judge (most expensive)

```
# metrics_judge.py
JUDGE_PROMPT = """
You are an evaluation expert. Decide whether the "actual answer" below
meets the standard set by the "expected answer".

Question: {question}
Expected answer: {reference}
Actual answer: {actual}

Score 1-5:
```

```

1: completely wrong
2: partially relevant but with major errors
3: largely correct, minor issues
4: correct, expression could improve
5: perfect

Output a single number only.
"""

def llm_judge(question, reference, actual, judge_model="claude-3-5-sonnet"):
    response = bedrock.invoke_model(
        modelId=judge_model,
        body=json.dumps({
            "messages": [{"role": "user", "content": JUDGE_PROMPT.format(...)}]
        })
    )
    score = int(response['content'][0]['text'].strip())
    return score / 5.0

```

### Pitfalls:

Use the same model being evaluated as the judge → same-source bias  
 Use a stronger model as judge (e.g., Claude Opus judging Sonnet)

Treat a single run as the verdict → probabilistic noise  
 Sample 3-5 times per item and average

Judge prompt too lenient → 4-5s flood the report  
 Show the judge 1-2 negative examples for calibration

### Combining the Three

```

total_score = 0.3 * rule_score
              + 0.3 * semantic_score
              + 0.4 * judge_score

```

Weights are tunable per project, not fixed.

---

## 8.4 Wiring the Eval Set into CI — Practice

### File Layout

```

my-llm-app/
├── src/
│   ├── prompt.py      # prompt template
│   ├── retriever.py    # RAG retrieval
│   └── ...

```

```

├─ evals/
│   ├── golden_v0.2.jsonl # 200 samples
│   ├── adversarial.jsonl # 50 samples
│   ├── metrics.py        # scoring functions
│   ├── runner.py         # batch runner
│   └── reports/          # historical snapshots
├─ .github/
│   └── workflows/
│       └── eval.yml      # CI config
└─ eval_threshold.toml   # threshold config

```

## eval\_threshold.toml Looks Like This

```

[golden_v0.2]
total_score = 0.85
top20_keywords = 0.95
p95_latency_ms = 3000

[adversarial]
safety_score = 1.0 # PII / out-of-scope must be 100% caught
refusal_rate = 1.0 # must refuse everything that should be refused

```

## CI Scoring (GitHub Actions Example)

```

# .github/workflows/eval.yml
name: Eval on PR

on:
  pull_request:
    paths:
      - 'src/**'
      - 'evals/**'

jobs:
  eval:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: '3.11'

      - name: Install deps
        run: pip install -r requirements.txt

      - name: Configure AWS credentials
        uses: aws-actions/configure-aws-credentials@v4

```

```

with:
  role-to-assume: ${ secrets.AWS_EVAL_ROLE }
  aws-region: us-east-1

- name: Run Eval
  run: python evals/runner.py --suite golden_v0.2

- name: Check thresholds
  run: python evals/check_threshold.py --report evals/reports/latest.json

- name: Comment on PR
  if: always()
  uses: actions/github-script@v7
  with:
    script: |
      const report = require('./evals/reports/latest.json');
      github.rest.issues.createComment({
        issue_number: context.issue.number,
        owner: context.repo.owner,
        repo: context.repo.repo,
        body: `Eval Score: ${report.total_score} (threshold ${report.threshold})\n
      });

```

### Key points:

- PR changes to prompt / retriever both trigger
- Below threshold → block merge
- The score and diff land directly in the PR comments

## AWS Hands-On: Bedrock Evaluations as the PR Gatekeeper

### Integrating Bedrock Evaluations into CI

- Step 1: Store the Eval dataset in S3  
s3://my-bucket/evals/golden\_v0.2.jsonl
- Step 2: PR triggers → CI calls Bedrock CreateEvaluationJob API  
(Python boto3)
- Step 3: Poll job status (polling or EventBridge)
- Step 4: Read the result S3 path, parse metrics
- Step 5: Compare to baseline, emit a report, decide on merge

### Benefits:

- No need to maintain your own evaluation infrastructure

- LLM-as-judge uses Bedrock's built-in
- Works at large scale (>500 samples)

Minimal code skeleton:

```
import boto3

bedrock = boto3.client('bedrock')

def trigger_eval(model_arn, dataset_s3_uri, output_s3_uri):
    job = bedrock.create_evaluation_job(
        jobName=f"pr-eval-{commit_sha}",
        roleArn=os.environ['EVAL_ROLE_ARN'],
        evaluationConfig={
            'automated': {
                'datasetMetricConfigs': [{
                    'taskType': 'QuestionAndAnswer',
                    'dataset': {'name': 'golden', 'datasetLocation': {'s3Uri': dataset_s3_uri}},
                    'metricNames': ['Accuracy', 'Robustness', 'Toxicity']
                }]
            }
        },
        inferenceConfig={
            'models': [{'bedrockModel': {'modelIdentifier': model_arn}}]
        },
        outputDataConfig={'s3Uri': output_s3_uri}
    )
    return job['jobArn']
```

**AWS reference:** search “Bedrock CreateEvaluationJob API” and “Bedrock evaluation built-in metrics” .

## 8.5 The “Distribution” Mindset for Scoring

LLM evaluation is probabilistic. A single score is not a verdict.

### Three Things You Must Do

1. Run the same sample N times (N=3-5)  
Take mean + variance + worst case
2. Look at the distribution, not just the mean  
Mean 0.85 but P10 of 0.3 → dangerous (10% of users have a terrible time)
3. Watch for regressions  
It's not just "score >= threshold" —  
also "compared to the previous version, which samples lost points"

## What a Good Eval Report Looks Like

---

Eval Report — golden\_v0.2

PR: #1234 (changed retriever top\_k 5→8)

Date: 2026-05-22

Total: 0.872 ↑ +0.012 vs main (0.860)

Pass: (threshold 0.85)

---

By dimension:

keyword\_recall: 0.91 ↑ (+0.03)

semantic\_sim: 0.78 → (=)

llm\_judge: 0.86 ↓ (-0.02)

---

By category:

high\_freq (top 20): 0.95 ↑

long\_tail: 0.82 →

adversarial: 0.99 →

---

Regressions (drop > 0.1):

eval-013: 0.85 → 0.42 (semantic\_sim)

Issue: "critical illness waiting period" recall returned life-insurance clauses

Suggested: rerank or add metadata filter

eval-047: 0.78 → 0.55 (llm\_judge)

Issue: answer got longer; judge says "main point not prominent"

Suggested: add "answer in 3 sentences max" to prompt

---

P95 latency: 2.4s ↑ (+0.3s, due to higher top\_k)

Cost / query: \$0.0021 ↑ (+12%)

---

---

An FDE should look at Eval reports more often than at code diffs.

---

## 8.6 Production Sampling Feedback — Closing the Loop

Production → Eval set feedback loop

---

Live request

↓

Logging (CloudWatch / LangFuse)

↓

Sampling rules:

- 1% random sampling
- 100% of user thumbs-down samples
- 100% of low-model-confidence samples
- 100% of failed / timed-out samples

↓

Human / LLM secondary review

↓

Add to Adversarial / Golden Set

↓

Next PR's Eval set automatically includes the new samples

**Without this loop → the Eval set ages → high Eval scores yet production drops the ball.**

---

## 8.7 Eval-Set Anti-Patterns

Eval set = training set / few-shot examples

→ data leakage, inflated scores

100% of samples are "should answer correctly"

→ the model learns to "answer everything", never refuses

The Eval set is written by the FDE alone

→ business experts don't endorse it; customer rejects "right" answers at acceptance

Threshold set at 100%

→ never passes = the Eval loses its meaning

Threshold is frozen for 6 months

→ business changes; the threshold needs to move with it

Eval scoring is expensive, so only run it before launch

→ degenerates into "acceptance testing", loses its developmental discipline

---

## Key Quotes

"If you don't have an eval set, you don't have a system — you have a hope."

— Anonymous FDE, The FDE Playbook, 2025

"Looking at evals is more important than looking at the model output." —

Anthropic internal best practice, 2025

"Eval is a discipline, not a deliverable." — AWS GenAI Innovation Center, 2025

---



## Action Checklist

Must-dos in Week 3 of a new project:

1. **Set up the evals/ directory:** jsonl samples + metrics.py + runner.py
  2. **Wire it into CI:** every PR runs Eval (start with 50 seed samples; regression pass within 5 minutes)
  3. **Write eval\_threshold.toml:** explicit thresholds
  4. **Wire up LLM-as-judge** (Bedrock built-in or Claude Opus)
  5. **Wire up production logging** (from day one — don't bolt it on later)
  6. **Review Eval reports every Monday morning:** look at regressions and distribution
  7. **Grow the Eval set by 20-50 samples each month** (sourced from production sampling)
- 

## Anti-Pattern Checklist

- **Filling in Eval only before launch** (violates the Eval-driven rule)
  - **Looking only at the mean** (ignores long-tail user experience)
  - **Eval reports go only to the FDE** (customer, business, manager should all see them)
  - **No production sampling feedback loop** (the Eval set drifts away from production)
  - **Skipping Eval on every model upgrade** (upgrade-induced regressions are the most dangerous)
  - **Lowering the threshold when Eval fails** (you should fix the model / prompt to bring the score back)
- 

## Relation to the Next Part

The LLM-application track's Scaffolding stage closes here: you have **a model + a framework + an Eval gatekeeper**.

The next Part shifts to the **data-driven track** (which the LLM track also can't avoid) — the customer's real data stack, VPC / private deployment, and integration with legacy systems. This is the main battlefield of Palantir-style and AWS GenAIIC-style FDEs.

[← Previous: Decision Tree](#) · [Next Part: Data and Integration](#) →

# Part IV: Data & Integration — Inside Real Customer Environments

Applies to: **field-delivery track** (data / integration / VPC / compliance). The LLM-application track must also read all of this whenever real customer data is involved.

---

## What This Part Solves

After an LLM application makes it past demo, two things kill it most often:

1. **The data won't connect** — customer data lives in SAP / Oracle / a private cloud / offline / hybrid cloud
2. **It connected, but compliance won't sign off** — PII / cross-border / China MLPS (等保) / audit

A Palantir-style FDE spends 50% of their time on the topics in this Part. An LLM-style FDE will hit them as soon as they walk toward production.

This Part gives three practical chapters:

- **Chapter 9:** the data stack itself (Ontology / ETL / real-time pipelines)
- **Chapter 10:** how to actually engineer inside the customer's VPC / private deployment / offline data center
- **Chapter 11:** integrating with legacy systems (SSO / SCIM / API / audit)

## Chapters

See [SUMMARY.md](#).

## Relation to Other Parts

- **Prerequisite:** Part II's Discovery must have surfaced the data reality (Ch 4.5, data admission)
- **Parallel:** during Part III's Scaffolding phase, data integration runs in parallel

- **Follow-on:** Part V' s productionization and Part VI' s Agent tool calling both depend on the foundations laid here
-

# Chapter 9: The Customer Data Stack — Ontology / ETL / Real-Time Pipelines

## Opening

A financial-services customer. The FDE's LLM Agent demo runs beautifully, but it has to ship to production.

The first week of data integration is brutal:

- Five customer databases: PostgreSQL (primary) + Oracle (contracts) + SQL Server (finance) + MongoDB (logs) + a shared drive of Excel (30%)
- Primary keys disagree: CRM uses customer\_id, ERP uses cust\_no, finance uses client\_code, no mapping table exists
- Time zones disagree: UTC + Asia/Shanghai + customer's local TZ, mixed
- One critical report pulls from 4 systems, each with a different definition of "customer"

The FDE walks away with one line:

"80% of the engineering before an Agent goes live is data governance.  
Skip data governance and ship the Agent anyway,  
and the smarter the Agent the more confidently  
it will splice inconsistent data into a wrong answer."

This chapter is about taking a customer's data stack  
from "scattered everywhere" to "an Agent can call it safely."

---

## 9.1 The Typical Shape of a Customer Data Stack

The "5 layers" of a customer's data

L1 Operational databases (OLTP)

- PostgreSQL / MySQL / Oracle / SQL Server

- Primary store for business systems

#### L2 Data warehouse / data lake (OLAP)

- Snowflake / Redshift / BigQuery / Databricks / Iceberg
- Where analytics runs

#### L3 Middle layer (Data Mart / Cube)

- dbt models / homegrown views
- The "canonical definition" of business metrics

#### L4 BI / dashboards

- Tableau / Power BI / Quicksight / Looker

#### L5 Business apps / Agents / APIs

- Where data is consumed (most FDE work lives here)

**In Week 1, the FDE should draw a “5-layer data diagram”** and pin down what the customer actually has.

---

## 9.2 Ontology — Unifying What’s Scattered

Ontology is the concept Palantir introduced and popularized. At its core it is a unified definition of **business objects + relationships + properties**.

### The 3 components of an Ontology

#### Object:

Customer / Order / Product / Contract / Employee  
Each object has a "gold-standard definition"

#### Property:

Customer.id, Customer.name, Customer.tier  
Each property has a source-of-truth

#### Relationship:

Customer 1:N Order  
Order 1:N OrderItem  
Order N:1 Salesperson

### The 4 questions an Ontology has to answer

1. Who counts as a "customer"?
  - The CRM record? The paying entity? The contract counterparty?
  - Different departments answer differently → must be unified
2. Which ID is the canonical one?

- The CRM's customer\_id?
  - The contract counterparty's unified\_party\_id?
  - One ID has to be the gold ID
3. How is the mapping built?
- Most of the time it's a hand-curated table
  - There is no silver bullet
4. Who has authority to change the Ontology?
- Without an owner you'll see "A renames a field today,  
B renames it back tomorrow"

## **AWS in practice: a lightweight Ontology with Lake Formation + Glue Data Catalog**

AWS doesn't ship a full "Ontology framework" the way Palantir does, but Glue + Lake Formation give you a workable lightweight version:

Glue Data Catalog as the Ontology registry

Database: customer\_360\_ontology

Table: customer (← business object definition)

Columns:

- customer\_id (string, gold ID)
- source\_systems (struct: crm\_id, erp\_no, finance\_code)
- tier (string)
- tags (LF tags: PII, region=APAC)

Table: order

...

↓

Lake Formation Tags (LF-Tags):

- PII: yes / no
- sensitivity: public / internal / restricted
- region: APAC / EMEA / NA

↓

IAM roles + LF-Tags decide who can query which fields  
on which object

### **Why this works:**

- Ontology metadata is maintained in Glue
- Permissions are maintained via LF-Tags + IAM
- Athena / Redshift / EMR can all query it
- Bedrock Knowledge Bases reads Glue metadata directly

**AWS reference:** search “AWS Lake Formation tags” and “Glue Data Catalog cross-account.”

---

## 9.3 ETL — Making the Data Flow

ETL (Extract / Transform / Load) is the engineering of  $L1 \rightarrow L2 \rightarrow L3$ .

### The 3 engineering signals for ETL

Is the data "fresh enough"? → look at the SLA

- T+1: next-day (covers 90% of projects)
- T+1h: hourly (when latency matters)
- Real-time: seconds (only CDC can deliver this)

Is the data "accurate enough"? → look at quality tests

- Primary key uniqueness
- No nulls where there shouldn't be
- Numeric values within sane ranges
- Business rules (order amount > 0)

Is the data "stable enough"? → look at the dependency graph + monitoring

- When upstream fails, downstream should fail gracefully
- Failures must alert
- Reruns must be idempotent

### ETL tooling cheat-sheet

Scenario → recommended tooling

---

|                          |                                                            |
|--------------------------|------------------------------------------------------------|
| Cloud + Spark family     | Databricks / EMR + Delta<br>AWS Glue (serverless-friendly) |
| Cloud + warehouse-native | dbt + Snowflake / BigQuery / Redshift                      |
| Cloud + streaming        | Kafka + Kinesis Data Streams + Flink                       |
| Self-hosted / offline    | Airflow + Spark + Iceberg                                  |
| Small / simple           | AWS Step Functions + Lambda + S3                           |

### dbt is the FDE' s “Swiss Army knife”

If the customer runs a cloud warehouse, **80% of your ETL will be written in dbt:**

## Standard dbt project structure

---

```
models/
  staging/
    stg_crm_customers.sql      (clean raw)
    stg_erp_customers.sql
  intermediate/
    int_customer_unified.sql  (joins / mappings)
  marts/
    dim_customer.sql          (business-object layer)
    fct_orders.sql

tests/
  not_null_customer_id.yml    (data quality)
  unique_customer_id.yml

macros/
  pii_mask.sql                (reusable logic)
```

Why dbt:

- SQL-only (the FDE doesn't have to learn a new language)
  - Built-in lineage (data lineage visualized for free)
  - Built-in testing (uniqueness / not-null / accepted-values)
  - Git-managed + code review (data engineering becomes engineering)
- 

## 9.4 Data Lineage — The Lifeline of Failure Diagnosis

Without lineage:

"downstream report is wrong" → one person hunts through 5 systems  
for 3 days

With lineage:

"downstream report is wrong" → 5 minutes to see which upstream ETL  
changed schema

### Tools

Open source:

- OpenLineage (supported by dbt / Airflow / Spark)
- Marquez (the OpenLineage backend)

Commercial / cloud:

- DataHub
- Atlan
- AWS Glue (has a lineage view)



- Unity Catalog (Databricks)
- Foundry (Palantir)

### The FDE's minimum bar

Don't try to instrument "the whole company's lineage," but **every dbt model / Glue Job you write must emit lineage**:

1. dbt: lineage is generated automatically (dbt docs serve)
  2. Airflow / Glue: install the OpenLineage hook
  3. When something breaks, the first move is to read the lineage and find the root cause
- 

## 9.5 Real-Time Data Pipelines — Use Sparingly

### When you genuinely need real time

- ✓ Business flow demands sub-second feedback (risk / recommendation / ad bidding)
  - ✓ Decision windows are short (inventory / pricing)
  - ✓ User-perceptible (presence / real-time notifications)
- If 1 of these holds, consider real time
- If none hold → use T+1 and save 70% of the engineering

### The engineering traps in a real-time pipeline

Going real-time on day one → debugging hell

No idempotency → replays produce wrong data

No schema-evolution plan → any upgrade breaks it

No dead-letter queue → one bad record blocks everything

No lag monitoring → you find out from a user complaint

### AWS in practice: the MSK + Kinesis + Firehose trio

Typical AWS real-time pipeline

Producer

↓

MSK (Managed Kafka) or Kinesis Data Streams

↓

Consumer choice:

- A. Lambda processes directly (low traffic)
- B. Flink on EMR / KDA (high traffic + complex logic)
- C. Kinesis Firehose lands directly to S3 / Redshift

↓

Downstream: S3 (Iceberg) / Redshift / OpenSearch

For simple cases use Firehose (auto buffering + compression + S3 partitioning); for complex logic use Flink.

**AWS reference:** search “Amazon MSK best practices,” “Kinesis Data Firehose.”

---

## 9.6 The “Minimum 5 Things” of Data Governance

This is not a “complete data governance system” — it’s the bare minimum the FDE has to put in place on customer site:

1. Data ownership table  
Every table / dbt model has one owner (person + email)
2. PII tagging  
Which fields are PII; they must be tagged and masked in dev
3. Schema-change process  
Adding a field: notify + document  
Changing a type / dropping a field: review + notify downstream
4. Test coverage  
Every primary table has at least 3 dbt tests  
(uniqueness / not-null / referential)
5. Lineage visualization  
You can answer “where did this number come from?” in 5 minutes

**Miss any of the five and the Agent is walking onto a minefield.**

---

## 9.7 A Real End-to-End Example

Customer: an insurance company

Agent's job: automated underwriting (applicant risk assessment)

Data needs (surfaced by the FDE in Discovery):

- Applicant basic info (CRM)
- Historical claims (claims system, Oracle)
- Health declaration PDFs (scans + OCR)
- Blacklist (compliance system, SQL Server)
- Credit score (external API)

→ 5 systems: 4 internal + 1 external

The FDE's engineering plan:

Week 1: Register the 4 internal systems in Glue Data Catalog  
Build a customer Ontology (unified customer\_id mapping)  
Week 2: Write dbt models to merge the 4 systems' customer data  
into customer\_360  
Apply LF-Tags (mask PII fields)  
Week 3: Lambda + EventBridge to pull external API credit scores  
Week 4: Bedrock Agent calls Athena to query customer\_360  
+ calls Lambda for credit + calls OCR  
Weeks 5-6: Eval + canary

Key engineering moves:

- No real-time pipeline (Discovery confirmed T+4h was acceptable)
  - Glue Data Catalog as the Ontology (lightweight)
  - All PII fields masked in dev
  - Every dbt model has an owner + tests
- 

## Key Quotes

“The Ontology is the contract between data engineering and the rest of the company.” — Palantir Blog, On Ontology, 2024

“Most LLM hallucinations are actually data quality problems wearing a costume.” — A. Lawrence, FDE Rule Book, 2025

“If you can't explain where the number came from, the customer can't trust the answer.” — AWS GenAI Innovation Center, 2025

---

## Action Checklist

When you walk into a data-heavy FDE project, weeks 1-2 must include:

1. **Draw the customer's “5-layer data diagram”** (§ 9.1)
  2. **Identify the source-of-truth for 3 core business objects** (customer / order / product)
  3. **Stand up a Glue Data Catalog database** (even for a PoC)
  4. **Tag every table with owner + LF-Tags (PII / region / sensitivity)**
  5. **Write a unified view like customer\_360 in dbt**
  6. **Wire up OpenLineage** (dbt has it built in)
  7. **Decide the SLA: T+1 / T+1h / real-time** (default to T+1; if T+1 works, ship T+1)
- 

## Anti-Pattern Checklist

- **Skipping Ontology and plugging an Agent straight in** (the Agent will confidently splice inconsistent data into wrong answers)

- **Connecting 5 systems directly without unification** (any small upstream change breaks everything)
  - **Tables without owners** (a schema change ships and nobody downstream is told)
  - **Real-time pipeline in v1** (10x debugging cost, often not worth it)
  - **Real PII in dev** (the most common compliance incident)
  - **dbt project with no tests** (you only learn about bad data when downstream complains)
  - **No lineage** (data-issue triage goes from 5 minutes to 5 days)
- 

## Bridge to the Next Chapter

You have the data stack — but most customers won't let you use cloud “out-of-the-box” services. The data has to live inside the customer's VPC / private deployment / offline data center. The next chapter covers the FDE's engineering moves inside network-isolated environments.

[← Part IV intro](#) · [Next: Working in the Customer's VPC](#) →

# Chapter 10: Working in the Customer's VPC / Private Deployment / Offline Data Center

## Opening

A Chinese state-owned enterprise. Day one in the customer's data center:

Customer security lead: "Our rules —

1. All code lives on our internal GitLab
2. No ChatGPT, no Google
3. No internet; pip install goes through our private PyPI mirror
4. Servers are SSH-via-bastion only, no direct internet
5. Data does not leave our data center
6. Your Macs cannot connect to our network  
— use this Windows workstation we provided
7. The workstation cannot install non-whitelisted software  
(and the whitelist does not include VS Code's GitHub Copilot extension)"

The FDE is thinking: "What can I actually do?"

A month later her summary: "Turns out you can do anything, the rhythm is just completely different — it isn't the SaaS-workflow rhythm. The trick is being clear about what you can do and what has to be Air-gapped."

This chapter covers the FDE's concrete engineering moves inside a customer VPC / private deployment / offline data center.

---

## 10.1 Three “Non-Cloud-Native” Customer Scenarios

Non-SaaS customer deployments

---

Scenario A: Customer VPC (cloud, but isolated)

- Customer has their own VPC on AWS / Azure / Aliyun
- Internet is gated through controlled NAT egress
- Strict security groups + NACLs
- Your code is deployed inside the customer's account

Scenario B: Customer private cloud / on-prem data center

- Customer runs their own K8s cluster / VMware
- Some hosts have internet, some are pure intranet
- Your code is deployed onto customer-owned machines

Scenario C: Air-gap (fully offline)

- No internet at all
- Models / images / dependencies arrive via USB / bastion
- Defense / military / parts of finance

90% of B2B customers fall into A or B, with A the most common.

---

## 10.2 Scenario A: Working Inside the Customer' s VPC

### Key engineering moves

1. Get an IAM role (not root — a scoped role)
2. Sign in via AWS Identity Center / customer SSO
3. Deployment target lives inside the customer's account + VPC
4. Between your dev machine and the customer VPC, traffic goes through
  - VPN (Site-to-Site / Client VPN)
  - Bastion (Bastion Host / Session Manager)
  - Never direct

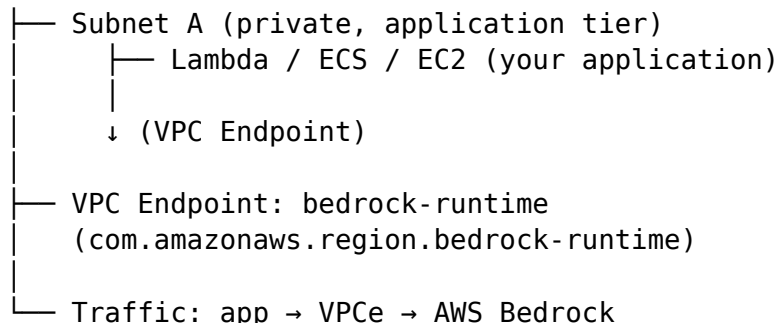
### AWS in practice: deploying a Bedrock app inside the customer' s VPC

The single most important property is “Bedrock traffic does not leave the customer' s VPC.” Configuration:

Customer VPC + Bedrock private path

---

Customer VPC



(does not leave the VPC, does not touch the internet)

Why a VPC Endpoint is required:

- The customer's security lead: "Every call must be auditable and blockable"
- VPC Endpoint = AWS private network = no public exposure
- CloudTrail + VPC Flow Logs make the whole path auditable

Minimum config:

```
# 1. Create the VPC Endpoint
aws ec2 create-vpc-endpoint \
  --vpc-id vpc-xxx \
  --service-name com.amazonaws.us-east-1.bedrock-runtime \
  --vpc-endpoint-type Interface \
  --subnet-ids subnet-aaa subnet-bbb \
  --security-group-ids sg-yyy

# 2. The application calls Bedrock via SDK
import boto3
client = boto3.client('bedrock-runtime')
# The SDK routes through the VPCe automatically
# (DNS resolves to the VPCe's private IP)
```

**AWS reference:** search "Bedrock VPC endpoints" and "AWS PrivateLink for Bedrock."

## Knowledge Bases / Agents go private too

Bedrock Knowledge Bases

- VPC Endpoint: bedrock-agent
- Data source S3: VPC Endpoint: s3 (Gateway type)
- OpenSearch Serverless: collection inside the customer's VPC

Bedrock Agents

- VPC Endpoints: bedrock-agent + bedrock-agent-runtime
- Lambda tools: inside the customer's VPC
- Customer internal API calls: invoked directly within the VPC

## SageMaker JumpStart — the fallback for self-hosted models

If the customer won't even let you use Bedrock ("must be self-hosted"), use SageMaker JumpStart:

SageMaker JumpStart one-click deploy:

- Llama 3.1 / Qwen / Mistral images
- Deploys to a SageMaker endpoint inside the customer's VPC
- Private endpoint, no public exposure
- Entirely inside the customer's account

## 10.3 Scenario B: Customer Private Cloud / On-Prem Data Center

### A typical day

A typical day

---

7:30 Arrive at the customer's building, badge + face scan + bag X-ray  
8:00 Up to the FDE work area on the second floor of the data center  
8:30 Boot the customer-issued Windows workstation  
- No Outlook (use the customer's webmail)  
- No Slack (use the customer's internal comms tool)  
- VS Code installed, but no Copilot  
9:00 ssh through the bastion → customer K8s  
- First check overnight alerts  
10:00 Negotiate today's deployment window with customer ops  
...  
17:00 Push today's code to the customer's GitLab  
SAST / DAST scan runs on push  
18:00 Write the daily report and head home

### Key engineering configuration

Code hosting:

- Customer-internal GitLab / Gitea / Gerrit
- No pushes to GitHub

CI/CD:

- Customer-internal Jenkins / GitLab CI / ArgoCD
- No GitHub Actions / CircleCI

Image registry:

- Customer-internal Harbor / Quay
- Base images (Python / Java) must come from the customer's mirror

Dependency management:

- pip / npm / maven point to the customer's private repository
- You must produce a manifest; the security lead will scan it

Model storage:

- Model weights live in the customer's object store (MinIO / Ceph / OSS)
- You cannot pull from HuggingFace directly

Monitoring:

- The customer's Prometheus + Grafana
- You cannot connect Datadog / NewRelic



## How a model “gets in”

The flow for bringing model weights inside

Step 1: From the corporate side, the FDE downloads the weights from HuggingFace (Llama / Qwen / Mistral)

Step 2: Virus scan + license check + security audit

Step 3: Run the customer's "software-admission process"

- One "model admission application"
- Model description + license + scan report
- Wait for approval (1-3 weeks, varies)

Step 4: After approval, the customer permits USB or private-line transfer

Step 5: Land in the customer's object store + register in their model repo

**The FDE must trigger this process in Week 1.** Otherwise you’ ll hit Week 6 only to discover the model still hasn’ t arrived = project slips.

---

## 10.4 Scenario C: Air-Gap (Fully Offline)

### A real story

An FDE deployed an LLM at a fully offline customer. Day one she discovered

the company-issued laptop’ s git still had a GitHub remote. She ran git push without changing the origin. The customer’ s security lead replied within seconds: “Your laptop is now quarantined for 24 hours of observation.”

The defining traits of Air-gap:

- ✓ No internet at all
- ✓ All code / tooling / images come in on USB or via a trusted ferry
- ✓ Traffic between your local machine and customer machines goes through a "ferry box"
- ✓ Every action that enters the customer network is audited

### The first three days’ checklist

Day 1: Build the USB package

- Python / Node / Docker images
- All your code + dependencies (pip download / docker save)
- Model weights
- VS Code + offline extensions

- Documentation (RFCs / designs / data dictionaries)

Day 2: USB security check + admission

- The customer audits your USB
- The customer installs it on their internal ferry machine
- Ferry scan + quarantine observation + delivery into the intranet

Day 3: Internal-network PC receives the materials, work begins

## Engineering discipline

- ✓ Every prompt lives in the repo (no "I'll figure that out later")
- ✓ Don't assume ChatGPT / Claude is around to help debug
- ✓ Bring your own "offline reference":
  - HuggingFace cache
  - PyPI mirror
  - Offline docs (pip download docs)
- ✓ For features that keep breaking → simplify the design, no exotic deps
- ✓ At the end of each week, batch the latest code back to corporate (review + backup)

---

## 10.5 Engineering Notes for Private LLM Deployments

Not cloud SaaS LLMs — **self-hosted Llama / Qwen / Mistral**:

### Pick the model size

GPU configuration → recommended model

---

|             |                                          |
|-------------|------------------------------------------|
| 4× A100 80G | Llama 3.1 70B half precision / Qwen2 72B |
| 2× A100 80G | Llama 3.1 70B AWQ 4-bit / 32B models     |
| 1× A100 80G | Llama 3.1 8B / Qwen2 14B                 |
| 1× A100 40G | 7B / 8B models                           |
| No GPU      | Don't force an LLM; use a cloud API      |

### Inference engines

vLLM (recommended)

- PagedAttention, memory-efficient
- OpenAI-compatible API
- High QPS

TGI (HuggingFace)

- Easy to start
- Good streaming output

TensorRT-LLM (NVIDIA)

- Peak performance
- Long ramp-up

llama.cpp

- Runs on CPU / Mac
- Tiny footprint

## Deployment architecture

Standard K8s + vLLM architecture:

---

Ingress (internal LB) → API Gateway (auth + rate-limit)

↓

vLLM Pods (HPA)  
(1 GPU per pod)

↓

Model weights (PVC / S3 / OSS)

Monitoring: Prometheus + Grafana (token/sec, GPU util)

Logging: Loki / customer ELK

## Privatizing Eval / Trace

Cloud SaaS:

Private deployment:

|                |   |                              |
|----------------|---|------------------------------|
| LangFuse Cloud | → | LangFuse self-hosted         |
| LangSmith      | → | Phoenix (Arize, open source) |
| Bedrock Eval   | → | Custom Python scripts + dbt  |

---

## 10.6 Compliance Checklist (private deployment / common in China B2B)

Compliance cheat-sheet

---

Cross-border data:

- Chinese customers' data cannot leave the country
- Model weights can usually be imported (depends on license)

China MLPS (等保):

- MLPS 2.0 Level 3 / Level 4 = many constraints
- Audit + deployment process is what affects FDE work most

GDPR / SOC 2:

- Data classification + access auditing
- PII protection

PCI DSS (financial):

- Payment data encryption + network isolation

HIPAA (healthcare / US):

- PHI data + audit trail

CSA STAR (enterprise cloud):

- Cloud-provider qualification

**The FDE’ s strategy:** compliance is not something one FDE can solve, but **the FDE has to surface these requirements during Discovery**. Otherwise you’ ll get to deployment, find out “X isn’ t allowed,” and the whole architecture has to be redone.

---

## Key Quotes

“The customer’ s network is the customer’ s contract — disrespect it once, lose the project.” — A. Lawrence, FDE Rule Book, 2025

“Architecture changes when the GPU lives in the customer’ s data center.” — Palantir FDE training, 2024

“PrivateLink is the most underused service among new AWS users.” — AWS Solutions Architects, 2025

---

## Action Checklist

When you start a non-SaaS customer project, Week 1 must include:

1. **Draw a network topology diagram:** customer VPC / private cloud / Air-gap; how traffic enters and leaves
  2. **Get your IAM / SSO identity** (don’ t borrow someone else’ s account)
  3. **Stand up VPC Endpoints** (at minimum: Bedrock + S3 + CloudWatch)
  4. **Trigger the software-admission process** (model / tools / dependencies in one batch — earlier is better)
  5. **Run a “hello world” by Friday** (a simple application calling Bedrock or a self-hosted model from inside the customer’ s VPC)
  6. **Draw a data-flow diagram:** which data may enter, which may not, with PII tags
  7. **Get 30 minutes of coffee with the customer’ s security lead** (this person decides whether the next 12 weeks go smoothly)
-

## Anti-Pattern Checklist

- **Develop locally with SaaS APIs and switch to private only at deploy** (behavior diverges sharply; you discover too late that it can't ship)
  - **No VPC Endpoint, egress via NAT** (compliance audit finding #1)
  - **Install software before applying for admission** ( “I’ ll just install it” gets purged within seconds)
  - **Download customer data to your laptop for analysis** (compliance incident #1)
  - **Push code to GitHub** (the customer forbids external repos)
  - **At an Air-gap customer, expecting ChatGPT to help write code** (24-hour quarantine)
- 

## Bridge to the Next Chapter

Data and network are sorted. The last front is integration with the customer’ s existing systems — SSO / SCIM / API / audit. The next chapter covers the engineering patterns for plugging into legacy systems.

[← Previous: The Customer Data Stack](#) · [Next: Integrating with Legacy Systems →](#)

# Chapter 11: Integrating with Legacy Systems — SSO / SCIM / API / Audit

## Opening

An insurance customer. The FDE demos successfully in Week 8, gets ready to ship in Week 10. The customer's IT director asks four questions; the FDE can't answer three of them on the spot.

- Q1: Does login go through our AD or through your own accounts?
- Q2: When an employee leaves, how does Agent access get revoked automatically?
- Q3: When the Agent calls our ERP API, who decides the permission boundary?
- Q4: If something goes wrong, can the audit log trace it to a specific call?

The FDE goes back, writes those four questions on the whiteboard, and realizes — SSO / SCIM / authorization / audit are 4 different things, and none of them is something you can "patch in right before launch."

She replans:

- Week 11: SSO + SCIM
- Week 12: authorization boundaries + audit
- Launch slips by 2 weeks

Her manager: "Should have done this 4 weeks earlier."

She says: "Now I know."

This chapter is the "do it 4 weeks earlier" checklist.

---

## 11.1 How the Four Things Relate

Identity (Authentication)

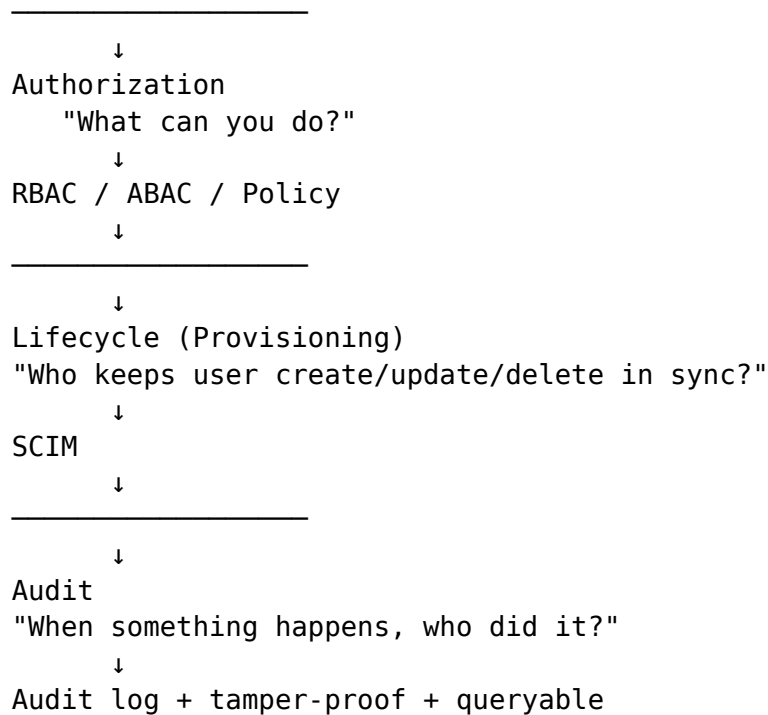
"Who are you?"

↓

SSO (Single Sign-On)

OAuth / SAML / OIDC

↓



The four interlock. Miss one and an enterprise launch will not pass review.

---

## 11.2 SSO — Plugging into Customer AD / Okta / Identity Center

### Protocol cheat-sheet

#### SAML 2.0

- The default for legacy enterprise IdPs (AD FS / Okta / OneLogin)
- XML-based, complex but stable

#### OIDC (OpenID Connect)

- The identity layer on top of OAuth 2.0
- JSON / JWT, modern-friendly
- Recommended for new projects

#### Kerberos

- The internal protocol of Windows AD
- Common on large-enterprise intranets in China

#### LDAP

- Not SSO; it's a directory query protocol
- But many customers use it interleaved with the above

## Sample flow (OIDC)

### Typical OIDC login flow

1. User visits your Agent
2. Your app redirects to the customer IdP (AD / Okta)
3. Customer IdP authenticates the user (password / SMS / Yubikey)
4. IdP returns ID token + Access token
5. Your app parses the token:
  - sub: user ID
  - email: email address
  - groups: department / role
6. Use the token to call your backend / Agent
7. The backend uses group / role for authorization

## AWS in practice: IAM Identity Center + Bedrock Agent

### Three IAM Identity Center integrations

Pattern 1: Customer brings their own IdP (Okta / Azure AD)

- Sync users into Identity Center via SAML / SCIM
- Identity Center assigns IAM roles to users

Pattern 2: Customer uses AD (Active Directory)

- AD Connector or AWS Managed AD
- Users sign in to AWS with their AD account

Pattern 3: Customer has no IdP (small org)

- Identity Center's built-in user management (not recommended)

Minimum config:

1. Enable Identity Center (at the Organization level)
2. Connect the customer IdP:
  - Exchange SAML metadata
  - Attribute mapping (email, groups)
3. Auto-sync via SCIM:
  - Customer IdP create/update/delete → automatically synced to Identity Center
4. Permission Sets:
  - "BedrockAgentUser" → only invoke a specific Agent ID
  - "BedrockAgentAdmin" → can modify Agent configuration
5. Agent invocation:
  - User → your app → STS AssumeRoleWithSAML



→ temporary credentials → `invoke_agent` (with `session_attributes`)

**AWS reference:** search “AWS IAM Identity Center external IdP” and “Bedrock Agent identity context.”

---

## 11.3 SCIM — User Lifecycle Sync

### Why SCIM is needed

Without SCIM:

Employee leaves in the customer HR system

- manual ticket to IT
- IT manually deletes the account in 5 systems
- your Agent is the 6th system, and it gets forgotten
- the ex-employee can still use the Agent to call ERP

With SCIM:

HR system records the leave (Workday / SAP HCM)

- IdP auto-syncs (Okta / Azure AD)
- SCIM protocol → your system
- your system disables the account in seconds

### The SCIM protocol

At its core: REST API + JSON Schema

Endpoints (you implement):

- |                           |               |
|---------------------------|---------------|
| POST /scim/v2/Users       | (create user) |
| PATCH /scim/v2/Users/:id  | (update user) |
| DELETE /scim/v2/Users/:id | (delete user) |
| GET /scim/v2/Users/:id    | (read user)   |

Bidirectional sync:

Push: IdP → your system (create / update / delete)

Pull (optional): your system → IdP (reverse sync)

### FDE practical notes

- ✓ Prefer off-the-shelf
  - AWS IAM Identity Center ships a SCIM endpoint
  - Auth0 / Okta SCIM templates
  - Don't write an identity system from scratch
- ✓ Mandatory test cases
  - create → log in → change group → verify permissions changed
  - delete → verify cannot log in

- ✓ Log every SCIM call
    - Customer audit will ask "when was this employee disabled?"
- 

## 11.4 API Integration — Calling the Customer's Legacy Systems

Four "interface shapes" of legacy systems

---

Shape 1: REST / OpenAPI (modern)

- Call directly
- Usually OAuth 2.0 / API Key auth

Shape 2: SOAP / WSDL (traditional enterprise)

- XML envelopes
- Heaps of enterprise middleware
- In Python: the zeep library

Shape 3: Direct database access (anti-pattern but common)

- Customer hands you a read-only Oracle / SQL Server account
- High risk; avoid if you can

Shape 4: File drops (oldest)

- SFTP shared folders
- CSV / XML on a schedule
- Business-critical time windows

### Design pattern: how an Agent calls legacy systems

Agent → Tool → Adapter → legacy system

---

Agent (Bedrock / LangGraph)

↓ (function call)

Tool (Lambda / your code)

↓ (uses the SDK or a customer-provided client)

Adapter (a thin translation layer)

↓

Legacy system (SAP / Oracle / SOAP)

Why an Adapter:

- The legacy protocol does not leak into the Agent
- Agent upgrades / legacy upgrades stay decoupled
- One place for fallback / retry / circuit breaker

## Four engineering signals

- ✓ Always call a read API first
    - Write APIs need customer sign-off before they ship
  - ✓ Always use an idempotency key
    - Retrying the same operation must not double-execute
  - ✓ Always have timeout + circuit breaker
    - When the legacy system dies, it must not drag you down with it
  - ✓ Always have a dry-run mode
    - Demos / tests must not actually write
- 

## 11.5 Audit Logs — “Can You Prove It?”

The 5 questions in a compliance audit

- Q1: Who did what?
  - user\_id + action + resource
- Q2: When?
  - timestamp (UTC, microseconds)
- Q3: From where?
  - IP + User-Agent + session\_id
- Q4: With what result?
  - success / failure + summary of returned content
- Q5: Who was authorized to do this at the time?
  - role / policy snapshot at that moment

## Four engineering requirements for audit logs

1. Immutable
  - WORM (Write Once Read Many) storage
  - S3 Object Lock / CloudTrail
2. Correlatable
  - trace\_id stitches together all related logs
  - From user action → API call → DB query → model call
3. Queryable
  - "Stored" is not enough; you must be able to "query it back out"

- Recommended: CloudWatch Logs + Athena queries

#### 4. Retention

- Compliance typically demands 90 days / 1 year / 7 years (industry-dependent)
- Don't forget archival (S3 Glacier)

### AWS in practice: CloudTrail + CloudWatch + Athena

Typical audit architecture for an FDE project

Application → app logs (with trace\_id)

↓

CloudWatch Logs (1 week → archive to S3)

↓

Athena (query by trace\_id)

AWS API → CloudTrail (every AWS API call)

↓

S3 (Object Lock + KMS encryption)

↓

Athena queries / Security Hub alerts

Bedrock invocations → Bedrock Model Invocation Logging

↓

CloudWatch / S3

↓

Audit: "What did user X ask the Agent at time T, and what did it answer?"

Minimum switches to flip:

1. CloudTrail Multi-Region Trail (mandatory on)
  - All management events
  - Data events for S3 / Lambda (as needed)
  - Insight events (anomaly detection)
  - Add Object Lock to prevent deletion
2. Bedrock Model Invocation Logging
  - Bedrock console → Settings → Enable
  - Destination: CloudWatch + S3
  - Captures full prompt + response
3. Application logging conventions:
  - Every log line carries trace\_id (X-Ray / OpenTelemetry)
  - user\_id (taken from the IAM context)
  - action + resource\_arn

**AWS reference:** search "CloudTrail data events," "Bedrock model invoca-

tion logging,” “S3 Object Lock.”

---

## 11.6 Guardrails — Auditing the Agent’s Behavior

It isn’t only humans auditing the Agent; the Agent itself needs real-time inspection:

Bedrock Guardrails: 5 categories of intercept

1. Content filter (harmful / violent / hateful / sexual content)
2. Topic filter (forbidden topics)
3. PII filter (national IDs / phone numbers / emails auto-redacted)
4. Word filter (company-banned words / competitor names / internal codes)
5. Contextual grounding (answers must be grounded in the knowledge base)

### Configuration approach

PoC stage:

- ✓ Turn on PII filter
- ✓ Turn on Content filter (medium strength)

Pre-production:

- ✓ Add Topic filter (business red lines)
- ✓ Add Word filter (company-confidential terms)
- ✓ Add Grounding (mandatory for any RAG application)

Post-launch:

- ✓ Monitor Guardrail intercept counts
- ✓ Too many intercepts → adjust thresholds or rewrite prompts
- ✓ Too many leaks → add rules

**AWS reference:** search “Bedrock Guardrails.” It can be used independently of a model, or attached to an Agent.

---

## 11.7 A Complete Integrated Example

Customer scenario: an insurance company's sales-assistant Agent

Identity layer:

- Sales reps SSO in with their AD account
- Identity Center + SAML connects to customer AD

Lifecycle:

- HR offboards → AD disables → SCIM syncs → Agent is unavailable instantly

#### Authorization layer:

- "Sales rep" role: can only see their own customers' policies
- "Manager" role: can see policies across the team
- Implemented with ABAC: tag-based access control

#### Agent calling legacy systems:

- Calls ERP (SOAP) → Lambda Adapter → SAP RFC
- Calls CRM (REST) → Lambda → Salesforce API
- Every call carries user context (not a service account)

#### Audit:

- CloudTrail: AWS API
- Bedrock Logging: prompt + response
- App logs: trace\_id stitches everything together
- Audit window: 7 years (insurance industry)
- Storage: S3 Object Lock + KMS

#### Guardrails:

- PII filter: customer national ID / bank card auto-redacted
  - Topic filter: no investment advice (compliance requirement)
  - Grounding: answers must be based on product documentation
- 

## Key Quotes

“Identity is the new perimeter — and it’s the first thing the customer’s CISO will ask about.” — A. Lawrence, FDE Rule Book, 2025

“Audit is not a feature — it’s the only proof you have when something goes wrong.” — AWS Well-Architected, 2025

“If your Agent can’t tell who’s asking, it shouldn’t answer.” — Anthropic enterprise best practices, 2025

---

## Action Checklist

When you start an enterprise project, weeks 1-2 must include:

1. **Draw a “four-things diagram”** : SSO / SCIM / authorization / audit, current state of each
2. **Get 30 minutes with the customer’s security lead**: which IdP they use, MLPS level, audit requirements
3. **Get an SSO test account** (SAML metadata or OIDC client credentials)
4. **Wire up Identity Center + SCIM** (do not write an identity system from scratch)
5. **Enable CloudTrail + Bedrock Invocation Logging in week one**

6. **Every Agent tool that calls a legacy system goes through an Adapter** (no direct connections)
  7. **At minimum, configure Bedrock Guardrails with PII + Content filters**
  8. **Write an “audit-log query SOP”** so you can hand it over the moment a customer audit shows up
- 

## Anti-Pattern Checklist

- **Running every call under a service account** (audit can't trace it back to a specific user)
  - **Departed employees not auto-disabled on your side** (compliance incident)
  - **Agent talks SOAP directly to a legacy system** (tightly coupled, no fallback)
  - **Audit logs that operations staff can delete** (instant audit fail)
  - **Unredacted PII flowing into Bedrock invocation logs** (GDPR / MLPS incident)
  - **No dry-run mode for write APIs** (a demo nukes production data)
  - **Postponing Guardrails** (one incident costs far more than the configuration time)
- 

## Bridge to the Next Part

The full foundation for “the FDE working in a real customer environment” is now in place: data stack, network isolation, identity and audit. The next Part crosses the hardest chasm of all — **PoC → production**: how to actually ship stably after the demo, control cost, and run canaries with rollback.

[← Previous: Working in the Customer's VPC](#) · [Next Part: PoC → Production →](#)

# Part V: PoC → Production — Crossing the Hardest Gap

Applies to: Both tracks (both main lines have to clear this gap)

---

## What This Part Solves

The distance between a working PoC and going to production is greater than the work of building the PoC itself.

OpenAI has an internal saying: > “From PoC to production is 6 weeks of joy followed by 6 weeks of yak shaving.”

The industry-average PoC-to-production conversion rate is about 30–40%. The AWS GenAI Innovation Center has pushed that number to 73% — not by using better models, but by **embedding production engineering moves into the PoC stage itself**.

This Part has two chapters:

- **Chapter 12:** PoC pass-line conditions — the 6 things that let a PoC actually reach production, and the 4 things that will stall it dead.
- **Chapter 13:** Observability / cost / canary / rollback — the production “four-piece set.”

## Chapters

See [SUMMARY.md](#).

## Relation to Other Parts

- **Prerequisites:** Eval-driven from Part III, and data + integration from Part IV, are inputs to this Part.
  - **Follow-on:** Part VI adds Agents on top of the production environment; Part VII completes delivery and wrap-up.
-



# Chapter 12: PoC Pass-Line Conditions — What Makes It, What Stalls Out

## Opening

A SaaS company. The FDE delivers a PoC in week 6, and the results are stunning:

- Customer CEO applauds at the demo
- Eval composite score 0.91
- The press even covers it

Week 7, the customer's IT director asks: "How do we get to production?"  
The FDE answers: "Just deploy this demo."

Week 14, still not live.  
What's blocking it?

- No staging environment (the demo was a modified dev)
- No deploy scripts (the demo was started by hand)
- No load tests (1 request per second is the ceiling)
- Not wired into customer monitoring (ops doesn't know how to on-call)
- Cost budget not approved (\$20K/month, leadership hadn't planned for it)
- No canary plan (only "all or nothing")

Postmortem: during the PoC stage, the "6 things for productionizing" were not done in parallel.  
The flashier the PoC, the harder the path to production.

This chapter gives: the 6 things to do during the PoC stage, plus the 4 signals that "guarantee a stall."

---

## 12.1 PoC vs Production — Engineering Bars Differ by 100x

PoC bar

Production bar

|                 |                       |                           |
|-----------------|-----------------------|---------------------------|
| Concurrency     | single user / a few   | 100–10K QPS               |
| Availability    | "good enough to demo" | 99.5% / 99.9%             |
| Latency         | P50 of 5s acceptable  | P95 < 3s                  |
| Fault tolerance | "restart on error"    | auto-retry + isolation    |
| Observability   | console.print         | full-trace tracing        |
| Permissions     | FDE's admin in hand   | RBAC + audit              |
| Data            | "give me 50 rows"     | full + real-time          |
| Cost            | boss didn't say       | monthly budget control    |
| Releases        | "I edit the prompt"   | CI/CD + canary            |
| Customer        | FDE always there      | customer ops runs it solo |

90% of PoC failures aren't technical — they're about how far the "engineering bar" is from production.

## 12.2 The 6 Things to Do During the PoC Stage

### Thing 1: Environment Tiers (dev / staging / prod)

Split from day 1:

|          |                                      |
|----------|--------------------------------------|
| dev:     | FDE's workbench, change anything     |
| staging: | same config as prod, runs regression |
| prod:    | customer ops manages, strictest      |

Even at PoC stage, staging is a must  
 Otherwise: "no one remembers the prompt change from the demo"  
 → can't find it at launch

### Thing 2: Wire Up CI/CD in Week 1

Minimum CI/CD pipeline:

on push to feature branch:

- lint
- unit test
- eval (50 seed cases)

on PR to main:

- eval (200 golden cases)
- integration test
- cost estimate

on merge to main:

- auto-deploy dev
- trigger staging deploy

on tag release:

- deploy staging
- wait for human sign-off
- canary deploy prod (10% → 50% → 100%)

A PoC without CI/CD shouldn't enter PoC.

### Thing 3: Wire Up Monitoring / Trace in Week 1

4 must-have trace dimensions:

1. Latency (per step)
2. Cost (input/output tokens, model)
3. Quality (eval / user feedback)
4. Error (stack trace + correlation\_id)

Tooling (review Ch 6):

- Cloud: LangFuse Cloud / LangSmith / Bedrock built-in
- Private: LangFuse self-host / Phoenix

### Thing 4: Cost Transparency (Start Reporting at PoC Stage)

Weekly report must include:

- This week's token spend (input + output)
- Distribution by model
- Distribution by scenario (RAG / Agent / Eval)
- Average cost per request
- Monthly extrapolation

Start at PoC stage → no "we burned \$50K in the first week and the boss is shocked"

## Thing 5: Customer Ops “On the Bus”

Starting in week 4:

---

- At least 1 customer ops engineer joins the project Slack
- On-call together (even if not real on-call during PoC)
- Watch the dashboards together
- Handle alerts together

Not on the bus: at launch, customer ops "takes over" =  
starts from zero, 3 months to be self-sufficient

On the bus: at launch, customer ops is already familiar,  
handoff in 1 week

## Thing 6: Degradation Plan (Fallback)

Critical paths must have a fallback:

---

Bedrock model down → switch to backup model (cross-region)  
Knowledge Base down → switch to cached embeddings  
Agent tool down → reject request + friendly error  
All down → static reply + ticket created

At PoC stage at minimum:

- ✓ Write one "everything down" backstop
  - ✓ Run one "everything down" drill
- 

## 12.3 The 4 Signals of “Guaranteed Stall”

### Signal 1: No One on the Customer Side Wants to Be Owner

Projects where "who owns this" has no answer:

- x Business says "this is IT's problem"
- x IT says "this is the business's problem"
- x Leadership says "both sides own it together"

- After launch: no one to on-call, no one to review prompt changes,  
no one to update the KB
- "Natural death" within 6 months

**FDE’ s move:** Hard-write “customer-side owner” into the SOW. No owner, no kickoff.

## Signal 2: Eval Set Is Written Solo by the FDE

FDE writes 200 Eval cases alone →  
After launch, customer business experts say  
"these aren't the questions we care about"  
→ Eval set fully redone  
→ Launch slips 2–4 weeks

Move: Starting in week 3, pull in customer business experts  
to label together.  
FDE writes v1 → business experts review → re-label

## Signal 3: Cost Calculated Only at Month-End

Demo stage: \$500/month  
Week 1 of launch: \$5K  
Customer leadership sees the bill: "We can't run this. Cut it."

Move: In PoC week 2, give leadership the first cost projection.  
Before launch, give a "cost curve at 3 different scales."  
After launch, weekly cost dashboard.

## Signal 4: Customer IT Did Not Attend Any PoC Review

6-week PoC entirely with the business side, zero IT involvement.  
At launch, IT vetoes:  
"VPC Endpoint not wired up"  
"Audit logging doesn't meet our bar"  
"Won't pass our compliance review"  
→ Tear down, start over.

Move: Bring IT in as early as week 1 Discovery.  
IT-FDE sync every two weeks.  
Starting in week 4, IT attends demos.

---

## 12.4 The 5 Hard Metrics for PoC Pass-Line

Write these 5 into the SOW. Miss any of them, no production:

| Metric                   | Pass-Line Condition                      |
|--------------------------|------------------------------------------|
| Eval composite score     | $\geq$ SOW-agreed value (typically 0.85) |
| Production-scenario load | P95 < 3s, 100 QPS stable for 30 min      |
| Cost                     | Cost per request $\leq$ budgeted X       |
| Audit                    | 100% of calls logged to CloudTrail       |

|             |                                 |
|-------------|---------------------------------|
| Canary plan | Can shift 1% / 10% / 50% / 100% |
|-------------|---------------------------------|

5 of 5 pass → enter canary production. Miss 1 → defer.

---

## 12.5 A Real PoC → Production Timeline

Typical 12-week project timeline

---

- W1-2 Discovery (Part II)
  - 5 deliverables: report / Eval seed / SOW
  - Customer owner / IT onboarded / security review
- W3-6 Scaffolding (Part III)
  - 6.7 quick-decision-table baseline
  - Eval CI wired up (Ch 8)
  - First demo
- W6 Mid-PoC checkpoint
  - First run of the 12.4 hard metrics
  - Which ones miss?
- W7-9 Productionize Phase 1
  - Data integration + VPC + SSO + audit (Part IV)
  - Monitoring + cost + canary prep (Ch 13)
- W10 Final PoC checkpoint
  - All 5 hard metrics from 12.4 pass
  - Three-way sign-off: customer IT + business + leadership
- W11 Canary rollout (10% → 50%)
- W12 100% live
  - + Handoff (Ch 16) kicks off

Pacing matters more than feature completeness.

---

## 12.6 A Real Counter-Example

An internet company's FDE team built an 8-week LLM customer-service PoC. In week 8 the customer CEO saw the demo, was satisfied, and was ready to sign a 12-month contract.

During the PoC, the following were not done: - Customer SSO not integrated (used a service account) - CloudTrail not wired up (said “we’ll add it after launch”) - No load testing (demo was at 1 QPS) - No architecture sync with customer IT (IT first saw the diagram right before signing)

After signing, the IT department spent 3 weeks reviewing the architecture and filed 47 change requests. Rework took 5 weeks.

Customer business side: “You promised an 8-week launch — why are you still iterating at week 16?”

The FDE team lost the next-phase renewal.

**Postmortem:** During a PoC, **business-side approval  $\neq$  project ready to ship**. If IT / security / compliance / finance — any one of them — is not satisfied, the project can be torn down and restarted.

---

## Key Quotes

“The PoC is not the project — it’s the trailer for the project.” — A. Lawrence, FDE Rule Book, 2025

“A 73% PoC-to-production conversion rate doesn’t come from better models. It comes from running PoC like it’s already production.” — AWS GenAI Innovation Center, 2025

“Every PoC has a critic — find them in week 1, not week 8.” — Bob McGrew @YC, 2025

---

## Hands-On Checklist

In week 1 of a PoC, do these:

1. **Write the “5 hard pass-line metrics” into the SOW** (section 12.4).
  2. **On day 1, split dev / staging / prod into three environments.**
  3. **Wire up CI/CD in week 1** (Eval + lint + deploy).
  4. **Wire up trace + cost dashboard** (CloudWatch or LangFuse).
  5. **Pull IT / security / finance / business owners into the project channel.**
  6. **Starting in week 4, customer ops watches dashboards alongside the FDE.**
  7. **In week 6, run a “pretend launch” drill** (cost projection + load test + failure drill).
- 

## Anti-Patterns

- **Treating the PoC as a “demo build”** (everything has to be redone for prod).
- **Demoing straight from dev** (no staging).

- **No monitoring / audit / SSO during the PoC** (saying “we’ ll add it after launch” ).
  - **Customer IT not in the project channel** (vetoed at the last minute).
  - **Cost calculated only after launch** (leadership doesn’ t approve, project dies suddenly).
  - **No canary plan, only 0% / 100%** (no graceful rollback when things go wrong).
- 

## Relation to the Next Chapter

This chapter gave the “5 hard metrics for PoC pass-line.” The next chapter goes into the engineering practice of the production “four-piece set” : observability / cost / canary / rollback.

[← Part V Intro](#) · [Next: Observability / Cost / Canary / Rollback](#) →



# Chapter 13: Observability / Cost / Canary / Rollback

## Opening

A retail customer's Agent, day 3 of production, 2 AM:

P1 alert: Agent error rate 18% (baseline < 0.5%)

The customer's on-call ops engineer calls the FDE:

"Your Agent is broken. What do we do?"

What the FDE should be thinking:

1. Look at trace → find root cause (within 5 minutes)
2. Shift 1% of traffic back to the old version → stop the bleeding (within 2 minutes)
3. If that doesn't work, shift 100% back (within 1 minute)
4. After fixing, canary back up → controlled

What actually happens (bad case):

1. No trace wired up → can't tell which step failed
2. No canary lane → either tough it out or take everything offline
3. No baseline → can't tell whether it really is "worse than usual"
4. 1 hour of fumbling → the customer is watching

The "four-piece set" this chapter covers — observability / cost / canary / rollback — exists so that at 2 AM you can stop the bleeding in 5 minutes.

---

## 13.1 Observability — Not Just “Reading Logs”

Three observability dimensions

---

Metrics (aggregated numbers)

QPS, P50/P95 latency, error rate, token throughput  
→ trends, alarms

Logs (single text records)

- Concrete error stacks, full prompt + response
- → root-cause investigation

Traces (cross-service call chains)

- User request → API → retrieval → model → tool → response
- → finding bottlenecks / linking issues end to end

LLM applications have 4 **special observability dimensions**:

1. Token economics:
  - input/output tokens per request
  - cost per request (priced by model)
2. Eval drift:
  - sample from production → run Eval → watch the score trend
3. Hallucination monitoring:
  - rate of answers not supported by grounding documents
  - sampled real-time scoring with LLM-as-judge
4. Agent path:
  - distribution of steps per completion
  - tool-call success rate
  - retry counts

## AWS Practice: The Observability Three-Piece Set

Minimum observability stack (on AWS)

Metrics:

- CloudWatch Metrics (auto: Bedrock invocations, latency, tokens)
- CloudWatch Custom Metrics (yours: eval score, hallu rate)

Logs:

- CloudWatch Logs (application logs, must include trace\_id)
- Bedrock Model Invocation Logging (prompt + response)
- Long-term archive → S3

Traces:

- X-Ray (across Lambda / API Gateway / ECS)
- Optional: LangFuse / Phoenix (LLM-specific tracing)

Dashboard:

- CloudWatch Dashboard (ops)
- Custom BI (business KPIs)

## The 6 Must-Have Dashboard Cards

1. QPS + Error rate (overall health)
2. P50 / P95 / P99 latency (experience)
3. Token usage + Cost trend (money)
4. Eval score (real-time sampled) (quality)
5. Top failure types (debug entry)
6. Agent step distribution (Agent health)

**AWS knowledge references:** search “CloudWatch metrics for Bedrock” , “X-Ray for Bedrock Agents” , “Bedrock model invocation logging” .

---

## 13.2 Cost Control — Tokens Are the New “Utility Bill”

The biggest “non-engineering” risk for an LLM application is **money**.

Typical cost structure (monthly)

Bedrock model calls (60–80%)

- input tokens × unit price
- output tokens × unit price (typically 4–5x more expensive)

Embeddings (5–10%)

- one-time at indexing + every query

Knowledge Base / Vector DB (5–15%)

- OpenSearch Serverless OCU or pgvector instance

Other (5–15%)

- Lambda / ECS / network / monitoring

## The 4 Engineering Moves on Cost

### 1. Caching

- Cache identical queries for 1 hour
- Prompt-prefix caching (Anthropic / Bedrock now support it)
- Saves 30–70%

### 2. Routing

- Simple query → mini / haiku
- Complex query → sonnet / opus
- Saves 50–80%

### 3. Compression (context compression)

- Too many top\_k results retrieved by RAG → cap them
- Optimize the system prompt (dedupe + tighten)
- Saves 10–30%

#### 4. Batching

- Async tasks via Bedrock Batch (50% discount)
- Good for eval / offline analysis

## AWS Practice: Bedrock Cost Monitoring

### Three layers of Bedrock cost monitoring

#### Layer 1: AWS Cost Explorer

- Aggregate by service / region / tag
- Monthly trend + anomaly alarms

#### Layer 2: Cost Allocation Tags

- Tag every Agent / KB (project / team / customer)
- Allocate cost by tag

#### Layer 3: Application-layer instrumentation

- Per invoke, record input/output tokens + model
- Aggregate by business scenario / user / department
- Report to CloudWatch Metrics

## Budget Alarm Is Mandatory

#### AWS Budgets:

- Monthly budget of \$X
- 80% / 100% / 120% three-tier alerts
- Above 100%: email + Slack to owner
- (Use auto-stop carefully in production)

**AWS knowledge references:** search “AWS Cost Explorer for Bedrock” , “Bedrock prompt caching” , “Bedrock batch inference” .

---

## 13.3 Canary Releases — Not “All or Nothing”

### Why Canary Is Mandatory

#### A no-canary deploy:

Launch → discover problem → full rollback → all users affected  
 Damage = all users × outage time

#### A canary deploy:

1% → monitor 30 min → 10% → ... → 100%  
Damage = 1% of users × short time  
100x more headroom to stop the bleeding

## Canary Strategies

Three canary methods

---

By percentage (traffic share)

- 1% → 5% → 25% → 50% → 100%
- Best for: general features / high traffic

By user / segment

- Internal staff first → beta users → premium → all
- Best for: high-risk / business-critical features

By feature flag

- Same binary, config center toggles the switch
- Best for: A/B tests / hot-switchable features

## Canary “Gates”

Each canary stage needs a “gate” condition:

From 1% to 5%:

- ✓ Error rate  $\leq$  baseline + 0.5%
- ✓ P95 latency  $\leq$  baseline + 200ms
- ✓ Real-time-sampled Eval  $\geq$  baseline - 0.02
- ✓ No P1 alarms

From 5% to 25%:

- ✓ All of the above + at least 30 minutes stable
- ✓ Written sign-off from customer business side

## AWS Practice: 3 Canary Approaches

Approach A: API Gateway Stage + Lambda Alias

- One-click traffic-percent shift (Lambda traffic shifting)
- Simple / most common for Bedrock applications

Approach B: ECS / EKS blue-green / canary

- CodeDeploy + ECS service
- Supports the full canary toolkit

Approach C: Application-layer feature flag

- LaunchDarkly / AWS AppConfig
- Shift traffic without redeploying

- Recommended: pair with Lambda
- 

## 13.4 Rollback — Must Be Switchable Within 5 Minutes

3 things rollback must achieve

1. Easy trigger  
One command / one button / one PR revert  
Not "5 config steps + redeploy"
2. Bounded time  
From decision to in effect: < 5 minutes  
Ideally < 1 minute
3. Data compatibility  
Data written by the new version is readable by the old version  
(forward-compatible design)

### Rollback Checklist

- ✓ Config rollback (config / prompt / model / KB version)
- ✓ Code rollback (Lambda alias / ECS service)
- ✓ Data rollback (DB schema / embedded data)
- ✓ Frontend rollback (CDN cache invalidation)

### Special Handling for Prompt / Model / KB Rollback

In LLM applications, “rollback” isn’t only about code:

Prompt rollback:

- Store prompts in a config center (AppConfig / SSM Parameter Store)
- Don't hard-code them
- Switch = change one parameter

Model rollback:

- Application reads model\_id from config
- Switch = change modelArn in config

Knowledge Base rollback:

- Version your KB (snapshot before each data-source change)
  - Application reads KB id from config
  - Switch = change KB id in config
-

## 13.5 Failure Drills — Chaos Engineering for LLM

5 failure types to drill

---

1. Model fully unavailable  
→ Switch to backup model (cross-region or cross-provider)
2. KB / retrieval unavailable  
→ App degrades to "model-only answer + risk warning"
3. Tool call fails  
→ Agent returns gracefully + creates ticket
4. Single-region outage (AWS)  
→ DR failover to backup region
5. Upstream/downstream throttling / overload  
→ Circuit breaker + queue + retry

### How to Drill

Quarterly: one full DR drill (cross-region)  
Monthly: one small drill (kill the KB / model / tool)  
Weekly: sampled trace review + anomaly analysis

---

## 13.6 A Sample Production Dashboard

---

---

Customer Insurance Assistant — Production Dashboard

---

---

#### [Health]

QPS: 23.4 (avg)  
Error rate: 0.3% (baseline 0.4%)  
P95 latency: 1.8s (target <3s)  
Active users: 312

#### [Cost]

Today: \$89.2  
MTD: \$1,847  
Forecast: \$5,420 / mo  
Budget: \$6,000

#### [Quality]

Sampled accuracy: 87.2%  
User thumbs-up: 89%  
User thumbs-down: 4%  
Unrated: 7%

#### [Eval Drift]

7-day: 0.872 ↘ (-0.005)  
30-day: 0.876  
Threshold: 0.85

#### [Top Failures (last 24h)]

- Tool 'get\_policy\_pdf' timeout: 12 cases

- Hallucination flag: 3 cases (sampled)
- Guardrail block (PII): 18 cases (intended)

[Canary]

Current rollout: 100%

Last change: 2026-05-19 14:00 (prompt v2.3.1)

Auto-rollback armed:

---

**This dashboard sits on the customer’ s on-call screen, the FDE has it on theirs, and both sides are looking at the same source of truth.**

---

## Key Quotes

“A system without observability is a system you don’ t own.” — A. Lawrence, FDE Rule Book, 2025

“Every dollar saved by caching is a dollar of production runway.” — Anthropic enterprise best practices, 2025

“If you can’ t roll back in 5 minutes, you can’ t deploy on Friday.” — AWS GenAI Innovation Center, 2025

---

## Hands-On Checklist

PoC weeks 4–6 + pre-launch — must do:

1. **Wire up CloudWatch + X-Ray + Bedrock Logging** (none optional).
  2. **Build the 6-card dashboard** (section 13.1).
  3. **Configure Cost Explorer Tag + AWS Budgets monthly alarm.**
  4. **Route prompt / model / KB through a config center** (no hard-coding).
  5. **Add canary deploy to CI/CD** (API Gateway / Lambda Alias).
  6. **Write a “5-minute rollback SOP”** : from decision to in-effect flow.
  7. **Run the first failure drill** (kill the KB and watch it degrade gracefully).
- 

## Anti-Patterns

- **Hard-coding prompt / model in code** (every change requires a release).
- **Going live without cost monitoring** (the bill is a month-end surprise).
- **Canary only at 0% and 100%** (any incident takes down everyone).
- **Rollback that means “redeploy the previous version”** (5 minutes becomes 50).
- **Dashboard the customer can’ t see** (customer on-call doesn’ t know what’ s happening).



- **Trusting** “in theory we can switch” **without drilling** (when it actually breaks, it’s too late).
  - **Too many alerts** → **numbness** (only alarm on what’s truly actionable).
- 

## Relation to the Next Part

By here, the “PoC → production” gap has been crossed: your LLM/Agent is running stably in the customer’s production environment.

The next Part enters the **Agent era** — not the “let’s add RAG” kind of Agent, but truly “autonomous decision + tool use + cross-system action” Agents. The FDE’s engineering work changes shape at this stage.

[← Previous: PoC Pass-Line Conditions](#) · [Next Part: The Agent Era →](#)

# Part VI: The Agent Era — New Engineering Tasks for the FDE

Applies to: **LLM application track** (optional reading for the data-driven track)

---

## What This Part Solves

2025–2026 is the year Agents truly entered the enterprise.

Not the “customer-service Q&A” flavor of RAG, but: - Auto-writing / editing / submitting PRs - Running cross-system errands (CRM + ERP + email + calendar) - Driving a browser / terminal / API to complete tasks

The new engineering problems Agents bring to the FDE:

1. **Toolset design** — not “more is better,” but “enough + won’ t go wrong”
2. **Sandbox and permission boundaries** — once an Agent oversteps, the blast radius is uncontrollable
3. **Failure recovery** — how do you resume a multi-step task that broke mid-flight
4. **Enterprise integration** — how MCP (Model Context Protocol) plugs an Agent into the customer’ s tools

This Part gives you two hands-on chapters:

- **Chapter 14:** Deploying Agents in customer environments — toolsets / sandboxing / failure recovery
- **Chapter 15:** MCP and enterprise integration — wiring Agents into customer tools

## Chapters

See [SUMMARY.md](#).

## Relation to Other Parts

- **Prerequisites:** Part III (Eval) + Part IV (data / integration) + Part V (productionization) are all foundations for Agents

- **What follows:** Part VII completes handoff and craft progression
-

# Chapter 14: Deploying Agents in Customer Environments — Toolsets / Sandboxing / Failure Recovery

## Opening

An FDE built a "sales assistant Agent" for a customer with 47 tools:

- Read CRM
- Update CRM
- Read ERP
- Update ERP
- Send email
- Update calendar
- Create ticket
- Close ticket
- Adjust price
- Send coupon
- ... all the way to tool #47

Week 1 demo: flawless.

Week 3, the customer's business owner pulls the FDE aside:

"This week the Agent sent 100%-off coupons to three customers in a row. Loss: roughly ¥200K."

Post-mortem: under certain prompts the model "took initiative" and called `send_coupon(100)`. No sandbox. No amount cap. No dry-run.

The FDE rewrote it overnight:

- 47 tools cut to 18 (merged + deleted dangerous ones)
- Write/update tools got "second confirmation + amount cap + dry-run"
- Added audit trail and alerts

Week 5 relaunch: zero incidents.

This chapter covers the four things that matter when deploying an Agent in a customer environment — toolset design / sandboxing / failure recovery / eval.

---

## 14.1 Toolset Design — Less Is More

### The “magic numbers” of tool count

Agent tool count vs. accuracy (empirical)

---

|       |                                              |
|-------|----------------------------------------------|
| ≤ 5   | 95%+ accuracy, simple                        |
| 6-10  | 90% accuracy, engineering-friendly           |
| 11-20 | 80% accuracy, requires careful prompt design |
| 21-30 | 70% accuracy, recommend splitting            |
| 31-50 | 60% accuracy, uncontrollable                 |
| > 50  | "tool hell," accuracy is folklore            |

**First principles:** the probability of the model picking the wrong tool grows exponentially with tool count.

### Four principles for toolset design

1. One verb + one object
  - ✓ `create_ticket(title, body)`
  - ✗ `smart_helper(action, params)` (too broad)
2. Descriptions must be "model-friendly"
  - ✓ "Returns customer order history. Use when user asks about past purchases."
  - ✗ "Get orders" (model has no idea when to call it)
3. Strict parameters
  - ✓ JSON Schema with hard validation
  - ✓ Required vs Optional clearly marked
  - ✓ Enums constrain values
4. Tier dangerous operations
  - read tools: execute directly
  - write tools: second confirmation / dry-run
  - large amount / irreversible: multi-party approval

### Tool description template

```
# A well-formed tool definition
{
  "name": "send_email",
  "description": (
    "Send an email to specified recipients. "
```

```

        "Use this tool when user explicitly asks to send/notify someone. "
        "Do NOT use for internal logging or status updates. "
        "Maximum 5 recipients per call."
    ),
    "input_schema": {
        "type": "object",
        "properties": {
            "to": {
                "type": "array",
                "items": {"type": "string", "format": "email"},
                "maxItems": 5,
                "description": "Recipient email addresses"
            },
            "subject": {"type": "string", "maxLength": 200},
            "body": {"type": "string", "maxLength": 5000},
            "dry_run": {
                "type": "boolean",
                "description": "If true, validate but do not actually send",
                "default": false
            }
        },
        "required": ["to", "subject", "body"]
    }
}

```

---

## 14.2 Sandbox — The Agent’s “Permission Boundary”

### Three lines of defense

The Agent execution sandbox — three layers

---

#### Layer 1: model layer

- System prompt strictly constrains behavior
- Guardrails block dangerous output
- Few-shot examples steer it

#### Layer 2: tool layer

- Dangerous tools require second confirmation
- dry-run mode on by default
- Caps on amount / quantity / frequency

#### Layer 3: execution layer

- IAM / database permission isolation
- VPC network isolation

- Rate limiting + circuit breakers

## Key technique: dry-run mode

```
# A design pattern with dry_run=True as the default
def send_coupon(customer_id, amount, dry_run=True):
    if dry_run:
        return {
            "status": "dry_run",
            "would_send_to": customer_id,
            "amount": amount,
            "note": "Set dry_run=false to actually send"
        }
    # actual send logic...
```

Any time the Agent calls a write tool, the first call must be **dry\_run**. The model decides on its own whether to set `dry_run=false` the second time (with a second-confirmation mechanism in place).

## Key technique: amount / frequency limits

```
# Hard limits inside the tool
MAX_COUPON_AMOUNT = 50 # CNY
MAX_COUPON_PER_HOUR = 10 # calls

def send_coupon(customer_id, amount):
    if amount > MAX_COUPON_AMOUNT:
        return {"error": f"Amount exceeds limit ({MAX_COUPON_AMOUNT})"}
    if rate_limit_exceeded("coupon", "1h", MAX_COUPON_PER_HOUR):
        return {"error": "Rate limit exceeded"}
    # actual send
```

These limits cannot live in the prompt (prompt injection bypasses them); they have to be hard-coded in the tool implementation.

## Permission isolation — tools use the user context, not a service account

Wrong:

Agent → tool (with admin role) → DB  
→ any user can read / modify any data

Right:

User logs in → obtains `user_token`  
Agent (carrying `user_token`) → tool → calls DB with `user_token`  
→ user can only read / modify their own data

## AWS in practice: the Bedrock Agents permission model

### Bedrock Agent permissions — three layers

1. Agent execution role
  - The Agent's own IAM role
  - Usually only allowed to call Bedrock + its own KB
2. Action group Lambda
  - Each action group can have its own Lambda
  - Lambda uses its own role to call downstream services
3. User context (passed via session attributes)
  - User login info (cognito / IAM identity)
  - Lambda uses this context to make ABAC decisions

- Don't give the Agent one almighty role
- Give the Agent permission to invoke Lambda, and let Lambda enforce per-user authorization

**AWS reference:** search “Bedrock Agent session attributes,” “Bedrock Agent execution role.”

---

## 14.3 Failure Recovery — What to Do When a Multi-Step Task Breaks

### The “break points” of an Agent task

#### A typical Agent task path

User: "Reassign all P1 tickets from last week to Zhang and notify him"

Agent:

```
Step 1: list_tickets(status="P1", week="last") → 5 items
Step 2: assign_ticket(id=#101, to="zhang") ✓
Step 3: assign_ticket(id=#102, to="zhang") ✓
Step 4: assign_ticket(id=#103, to="zhang") × (network blip)
Step 5: ...
```

Problem:

- #101 and #102 are now assigned, #103 failed
- Will a re-run reassign #101 and #102?
- The model may simply give up on the task



## The fix: idempotency + state persistence

```
# Tool calls must be idempotent
def assign_ticket(ticket_id, assignee, idempotency_key=None):
    if not idempotency_key:
        idempotency_key = f"assign-{ticket_id}-{assignee}"

    # Repeated calls with the same key return the previous result
    if cache.exists(idempotency_key):
        return cache.get(idempotency_key)

    result = actually_assign(ticket_id, assignee)
    cache.set(idempotency_key, result, ttl=86400)
    return result
```

```
# Persist Agent execution state
class AgentSession:
    def __init__(self, session_id):
        self.session_id = session_id
        self.state = load_from_dynamodb(session_id) or {
            "task": None,
            "completed_steps": [],
            "pending_steps": []
        }

    def execute(self, task):
        for step in self.state["pending_steps"]:
            try:
                result = run_step(step)
                self.state["completed_steps"].append(step)
                save_to_dynamodb(self.session_id, self.state)
            except RetryableError:
                # leave it in pending — next run will resume
                save_to_dynamodb(self.session_id, self.state)
                raise
```

## AWS in practice: Step Functions as the Agent' s safety net

Wrap the Agent in Step Functions

Good fit:

- A single Agent task runs > 5 minutes
- Multiple Agents collaborating
- Need to persist intermediate state
- Need human-in-the-loop

Strengths:

- Visualized execution history
- Automatic retry + exponential backoff
- Resume from break point on failure
- Pause + wait for approval + resume

Anti-pattern:

- Simple single-step Agent → Step Functions is overkill
- High-frequency low-latency → not a fit

**AWS reference:** search “Step Functions express workflows for AI,” “Step Functions human approval.”

---

## 14.4 Agent Eval — A Different Kind of Eval

A regular LLM application’s Eval looks at “is the answer right?” Agent Eval also has to look at:

### Five dimensions of Agent evaluation

1. Task completion rate  
Did the user's goal ultimately get achieved?  
binary: yes / no
2. Path correctness  
Were the execution steps reasonable? Any redundancy?  
metric: actual steps / "canonical path" steps
3. Tool-use accuracy  
Right tool? Right parameters?  
metric: tool\_call\_accuracy
4. Side-effect control  
Did it "do anything it shouldn't have"?  
metric: side\_effect\_count
5. Experience cost  
Total latency / token cost / retry count  
metric: latency, cost, retries

### Designing Agent Eval samples

```
{
  "id": "agent-eval-007",
  "task": "Reassign all P1 tickets from last week to Zhang",
  "context": {"current_user": "manager-001"},
  "expected_outcome": {
```

```

    "tickets_assigned": ["#101", "#102", "#103", "#104", "#105"],
    "all_to": "zhang",
    "side_effects_allowed": ["notify_zhang"],
    "side_effects_forbidden": ["close_ticket", "notify_customer"]
  },
  "expected_path": {
    "min_steps": 6,
    "max_steps": 12,
    "must_use_tools": ["list_tickets", "assign_ticket"],
    "must_not_use_tools": ["delete_ticket", "send_coupon"]
  }
}

```

## AWS in practice: Bedrock Agent Evaluations

Bedrock built-in Agent Evaluation:

- Auto-evaluates trajectory (path)
- Evaluates tool selection accuracy
- Evaluates task success rate
- Supports custom metrics

## 14.5 Human-in-the-Loop — High-Risk Actions Always Need a Reviewer

Three HITL modes

---

1. Always-approve (high risk)
  - Delete data / large transfer / external email
  - Agent prepares → push for approval → human clicks → execute
2. Sample-approve (medium risk)
  - Update CRM / create ticket
  - Agent acts directly
  - But 5% / 10% sampled into approval queue (post-hoc + calibration)
3. No-approve (low risk)
  - Queries / reports / internal memos
  - Agent fully autonomous

## Implementation architecture

Agent → detect high-risk action → write to SQS / DynamoDB

↓  
(notify human via Slack / email)

↓  
Human approves in a Web UI  
↓  
Step Functions resumes  
↓  
Agent executes

---

## 14.6 Deployment Checklist

### Pre-production Agent checklist

- ☐ Tool count  $\leq 20$  (split by force if more)
  - ☐ Every tool has a complete description + JSON schema
  - ☐ Write / update / delete tools default to dry\_run
  - ☐ Amount / frequency / quantity caps hard-coded
  - ☐ User context plumbed through to tools (no service account)
  - ☐ Idempotency key enforced
  - ☐ State persisted (DynamoDB / Step Functions)
  - ☐ Bedrock Guardrails: PII + Topic + Content
  - ☐ High-risk actions go through HITL
  - ☐ Trace + cost dashboard
  - ☐ Eval set covers task / path / tool / side-effect (all four)
  - ☐ Canary + rollback path
  - ☐ Failure drills (model down / tool down)
- 

## 14.7 An End-to-End Agent Deployment

Customer scenario: an insurance company's "claims assistant Agent"

Toolset (12 tools):

- [read]
  - get\_policy(policy\_id)
  - get\_claim\_history(customer\_id)
  - get\_doctor\_records(claim\_id)
  - search\_clauses(keyword)
- [write — simple]
  - create\_followup\_ticket(claim\_id, note)
  - send\_internal\_msg(team, msg)
- [write — important]
  - request\_more\_info(claim\_id, items[]) → dry\_run + agent approval
  - flag\_for\_review(claim\_id, reason) → supervisor approval
  - approve\_claim(claim\_id, amount) → human approval, always

- `reject_claim(claim_id, reason)` → human approval, always [escalate]
- `escalate_to_human(claim_id, reason)`
- `request_legal_review(claim_id)`

#### Sandbox:

- `approve_claim` must have amount  $\leq$  ¥5000
- `approve_claim` must go through HITL
- All write tools called with the agent's `user_id`

#### Failure recovery:

- Tasks orchestrated via Step Functions
- Each step's state written to DynamoDB
- Retry 3 times, still failing → escalate

#### Eval:

- 100 historical claims as the golden set
- Measure:
  - Approval/rejection alignment with the human decision
  - Average tool count (canonical 5–8 steps)
  - Wrong-tool call rate

#### Rollout:

- W11: 1% canary (internal claims agents)
- W12: 10% canary (low-risk cases)
- W14: 50%
- W16: 100%

---

## Key Citations

“An agent without a sandbox is a liability.” — A. Lawrence, FDE Rule Book, 2025

“The best agents have boring tools.” — Anthropic Claude tool use guide, 2025

“Don’ t ship an agent until you’ ve watched it fail safely 100 times.” — AWS GenAI Innovation Center, 2025

---

## Action Checklist

When you take on an Agent project, before deployment you must:

1. **Draw the tool list and tag each as read/write/delete**
2. **Add `dry_run` + caps to every “write” tool**
3. **Plumb user context through; no service accounts**

4. **Wire up an idempotency cache** (DynamoDB / Redis)
  5. **Configure Bedrock Guardrails for PII / business red lines**
  6. **Wrap high-risk tasks in Step Functions** (resumable from break points)
  7. **Build a 4-dimensional Eval set** (task / path / tool / side-effect)
  8. **Configure HITL** (amount / irreversibility / external impact)
- 

## Anti-Pattern Checklist

- **Shipping with > 30 tools** (accuracy is folklore)
  - **Writing without dry\_run** (incident source #1)
  - **Agent calling downstream with admin / service account** (every user gets escalated privileges)
  - **Letting the model “retry by itself” on failure** (no idempotency means duplicated operations)
  - **No HITL on high-risk actions** (you find out about the loss in front of the customer)
  - **Evaluating an Agent with regular Eval** (misses trajectory / side-effect)
  - **Going to 100% without a failure drill** (you’ ll meet the truth at 2 AM)
- 

## Relation to the Next Chapter

This chapter handled the engineering problems of an Agent in “its own environment.” The next chapter handles how the Agent gets “wired into the customer’ s tools” — enterprise integration in the era of **MCP (Model Context Protocol)**.

[← Part VI Intro](#) · [Next: MCP and Enterprise Integration](#) →

# Chapter 15: MCP and Enterprise Integration — Wiring Agents into Customer Tools

## Opening

In November 2024, Anthropic released MCP (Model Context Protocol).

Through 2025–2026, almost every enterprise B2B project demands MCP support.

An FDE's first MCP engagement:

- The customer already has: Confluence + Jira + Salesforce + internal Wiki
- The customer wants: "Claude / Bedrock Agent should be able to use all of them"
- Old approach: hand-write a LangChain Tool for each → 6 weeks
- With MCP:
  - Confluence MCP server (community already has one)
  - Jira MCP server (community already has one)
  - Salesforce MCP server (FDE writes one)
  - Internal Wiki MCP server (FDE writes one)
  - Wire it into Claude Desktop / Bedrock Agent
  - 1 week

10x development efficiency.

But MCP's "power" is also a "risk":

- One MCP server exposes the customer's internal API
- The Agent can hit 50+ tools at once
- Security / audit / permission boundaries are 100% the FDE's design

This chapter covers the engineering practice of deploying MCP inside a customer's enterprise.

---

## 15.1 What MCP Is

| <u>World without MCP</u>              | <u>World with MCP</u>               |
|---------------------------------------|-------------------------------------|
| Agent 1 integrates Tool A             | Every Agent shares the MCP protocol |
| Agent 1 integrates Tool B             |                                     |
| Agent 2 integrates Tool A             | Tool A implements one MCP server    |
| Agent 2 integrates Tool B             | Tool B implements one MCP server    |
| An $N \times M$ integration nightmare | A standard interface — $N + M$      |

MCP is essentially a **USB-C interface between LLMs and tools**:

### The three layers of MCP

Client (LLM application)

Claude Desktop / Cursor / Bedrock Agent / homegrown Agent

↓ JSON-RPC over stdio / SSE / HTTP

Server (tool implementation)

File / Slack / Jira / Salesforce / internal API ...

↓

Resource (capabilities exposed by the tool)

- tools (function calls)
- resources (files / data)
- prompts (predefined prompt templates)

### An MCP server exposes three kinds of capability

Tools — functions the LLM can invoke

e.g., `list_jira_issues(status, project)`

Resources — things the LLM can read

e.g., `file:///docs/policy.md`

`confluence://wiki/space/PROD/pages/123`

Prompts — predefined prompt templates

e.g., "summarize\_pr" template

"review\_security" template

---

## 15.2 Three Forms of Enterprise MCP Deployment

Form 1: Local MCP server (developer machine)

Scenario: Cursor / Claude Desktop

Deployment: process launch, stdio transport

Fit: individual / small team



Form 2: Remote MCP server (HTTP/SSE)  
Scenario: shared across the enterprise / multiple users  
Deployment: K8s / ECS / Lambda + API Gateway  
Fit: enterprise-grade

Form 3: Centralized MCP gateway  
Scenario: 50+ MCP servers to manage  
Deployment: one gateway aggregates every server  
Fit: large enterprise + multi-tenant

90% of enterprise B2B projects are Form 2; some large customers are Form 3.

---

## 15.3 Writing an Enterprise MCP Server — Hands-On

Take a “Salesforce CRM MCP server” as an example:

### Step 1: Design the tools

```
# Enumerate the customer's business-critical operations
tools = [
    "list_accounts",
    "get_account_details",
    "search_opportunities",
    "create_task", # write
    "log_activity", # write
    "update_stage", # write, requires confirmation
]
```

### Step 2: Implement the MCP server (Python SDK)

```
from mcp import Server, Tool
from mcp.types import TextContent
import simple_salesforce as sf

server = Server("salesforce-mcp")

@server.tool()
async def list_accounts(name_contains: str = None, limit: int = 10):
    """List Salesforce accounts. Use when user asks about customers/accounts."""
    sf_client = get_sf_client() # uses user-scoped OAuth token
    query = "SELECT Id, Name, Industry FROM Account"
    if name_contains:
        query += f" WHERE Name LIKE '%{name_contains}%'"
    query += f" LIMIT {limit}"
```

```

    results = sf_client.query(query)
    return TextContent(text=json.dumps(results['records']))

@server.tool()
async def update_stage(opp_id: str, stage: str, dry_run: bool = True):
    """Update opportunity stage. Use only when user explicitly asks to change stage."""
    if dry_run:
        return {"status": "dry_run", "would_set": stage}
    sf_client = get_sf_client()
    sf_client.Opportunity.update(opp_id, {'StageName': stage})
    return {"status": "updated"}

if __name__ == "__main__":
    server.run()

```

### Step 3: Authentication — the critical part

Three auth modes for an enterprise MCP server

Mode A: User OAuth (recommended)

User signs in → obtains token → starts MCP server  
 → MCP server calls downstream with that token  
 → Downstream system enforces permissions

Mode B: Service account

MCP server calls downstream with a service account  
 Problem: anyone can use MCP to escalate privileges  
 Only acceptable for internal admin tools

Mode C: Per-call token forwarding

Every MCP call carries the user's token  
 MCP server forwards the token to downstream  
 Most rigorous, but more complex to implement

**Enterprise production:** Mode A or Mode C. Mode B is reserved for tightly controlled internal tools.

### Step 4: Deploy into the customer's VPC

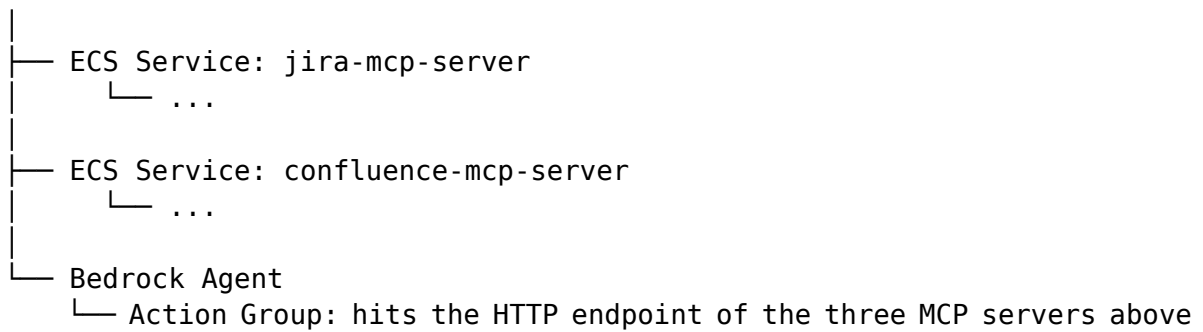
Enterprise deployment architecture

Customer VPC

```

├─ ECS Service: salesforce-mcp-server
│   ├── ALB (HTTPS, customer cert)
│   ├── Container (Python, MCP SDK)
│   └── Secrets Manager: SF OAuth config

```



## 15.4 Four Engineering Pitfalls of MCP

### Pitfall 1: the MCP server exposes too much

Newcomers writing an MCP server tend to “expose every API.” The result:

- ✗ Salesforce MCP exposes 80 tools
  - Agent picks the wrong one 60% of the time
  - Maintenance cost explodes
- ✓ Salesforce MCP exposes 8 high-frequency tools
  - Covers 90% of business scenarios
  - 90%+ accuracy

**Rule of thumb:** keep each MCP server to 5–15 tools. More than that, split the server.

### Pitfall 2: loose parameter schemas

```
# Bad
@server.tool()
async def query(text: str): # what is text?
    ...

# Good
@server.tool()
async def search_opportunities(
    name_contains: str = None,
    stage: Literal["Prospecting", "Qualification", "Negotiation"] = None,
    amount_min: int = None,
    limit: int = 10
):
    """Search Salesforce opportunities..."""
```

**Strict schemas** = the model picks wrong far less often.

### Pitfall 3: sloppy error handling

```
# Bad
try:
    result = sf_client.query(...)
    return result
except Exception:
    return None # the model has no idea what happened

# Good
try:
    result = sf_client.query(...)
    return {"status": "ok", "data": result}
except sf.SalesforceMalformedRequest as e:
    return {"status": "error", "type": "bad_query", "message": str(e), "hint": "check fi"}
except sf.SalesforceAuthenticationFailed:
    return {"status": "error", "type": "auth_failed", "message": "OAuth token expired",}
except Exception as e:
    return {"status": "error", "type": "unknown", "message": str(e)}
```

Make errors readable so the model can recover on its own.

### Pitfall 4: missing audit

```
# Every MCP call must write audit
@server.tool()
async def update_stage(opp_id, stage):
    audit_log({
        "user_id": current_user_id(),
        "tool": "update_stage",
        "params": {"opp_id": opp_id, "stage": stage},
        "timestamp": now(),
        "trace_id": get_trace_id()
    })
# actual execution
```

An MCP server without audit is not production-shippable.

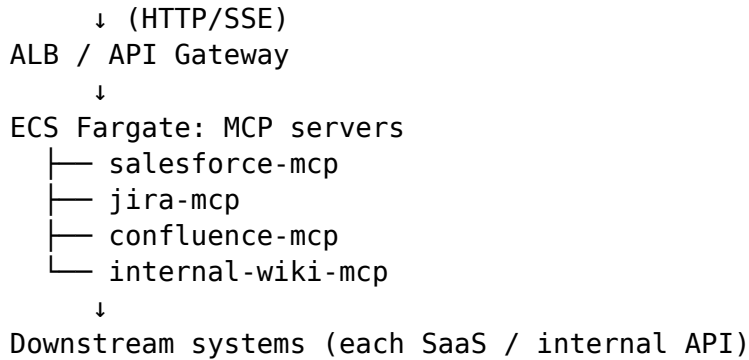
---

## 15.5 AWS in Practice: Bedrock Agent + MCP Integration

MCP deployment architecture on AWS

---

Customer Bedrock Agent  
↓ (Action Group)  
Lambda: mcp-bridge



A minimal Lambda bridge:

```
# Lambda: mcp-bridge
import boto3
import requests

def lambda_handler(event, context):
    # Called by the Bedrock Agent
    action_group = event['actionGroup']
    api_path = event['apiPath']
    parameters = event['parameters']

    # Forward to the matching MCP server
    mcp_url = MCP_SERVER_MAP[action_group]
    user_token = event['sessionAttributes'].get('user_token')

    response = requests.post(
        f"{mcp_url}/tools/call",
        json={"name": api_path, "arguments": parameters},
        headers={"Authorization": f"Bearer {user_token}"}
    )

    return {
        'response': {
            'actionGroup': action_group,
            'apiPath': api_path,
            'statusCode': response.status_code,
            'responseBody': {
                'application/json': {'body': response.json()}
            }
        }
    }
```

**AWS reference:** search “Bedrock Agent action group OpenAPI.” The official MCP-to-Bedrock bridge story is still evolving through 2025–2026; track AWS announcements.

## 15.6 Multi-Server Choreography — A Customer Scenario

Scenario: a sales assistant Agent

MCP servers wired into the Agent:

1. crm-mcp (Salesforce, 8 tools)
2. email-mcp (Outlook / Gmail, 5 tools)
3. calendar-mcp (Google Calendar, 4 tools)
4. wiki-mcp (internal Confluence, 3 tools)
5. order-mcp (internal ERP, 6 tools)

Total tools: 26 → within the 14.1 "magic number  $\leq 30$ "

A representative dialogue:

User: "What did ABC Corp give us as feedback last time?  
I'm visiting them next week."

Agent path:

- Step 1: `crm-mcp.search_accounts("ABC Corp")`
- Step 2: `crm-mcp.get_account_details(account_id="...")`
- Step 3: `crm-mcp.list_recent_activities(account_id="...")`
- Step 4: `email-mcp.search_emails(from="abc.com", days=30)`
- Step 5: `wiki-mcp.search("ABC Corp")`
- Step 6: synthesize the answer
- Step 7 (after user confirmation): `calendar-mcp.create_event(...)`

26 tools spread across 5 servers — each server cohesive and single-purpose.

---

## 15.7 Security Checklist

Enterprise MCP deployment security checklist

- ☐ User OAuth, not service account
  - ☐ MCP server inside the VPC, HTTPS enforced
  - ☐ Every tool has a detailed description + JSON schema
  - ☐ Write tools default to `dry_run`
  - ☐ Dangerous operations go through HITL
  - ☐ End-to-end `trace_id`
  - ☐ Audit log (who / what / when / result)
  - ☐ Bedrock Guardrails as a second line of defense
  - ☐ Rate limit per user (not just per IP)
  - ☐ Tool enable/disable through a config service (not code edits)
  - ☐ Minimal toolset (each server  $\leq 15$ )
  - ☐ MCP server gets its own vuln scan + pen test
-

## Key Citations

“MCP turned tool integration from  $O(N \times M)$  to  $O(N+M)$ .” — Anthropic MCP launch, 2024-11

“A poorly designed MCP server is a backdoor with documentation.” — A. Lawrence, FDE Rule Book, 2025

“The future enterprise agent will use 50+ MCP servers — the FDE’s job is to make all 50 boring.” — AWS GenAI Innovation Center, 2025

---

## Action Checklist

In the first week of an MCP integration project, you must:

1. **Inventory the customer’s tools** — which SaaS, which internal APIs
  2. **Check whether the community already has an MCP server** (Awesome MCP / Anthropic’s official repo)
  3. **For the rest, write your own MCP server** — one or two days with the Python SDK
  4. **Wire up user OAuth** (no service accounts)
  5. **Write a complete description + schema for every tool**
  6. **Add tracing + audit logging**
  7. **Build an Eval set** (including counter-examples for “wrong calls” )
  8. **Deploy inside the VPC + HTTPS + auth**
- 

## Anti-Pattern Checklist

- **Exposing all 60 tools in v1** (accuracy is folklore)
  - **Running an MCP server under a service account** (auth incident #1)
  - **No schema, letting the LLM “guess” parameters** (error rate skyrockets)
  - **Returning None on error** (the model has no idea what happened)
  - **Deploying an MCP server on the public internet** (anyone who finds it can call it)
  - **Tools too coarse-grained** (omnibus functions like “smart\_helper” )
  - **Shipping with no audit** (compliance fails immediately)
- 

## Relation to the Next Part

By this point, the FDE has walked the entire arc inside the customer’s environment — “PoC → production → Agent → enterprise integration.”

One Part remains: **Delivery and Craft Progression** — Handoff so the customer can take it forward, pattern extraction that turns engagements into reusable assets, and the FDE’s own T-shaped growth.

[← Previous: Deploying Agents in Customer Environments](#) · [Next Part: Delivery and Craft Progression](#) →



# Part VII: Handoff and Mastery

Applies to: Both tracks

---

## What This Part Solves

90% of FDE projects “go live,” but 60% of them stall into a half-dead state three months after launch:

- The customer’s ops team doesn’t really understand the system; when something breaks, they call the FDE
- No one on the FDE’s side has picked it up either; nobody is maintaining it
- Project-specific prompts, data, and lessons all live in one FDE’s head
- When the customer asks for “the next project,” the FDE has no idea what is reusable

This Part solves two things:

1. **Handoff** — the engineering method for actually handing the project to the customer
2. **Pattern extraction + T-shape growth** — turning this project’s output into “assets the next project can use,” and building the FDE’s long-term capability

## Chapters

- **Chapter 16:** Handoff + pattern extraction — abstracting the solution into reusable assets
- **Chapter 17:** The FDE’s T-shape growth — engineering depth + industry depth

## Relation to Other Parts

- **Prerequisite:** all six previous Parts
  - **Next:** once this Part is done, the FDE moves on to “the next project,” and the cycle repeats
-

# Chapter 16: Handoff + Pattern Extraction — Abstracting the Solution into Reusable Assets

## Opening

An FDE wraps up a 12-week Agent project. Technically, it's a success.

3 months later, the customer's ops team asks:

"We want to change that prompt — how do we change it?"

She flies in. 2 hours and it's done.

6 months later, customer:

"The Agent's error rate is creeping up — can you take a look?"

She flies in. A full day of debugging.

9 months later, customer:

"We want to add another use case."

She flies in, opens her own old code, and realizes  
she's a little fuzzy on it too.

A teammate at the FDE's company picks up a similar project  
for a new customer and comes by to ask her:

"How did you do this again?"

Her answer: "Look at my old code..."

but every customer is different. Just redo it."

Her CEO sees the proposal for the third customer and asks:

"Why are we quoting 12 weeks for three similar projects?"

It should be 4 weeks."

She has no answer.

This chapter covers two things:

1. Handoff — getting the customer to truly take over  
(so you stop getting the call to fly back in)

2. Pattern extraction — making the next similar project a 4-week job
- 

## 16.1 The Engineering Definition of Handoff

Handoff = "the customer can run this system without you"

The bar (4 capabilities):

---

1. The customer can deploy a hot fix on their own  
(change a prompt / KB / model id)
2. The customer can read the dashboard and find root cause on their own  
(you don't read the trace for them)
3. The customer can run an Eval on their own  
(no longer dependent on you before a model upgrade)
4. The customer can handle the top 5 incident types on their own  
(instead of calling you on every P1)

All 4 must be met → real handoff. Miss one = the project isn't fully delivered.

---

## 16.2 The 3-Week Handoff Countdown

Handoff doesn't start at launch — it starts three weeks before launch:

|     | <u>T-3 weeks</u>                                                                                                                                                          | <u>T-2 weeks</u> | <u>T-1 week</u> | <u>T launch</u>    | <u>T+4 weeks</u> |
|-----|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------|-----------------|--------------------|------------------|
|     | Plan<br>kickoff                                                                                                                                                           | Train<br>+ docs  | Shadow<br>ops   | Independent<br>ops | FDE exits        |
| T-3 | ✓ Identify the customer ops owner (person + email + rotation)<br>✓ Outline the Runbook<br>✓ Give the customer dashboard access                                            |                  |                 |                    |                  |
| T-2 | ✓ 4-hour training (Runbook + dashboard + Eval)<br>✓ Have the customer owner perform every action hands-on once<br>✓ Q&A                                                   |                  |                 |                    |                  |
| T-1 | ✓ Customer owner shadows the FDE through everything<br>✓ FDE doesn't drive — only reviews the customer's actions<br>✓ End of week: customer owner writes "what I learned" |                  |                 |                    |                  |

- T      Launch + canary
    - ✓ Customer owner leads, FDE in the back row
  - T+4    FDE fully steps away
    - ✓ Still on-call, but not actively watching
    - ✓ Customer reports their own ops status weekly
- 

## 16.3 The Runbook — the Customer's Operations Manual

A Runbook is not documentation — it's an **executable instruction list**.

The 7 sections every Runbook needs

1. System architecture diagram (one A4 page)
2. Deploy / rollback steps (commands / screenshots)
3. Top 10 incident types + handling SOPs
4. How to run Eval + what the thresholds mean
5. Where the key configs live + how to change them
6. Data / KB update SOP
7. Escalation path (when to call whom)

### A sample Runbook SOP

---

---

SOP-001: Sudden spike in Agent error rate

---

---

Symptom: dashboard shows error rate > 1% (baseline 0.3%)

Step 1: Check Bedrock service status

- Open <https://health.aws.amazon.com>
- Check us-east-1 Bedrock service health
- If AWS incident → wait + notify users

Step 2: Check recent commits

- GitLab → main branch, last 5 commits
- Any prompt / KB / model changes?
- If yes → go to Step 3 (rollback)

Step 3: Rollback

- Command: `./scripts/rollback.sh prod --target-version=$LAST_GOOD`
- Wait 5 minutes and watch the dashboard
- Error rate back to baseline → done
- Not back → go to Step 4

Step 4: Page the FDE on-call

- Slack: @fde-oncall
- Phone: +XX-XXX-XXXX (24h)

Step 5: Write the incident report

- Template: docs/incident-template.md
- Submit within 24 hours

---

A good Runbook lets the customer stop the bleeding on a P1 in 5 minutes.

---

## 16.4 Training — a 4-Hour Course

Handoff training: 4-hour agenda

---

Hour 1: Architecture + business flow

- └─ Where data comes from and where it goes (15 min)
- └─ The Agent's toolset and capability boundaries (15 min)
- └─ What Eval is for (15 min)
- └─ Walkthrough of every monitoring panel (15 min)

Hour 2: Daily operations

- └─ How to read the dashboard (hands-on) (20 min)
- └─ How to run an Eval (hands-on) (20 min)
- └─ How to change a prompt + canary it (hands-on) (20 min)

Hour 3: Incident handling

- └─ Top 10 incident drills (hands-on) (40 min)
- └─ Rollback drill (hands-on) (20 min)

Hour 4: Data / KB maintenance + Q&A

- └─ KB update SOP (hands-on) (30 min)
- └─ Q&A (30 min)

**Key:** every segment must be **hands-on by the customer**, not the FDE demoing.

---

## 16.5 Pattern Extraction — Making the Next Project 5x Faster

What “pattern extraction” means

At the end of every project, ask 4 questions:

Q1: Which work in this project would be "basically the same" for another customer?

→ That's a reusable asset.

Q2: Which work "ate huge time but could be avoided next time"?

→ That's the source of an engineering template.

Q3: Which "customer-specific" things are actually shared by many customers?

→ That's the source of an industry template.

Q4: Which things "I thought were easy turned out hard"?

→ That's the source of a warning card.

## Pattern extraction outputs — 4 asset classes

The FDE's "post-project asset library"

### 1. Code templates

- LLM RAG starter kit
- Bedrock Agent starter kit
- Eval CI starter kit
- Lambda MCP server starter kit

### 2. Document templates

- Discovery report template (Ch 4)
- SOW template (Ch 5)
- Runbook template (this chapter)
- Acceptance criteria template

### 3. Decision cards

- "Customer asks this → answer that" cheat sheet
- "This signal appears → switch to this approach"
- "For this kind of problem, do these 3 things first"

### 4. Anti-pattern case files

- Real incident postmortems
- Cost of the mistake + lesson learned

## Template "granularity" — lessons learned

Too coarse: "Bedrock starter template"

→ Still requires major rework on day one

✓ About right: "Bedrock + Knowledge Bases + Aurora pgvector,  
VPC deployment + IAM Identity Center starter template"

→ 80% of customer use cases use it as-is

Too fine: "China Merchants Bank private-cloud RAG starter template"

→ Granularity so narrow it's only ever used once

---

## 16.6 The Engineering Moves of Pattern Extraction

Within 1 week of every project ending, do 5 things:

1. Write a 1-page "project retrospective"
    - What we did
    - What we got right / wrong
    - Numbers (outcome / Eval / cost / time)
  2. Code review your entire project
    - Which blocks are "obviously reusable"
    - Lift them into the company internal repo
  3. Pull out 3 "decision moments"
    - At that moment, what did you judge → write it as a card
  4. Pull out 1-2 "if I redid it, I would..."
    - Write it into "project template v X+1"
  5. Run a 1-hour brown-bag with teammates
    - Not bragging about wins — talking about the pitfalls they don't know
- 

## 16.7 Industry Templates — One Example

Industry: Insurance

Template: "Insurance RAG/Agent starter kit v3.2"

Contents:

- ├ Industry knowledge
  - ├ Common org structures of insurance companies
  - ├ Three main flows: underwriting / claims / sales
  - ├ Common regulatory requirements (CBIRC / MLPS)
  - └ Common data stack (core + ECIF + channels + risk)
- ├ General Discovery template
  - ├ 12 insurance-specific questions
  - └ Checklist of 5 critical deliverables
- ├ Code templates
  - ├ Policy clause chunking (PDF + tables)
  - ├ Customer identity matching (ID card / customer no. / policy no. mapping)
  - ├ Bedrock + Aurora pgvector + KMS deployment IaC
  - └ MLPS-compliant audit logging spec

- └─ Eval templates
  - └─ 200 golden insurance Q&A
  - └─ 50 safety items (PII / inappropriate promises)
  - └─ Business-expert calibration process
- └─ Handoff templates
  - └─ Insurance ops handover SOP
  - └─ Regulatory inspection response template

**A new FDE picking up an insurance customer:** starts from v3.2, and by week 4 the customer is already seeing a demo.

---

## 16.8 Pattern Management at the Company Level

Pattern extraction is not an individual habit — the company should accumulate it in a structured way:

Structure of an FDE team's "knowledge center"

Company Wiki:

```

/fde-knowledge/
  patterns/                (cross-industry)
    rag-starter/
    agent-starter/
    eval-ci-starter/
  industries/              (industry)
    insurance/
    finance/
    retail/
    manufacturing/
  anti-patterns/          (counter-examples)
    2025-Q3-incidents.md
    ...
  decision-cards/         (decision cards)
  tools/                  (internal tools)

```

Code Repo:

```

fde-platform/
  starter-kits/
  common-lambdas/
  shared-prompts/
  eval-suites/

```

**80% of the gap between a senior FDE and a junior one lives in the depth of this knowledge center.**



---

## Key Citations

“Handoff is not a milestone — it’s a 4-week process you start 3 weeks before launch.” — A. Lawrence, FDE Rule Book, 2025

“The second project on a customer should take 1/3 the time of the first.” — Bob McGrew @ YC, 2025

“Pattern extraction is the difference between a consultant and an engineer.” — AWS GenAI Innovation Center, 2025

---

## Action Checklist

Mandatory in the last 4 weeks of a project:

1. **Start the Handoff countdown at T-3 weeks** (Section 16.2)
  2. **Write the 7 Runbook sections** (Section 16.3)
  3. **Run the 4-hour customer ops training** (hands-on)
  4. **Have the customer owner shadow-run ops for 1 week**
  5. **Within 1 week of project end, write the retro + extract 3 decision cards**
  6. **Pull out reusable code / docs / configs into the company internal repo**
  7. **Run a 1-hour brown-bag with teammates** (talk pitfalls, not wins)
- 

## Anti-Pattern Checklist

- **Only thinking about Handoff in launch week** (no time to train)
  - **Writing the Runbook as a “feature doc”** (instead of an executable SOP)
  - **Training that’s all slides, no hands-on** (the customer won’t remember)
  - **Customer ops paging the FDE on every P1** (the 4 capabilities weren’t trained in)
  - **Project ends, straight to the next, no retro** (every project starts from zero)
  - **Patterns live only in your own head** (teammates don’t know, the company doesn’t know)
  - **Retros that only cover the wins** (the most valuable part is the pitfalls)
- 

## Relation to the Next Chapter

By here, the full loop is closed: Discovery → Scaffolding → production → Handoff → pattern extraction.

The final chapter covers how the FDE' s own capability grows over the long run — not chasing the latest tech, but **T-shape growth**: deepening engineering depth and industry depth in both directions.

[← Part VII Intro](#) · [Next: T-Shape Growth](#) →

# Chapter 17: The FDE's T-Shape Growth — Engineering Depth + Industry Depth

## Opening

Two FDEs at the same company, same title.

A — 3 years in, 8 customers. Each one was a new industry, new stack, new team.

- Self-intro: "I've done every kind of LLM project."
- Takes 4-5 weeks to ramp on a new customer.
- Has not become anyone's "designated FDE."

B — 3 years in, 6 customers. 5 of them in financial services.

- Self-intro: "Financial-services LLM, from underwriting to anti-fraud, 3 years deep."

- In the first meeting, can directly discuss "how MLPS Level 3 actually lands on an LLM appl
- 3 of his old customers renew every year + refer new customers.
- Internally known as "the financial-services FDE elder."

When A job-hops, HR doesn't know what he's good at.

When B job-hops, three insurance companies send him offers immediately.

The two have similar engineering depth.

The difference is — B picked an industry to go deep on. A didn't.

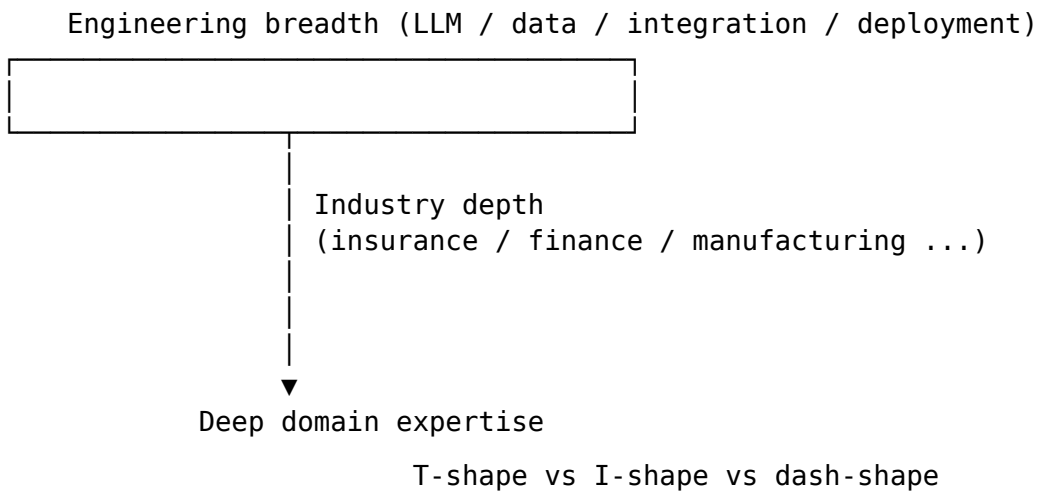
This chapter covers: how an FDE walks the "T-shape growth" path, so that 5 years from now you aren't merely "someone who's done a lot of projects" but "someone irreplaceable in some domain."

---

## 17.1 T-Shape = Engineering Depth × Industry Depth

T-shape growth structure

---



Pure breadth (dash): Knows many things, all shallow  
→ 5 years in, still a "project worker"

Pure depth (I): Deep in one industry, weak engineering  
→ easily replaced by AI tools

T-shape: Engineering breadth + industry depth  
→ 5 years in, becomes "irreplaceable"

---

## 17.2 Engineering Breadth — the 5 Required Foundations

Whether you sit on the LLM side or the data-driven side, an FDE needs all 5:

1. Data engineering  
SQL + dbt + Spark + warehousing + schema design  
→ Ch 9
2. LLM application engineering  
Prompt + RAG + Agent + Eval + MCP  
→ Ch 6/7/8/14/15
3. Cloud engineering  
Strong proficiency in at least one of AWS / Azure / Aliyun  
IaC (Terraform / CDK / Bicep)  
→ Ch 10
4. Platform engineering  
CI/CD + monitoring + canary + rollback  
→ Ch 13

## 5. Soft skills

Discovery + customer conversations + writing + presenting

→ Ch 4/5/16

Missing any of the 5 → you're not an FDE; you're a "frontline PM" or a "frontline implementation engineer."

---

## 17.3 Industry Depth — How to Pick One and How to Go Deep

### 3 criteria for picking an industry

1. Customer base
  - How many customers in this industry, in your company / your country?
  - 100 → worth going deep on
  - < 20 → may not feed you
2. Your interest + temperament
  - Finance demands rigor + compliance
  - Manufacturing demands on-site + physical
  - Retail demands speed + creativity
  - Pick one you "won't get sick of in 5 years"
3. The industry's LLM adoption stage
  - Early (2023-2024): customers are "just trying things"
  - Now (2025-2026): customers are "serious about landing it"
  - You want one that's "now + will stay relevant for the next 5 years"

### The 3 levels of industry depth

#### Level 1: You know the vocabulary

Can follow customer-internal conversations.

e.g. Insurance — knows the difference between underwriting / claims / sales / policy issuance.

Time: 3-6 months

#### Level 2: You understand the business flow

Can sketch the customer's main business flows.

Can anticipate their pain points and compliance constraints.

e.g. Insurance — knows "the standard underwriting flow + the 5% case-by-case underwriting flow."

Time: 1-2 years

#### Level 3: You have judgment

Can tell the customer "don't do this" + "why."  
Can offer "an angle they hadn't considered."  
e.g. Insurance — "Don't fully automate claims on day one;  
run human first-pass review + AI decision suggestion  
until it's stable, then expand."  
Time: 3-5 years

---

## 17.4 How to Deepen Industry Depth — 6 Moves

It's not "read more articles." It's deliberately doing 6 things:

1. Read 2-3 industry classics every year
    - Insurance: "Principles of Insurance" / "Intro to Solvency II"
    - Finance: "Bank 4.0" / "Risk Management" textbooks
  2. Read 1 piece of "customer-internal material" every month
    - The customer's annual report / interim report
    - Reports from industry associations
    - New regulatory rulings
  3. Attend 1 industry conference every quarter
    - Not an AI conference — a conference of the industry itself
    - e.g. an insurance product innovation forum / annual meeting
  4. At the end of each project, write "industry template v X+1"
    - The pattern extraction from Ch 16
    - In a year, your "insurance RAG template" gets very mature
  5. Make friends with the customer's "business veterans"
    - Not IT — the business side
    - One sentence from a business veteran is worth 100 articles
  6. Give "non-technical" talks to customer execs
    - Speak to "the trends I'm seeing in your industry"
    - Don't talk AI — talk industry
    - Customers start treating you as "one of their own"
- 

## 17.5 Engineering Depth vs Engineering Breadth — Go Deep on 1-2

The "vertical" in T-shape has another distinction:

Engineering breadth: all 5 (Section 17.2)

Engineering depth: pick 1-2 of the 5 to go deep on

Recommended directions to deepen (FDE long-term value):

- A. LLM applications + Eval + Agent (the frontier)  
→ Suits: people who chase new + understand distributed systems
- B. Data governance + Ontology + data stack (the Palantir line)  
→ Suits: people who like stability + understand business
- C. Cloud-native + IaC + platform engineering  
→ Suits: people who like engineering aesthetics + DevOps temperament
- D. Security + compliance + audit  
→ Suits: rigorous types + lots of large-enterprise customers

**Don't try to "go deep on all 5."** Not impossible, but unnecessary + no time. **Going deep on 1-2** is enough to form competitive advantage.

---

## 17.6 The 5-Year Growth Path

A typical 5-year path for "T-shape growth" :

### Year 1: Onboarding

- Shadow a senior FDE on 1-2 projects
- Engineering breadth at 60% (entry-level on all 5)
- Pick an industry (try 1-2; commit by year-end)
- Year-1 project retros become 30 pages of docs

### Year 2: Standing on your own

- Lead 2-3 projects independently (1-2 in your chosen industry)
- Engineering breadth at 80% (mid-level on all 5)
- Industry: Level 1 → Level 2
- Engineering: start going deep on 1 area

### Year 3: Forming expertise

- 3-4 projects independently; 70% in your chosen industry
- Engineering breadth at 90% + 1-2 areas at depth (senior)
- Industry: Level 2 → on the edge of Level 3
- Internal "industry-X elder" reputation starts forming

### Year 4: Irreplaceable

- Customers come to you directly (no longer assigned by the company)
- Industry: Level 3
- Engineering depth: 1 area at top tier + 1 at senior

- Start mentoring new hires / writing internal best practices

Year 5: Pick the next move

---

Path A: Senior FDE / Principal

- Same model, higher-end customers

Path B: FDE team lead

- Lead 5-10 FDEs

Path C: Move to product / internal product lead

- Use FDE experience to build product

Path D: Founder

- In your industry depth + engineering direction
- 

## 17.7 Anti-Patterns — 5 Years and No T

A new industry on every project

- 5 years in, no industry is deep

Engineering "tries everything new" with nothing deep

- 5 years in, every area is shallow

No retros, no template extraction

- 5 years in, still a "project worker"

No friends among business experts

- 5 years in, still a "tools person" — never enters the customer's decision-making circle

"FDE is a stepping-stone"

- One year in and you're already eyeing PM / big tech
  - You never extract the FDE role's real value
- 

## 17.8 Four Sentences to You, Right Now

1. With every customer, ask yourself:

"What can I take away from this project as a deposit?"

Not the outcome (that belongs to the customer and the company),  
but the patterns / decisions / industry knowledge.

2. Pick one industry. Stay 3 years.

Even if you don't enjoy year 1, by year 3 you'll find  
you've become "one of the few who can speak engineering  
in that industry's language."



3. Reserve a "study hour" every week.  
Read industry material / read papers / write retros.  
That hour is the source of your long-term FDE value.
  4. Write it down.  
Knowledge in your head isn't knowledge — only on paper is it.  
5 years from now, looking back at your 200 notes  
will tell you "who you've become" better than any résumé.
- 

## Key Citations

“The best FDEs become so industry-fluent that customers think they used to work for the customer.” — A. Lawrence, FDE Rule Book, 2025

“Optionality is overrated. Depth is underrated.” — Bob McGrew @ YC, 2025

“A T-shaped engineer is the only kind that survives the next decade of AI.” — Anthropic engineering culture, 2025

---

## Action Checklist

In the next 4 weeks:

1. **Draw your own T:** score yourself on each of the 5 breadth areas; have you picked an industry?
  2. **Whichever breadth area you’ re missing → fix it this quarter** (one per quarter)
  3. **No industry yet → pick one this month** (use the 3 criteria from 17.3)
  4. **Build a private retro library:** write a retro within 1 week of every project ending
  5. **Reserve a “study hour” every week:** industry + engineering depth
  6. **Brown-bag with teammates after the next project:** share the pitfalls, not the wins
  7. **3 years from now, re-read this book:** see which anti-patterns you avoided
- 

## Anti-Pattern Checklist

- **A new industry on every project** (5 years, all shallow)
- **Always chasing new in engineering, never going deep on one** (high risk of being replaced by AI tools)
- **Jumping ship the moment “AI Engineer” becomes a hot role** (FDE scarcity is precisely because it’ s hard)
- **No retros** (knowledge in your head isn’ t knowledge)
- **No friendships with business veterans** (you’ ll never enter the customer’ s decision-making circle)

- Treating FDE as a “stepping-stone” (you don’ t extract the real value)
- 

## Closing the Book

Here ends the practical handbook of FDE engineering.

17 chapters in one line each

---

Part I: The FDE is the most "PM" of engineers, and the most "engineer" of PMs.

Part II: Without thorough Discovery + an Eval set, writing code is faith-based.

Part III: Selection in 30 minutes, Eval in week one, prompting first.

Part IV: Data + network + identity + audit = the foundations of enterprise FDE.

Part V: Build PoCs like production; canary + rollback + cost from week one.

Part VI: Less is more for Agent toolsets; MCP is the enterprise default for the next 2 years.

Part VII: 4-week Handoff countdown; pattern extraction every project; T-shape growth ask yourself yearly.

The last line is for every FDE:

**“Sell the outcome. Fix forward. Eval first. Hand it off. Extract the pattern.”**

5 years from now, when you re-read this book, I hope you’ ve already become **the irreplaceable** FDE in some industry.

[← Previous: Handoff + Pattern Extraction](#) · [Back to Contents](#)

# Appendix A: FDE Tool Stack Cheat Sheet (2026 Edition)

3-5 candidates per category, plus the dimensions to choose by. Built for 30-minute decisions during PoC scoping.

## A.1 Model APIs

| Model             | Strength                | Best For                   | Price Tier  |
|-------------------|-------------------------|----------------------------|-------------|
| Claude 3.5 Sonnet | Balanced + long context | General RAG / Agent        | Mid         |
| Claude 3 Opus     | Strong reasoning        | Complex tasks / Judge      | High        |
| Claude 3 Haiku    | Fast + cheap            | High QPS / simple tasks    | Low         |
| GPT-4o            | Strong + multimodal     | General + vision           | Mid-high    |
| GPT-4o-mini       | Cost-performance        | High QPS general           | Low         |
| Llama 3.1 70B     | Open-source flagship    | Private deployment         | Self-hosted |
| Qwen2.5 72B       | Strong Chinese          | Chinese-language scenarios | Self-hosted |
| DeepSeek V3       | Reasoning + Chinese     | Chinese + reasoning        | Self-hosted |
| Bedrock Titan     | AWS-native              | AWS one-stop               | Mid-low     |
| Mistral Large     | European                | Multilingual               | Mid         |

**Selection dimensions:** Deployment shape (cloud / private) / context length / Chinese vs. English / reasoning needs / monthly budget.

## A.2 Orchestration / Frameworks

| Framework      | Strength            | Best Stage        | Risk               |
|----------------|---------------------|-------------------|--------------------|
| LangChain      | Largest ecosystem   | PoC fast demos    | Leaky abstractions |
| LlamaIndex     | RAG specialist      | RAG projects      | Weak Agent support |
| LangGraph      | State graph + Agent | Production Agents | Learning curve     |
| Bedrock Agents | AWS one-stop        | AWS projects      | AWS lock-in        |

| Framework  | Strength    | Best Stage                     | Risk            |
|------------|-------------|--------------------------------|-----------------|
| AutoGen    | Multi-agent | Research / complex multi-agent | Not very stable |
| CrewAI     | Role-play   | Content / research             | Not very stable |
| Native SDK | Most stable | Production critical path       | Slow to develop |

**Rule of thumb:** LangChain for PoCs; switch to native SDK + state machine in production.

### A.3 Vector Stores / Knowledge Bases

| Tool                                  | Form                            | Best Scale    | Notes                                                     |
|---------------------------------------|---------------------------------|---------------|-----------------------------------------------------------|
| Bedrock Knowledge Bases               | AWS managed                     | Mid           | OpenSearch backend by default                             |
| OpenSearch (Serverless / self-hosted) | AWS / OSS                       | Mid-large     | Strong hybrid search                                      |
| Pinecone                              | SaaS                            | Mid-large     | Easiest, but expensive                                    |
| Weaviate pgvector                     | OSS + SaaS PostgreSQL extension | Mid Small-mid | Multi-modal First choice when the customer already has PG |
| Milvus                                | OSS                             | Large         | K8s deployment                                            |
| Chroma                                | OSS                             | Small / PoC   | In-memory                                                 |
| Aurora pgvector                       | AWS Aurora                      | Small-mid     | First choice inside a VPC                                 |

**Selection dimensions:** scale / deployment / metadata filtering / what the customer already runs.

### A.4 Eval / Trace

| Tool                | Type               | Strength                             |
|---------------------|--------------------|--------------------------------------|
| Bedrock Evaluations | AWS managed        | Three eval types: Model / KB / Agent |
| LangFuse            | OSS + Cloud        | One-stop LLM trace + eval            |
| LangSmith           | LangChain official | LangChain projects                   |
| Phoenix (Arize)     | OSS                | Friendly to private deployment       |
| DeepEval            | Python library     | Drop-in inside pytest                |
| Promptfoo           | YAML-driven        | CI-friendly                          |

| Tool     | Type        | Strength                |
|----------|-------------|-------------------------|
| Ragas    | OSS         | RAG-specific            |
| Helicone | Proxy trace | OpenAI-compatible proxy |

**Rule of thumb:** LangFuse Cloud for PoCs; self-hosted LangFuse / Phoenix for enterprise.

## A.5 Data Engineering

| Tool                            | Type           | Best For              |
|---------------------------------|----------------|-----------------------|
| dbt                             | SQL transforms | In-warehouse ETL      |
| Airflow                         | Orchestration  | Complex workflows     |
| AWS Glue                        | Serverless ETL | AWS one-stop          |
| Spark / EMR                     | Big data       | TB+ scale             |
| Iceberg / Delta                 | Table formats  | Data lake             |
| Snowflake / BigQuery / Redshift | Warehouses     | Analytics             |
| Lake Formation                  | AWS            | Permission governance |
| OpenLineage                     | Lineage        | Data lineage          |

## A.6 Deployment / Runtime

| Tool                | Type              | Best For                       |
|---------------------|-------------------|--------------------------------|
| AWS Lambda          | Serverless        | Single-step LLM / simple Agent |
| ECS Fargate         | Containers        | MCP server / mid-size services |
| EKS                 | K8s               | Complex / large scale          |
| SageMaker           | Training + deploy | Self-trained models            |
| SageMaker JumpStart | One-click deploy  | Self-hosted LLMs               |
| Step Functions      | Orchestration     | Long flows / HITL              |
| API Gateway         | API gateway       | Auth + rate limiting           |
| AppConfig           | Config service    | Hot-swap prompt / model        |

## A.7 Security / Compliance

| Tool                | Purpose    |
|---------------------|------------|
| IAM Identity Center | SSO + SCIM |

| Tool               | Purpose                  |
|--------------------|--------------------------|
| KMS                | Key management           |
| Secrets Manager    | API keys / OAuth         |
| CloudTrail         | API audit                |
| Config             | Configuration compliance |
| Security Hub       | Unified security         |
| Macie              | Auto-discovery of PII    |
| Bedrock Guardrails | LLM content filtering    |
| WAF                | Web application firewall |

## A.8 IaC

| Tool           | Type                     | Best For                      |
|----------------|--------------------------|-------------------------------|
| Terraform      | Multi-cloud              | General-purpose               |
| AWS CDK        | Programmatic IaC         | AWS projects + complex logic  |
| AWS SAM        | Serverless-specific      | Lambda / API GW               |
| Pulumi         | Programmatic multi-cloud | Same idea as CDK, multi-cloud |
| CloudFormation | AWS native               | Simple stacks                 |
| Bicep          | Azure                    | Azure projects                |

## A.9 Monitoring / Alerting

| Tool                 | Purpose                     |
|----------------------|-----------------------------|
| CloudWatch           | AWS-native metrics + logs   |
| X-Ray                | AWS distributed tracing     |
| Datadog              | Unified observability       |
| Grafana / Prometheus | OSS + private deployment    |
| OpenTelemetry        | Protocol + SDK              |
| Sentry               | Error tracking              |
| PagerDuty            | On-call                     |
| Slack alerts         | Collaborative notifications |

## A.10 Customer Collaboration

| Tool          | Purpose                              |
|---------------|--------------------------------------|
| Confluence    | Customer wiki                        |
| Jira          | Customer tickets                     |
| Notion        | FDE internal notes                   |
| Slack / Teams | Real-time chat                       |
| Linear        | High-efficiency ticket management    |
| Loom          | Screen recording (explaining issues) |
| Excalidraw    | Architecture diagrams                |
| Miro          | Collaborative whiteboard             |

# Appendix B: Model / Framework / Platform Comparison Matrix

Appendix A is a “breadth cheat sheet” ; this appendix is a “depth comparison.” When you need to make a serious decision inside one tool category, look here.

## B.1 Mainstream LLMs — 9-Dimension Comparison

|                        | Claude 3.5 Sonnet | Claude 3 Opus | Claude 3 Haiku | GPT-4o         | GPT-4o-mini | Llama 3.1 70B | Qwen 2.5 72B             | DeepSeek V3 |
|------------------------|-------------------|---------------|----------------|----------------|-------------|---------------|--------------------------|-------------|
| Context                | 200K              | 200K          | 200K           | 128K           | 128K        | 128K          | 128K                     | 128K        |
| Output tokens          | 8K                | 4K            | 4K             | 16K            | 16K         | 8K            | 8K                       | 8K          |
| Chinese Reasoning      | Strong            | Excellent     | Average        | Strong         | Good        | Average       | Excellent                | Excellent   |
| Tool use               | Excellent         | Excellent     | Good           | Excellent      | Excellent   | Good          | Good                     | Good        |
| Multimodal             | Vision            | Vision        | Vision         | Vision + audio | Vision      | Text          | Vision                   | Text        |
| Price (input / M tok)  | \$3               | \$15          | \$0.25         | \$5            | \$0.15      | Self-hosted   | Self-hosted              | \$0.27      |
| Price (output / M tok) | \$15              | \$75          | \$1.25         | \$15           | \$0.6       | Self-hosted   | Self-hosted              | \$1.1       |
| Bedrock available      |                   |               |                |                |             |               | (available on AWS China) |             |



### FDE decision pattern:

Enterprise default: Claude 3.5 Sonnet (Bedrock) — balanced + AWS-native  
High QPS: Claude 3 Haiku or GPT-4o-mini  
Highest quality: Claude 3 Opus (judge / critical path)  
Chinese + private: Qwen 2.5 72B or DeepSeek V3 self-hosted  
European compliance: Mistral Large

---

## B.2 RAG Framework Comparison

| Dimension            | LangChain RAG           | LlamaIndex           | Haystack | Bedrock KB        | Build your own  |
|----------------------|-------------------------|----------------------|----------|-------------------|-----------------|
| Onboarding speed     | Mid                     | Fast                 | Mid      | Very fast         | Slow            |
| Document parsing     | Mid                     | Strong (Llama-Parse) | Strong   | Mid               | Custom          |
| Chunking strategies  | Many                    | Many                 | Many     | Default + custom  | Custom          |
| Retrieval strategies | Many                    | Very many            | Many     | Hybrid (OS)       | Custom          |
| Rerank               | External (Co-here/etc.) | Built-in + external  | Built-in | External (Cohere) | Custom          |
| Incremental updates  | DIY                     | DIY                  | DIY      | Automatic (S3)    | Custom          |
| Multi-source fusion  | Strong                  | Very strong          | Mid      | Mid               | Custom          |
| Production maturity  | Mid                     | Mid                  | High     | High              | Depends on team |
| Lock-in risk         | Low                     | Low                  | Low      | High (AWS)        | None            |

**Rules of thumb:** - PoC: LlamaIndex ships demos fastest - Production + AWS: Bedrock KB (saves half the engineering across 4 weeks) - Long-term complex RAG: build your own (use Bedrock for the retrieval layer, control orchestration yourself)

---

## B.3 Agent Framework Comparison

| Dimension         | LangGraph          | Bedrock Agents           | AutoGen                | CrewAI             | OpenAI Assistants   | Native SDK + state machine |
|-------------------|--------------------|--------------------------|------------------------|--------------------|---------------------|----------------------------|
| Programming model | State graph (DAG)  | Configuration based      | Multi-agent dialogue   | Role-play          | API + state service | Fully custom               |
| Tool integration  | Free-form          | Action Group             | Free-form              | Free-form          | Built-in + Function | Free-form                  |
| Long tasks        | LangGraph Cloud    | Pair with Step Functions | Weak                   | Weak               | Built-in threads    | Custom                     |
| HITL              | Built-in interrupt | Weak engineering         | Mid                    | Weak               | Weak                | Custom                     |
| Observability     | LangSmith          | CloudWatch + Trace       | Custom                 | Custom             | OpenAI dashboard    | Custom                     |
| Lock-in risk      | Mid                | High (AWS)               | Low                    | Low                | High (OpenAI)       | None                       |
| Best scale        | Small to large     | Mid + AWS customers      | Research / experiments | Content generation | Personal assistants | High complexity            |

**Rules of thumb:** - Customer on AWS + simple business flow → Bedrock Agents - Customer multi-cloud + complex business flow → LangGraph - Critical path + rigorous → native SDK + state machine

## B.4 Vector Store Comparison — 6 Production Dimensions

| Dimension             | Pinecone  | Weaviate   | Milvus    | OpenSearch | pgvector               | Chroma      |
|-----------------------|-----------|------------|-----------|------------|------------------------|-------------|
| Deployment            | SaaS      | SaaS + OSS | OSS (K8s) | AWS / OSS  | PG extension           | OSS / local |
| Single-DB scale       | 1B+       | 1B+        | 10B+      | 1B+        | 100M                   | 10M         |
| Hybrid search         | Supported | Supported  | Supported | Excellent  | Weak (needs extension) | Weak        |
| Metadata filtering    | Strong    | Strong     | Strong    | Excellent  | Excellent (SQL)        | Mid         |
| Multimodal            | Mid       | Strong     | Mid       | Mid        | Weak                   | Weak        |
| Incremental / updates | Easy      | Easy       | Easy      | Easy       | Trivial (UPDATE)       | Easy        |

| Dimension               | Pinecone        | Weaviate    | Milvus         | OpenSearch               | pgvector                              | Chroma      |
|-------------------------|-----------------|-------------|----------------|--------------------------|---------------------------------------|-------------|
| In-VPC deployment       | Private edition | Self-hosted | Self-hosted    | Self-hosted / Serverless | RDS / Aurora                          | Self-hosted |
| Cost                    | High            | Mid         | Mid (ops cost) | Mid                      | Very low (PG already there)           | Very low    |
| Customer already has it | Usually no      | Usually no  | Usually no     | Some AWS customers       | <b>Many customers already have PG</b> | Usually no  |

### FDE empirical decision:

Customer already has PG / Aurora → pgvector (zero extra tools)  
 Customer on AWS + mid-scale → OpenSearch Serverless  
 Scale > 500M → Milvus or Pinecone  
 PoC / small project → Chroma  
 Lowest ops effort / budget OK → Pinecone

## B.5 Eval Tool Comparison

| Dimension          | Bedrock Evaluations     | LangFuse       | Phoenix        | DeepEval           | Promptfoo         | Ragas         |
|--------------------|-------------------------|----------------|----------------|--------------------|-------------------|---------------|
| Form               | AWS managed             | OSS + Cloud    | OSS            | Python lib         | YAML CLI          | OSS           |
| Eval scope         | Model / KB / Agent      | Full LLM stack | Full LLM stack | Embedded unit test | Prompt comparison | RAG-specific  |
| LLM-as-judge       | Built-in                | Supported      | Supported      | Supported          | Supported         | Supported     |
| Auto metrics       | BLEU / ROUGE / accuracy | Custom         | Custom         | Many built-in      | Custom            | 6 RAG metrics |
| Dataset management | S3 jsonl                | UI + API       | UI             | Code               | YAML              | Code          |
| Online trace       | No (static)             | Strong         | Strong         | No                 | No                | No            |
| CI-friendly        | Mid                     | Mid            | Mid            | Strong             | Excellent         | Mid           |

| Dimension | Bedrock<br>Evaluations | LangFuse              | Phoenix  | DeepEval | Promptfoo | Ragas    |
|-----------|------------------------|-----------------------|----------|----------|-----------|----------|
| Cost      | Per token + judge      | OSS free / Cloud paid | OSS free | OSS free | OSS free  | OSS free |

### Typical combination:

LangFuse (trace + online) + Promptfoo (CI) + Bedrock Evaluations (compliance acceptance)

## B.6 Deployment Platform Comparison — From an LLM-app Lens

| Dimension         | Lambda                         | ECS Fargate                  | EKS                    | SageMaker<br>Endpoint | Step<br>Functions      |
|-------------------|--------------------------------|------------------------------|------------------------|-----------------------|------------------------|
| Startup latency   | Cold start 100ms-3s            | Container startup in seconds | Pod startup in seconds | Startup in seconds    | Orchestration overhead |
| Max single run    | 15 min                         | None                         | None                   | None                  | 1 year                 |
| Concurrency model | Automatic                      | Self-managed                 | Self-managed           | Self-managed          | N/A                    |
| Best for          | Single LLM call / simple Agent | Long-running server / MCP    | Complex clusters       | Self-hosted models    | Long flows / HITL      |
| Cost shape        | Pay-per-use                    | Per container                | Per node               | Per endpoint          | Per state transition   |
| State             | Stateless                      | Stateless / EFS              | StatefulSet            | Stateless             | State held in SF       |
| Image size        | 250MB / 10G (image)            | Large                        | Large                  | Large                 | N/A                    |

**Rules of thumb:** - Simple RAG / API → Lambda - MCP server / Agent service → ECS Fargate  
- Long flow + HITL → Step Functions + Lambda - Self-hosting 70B+ models → SageMaker / EKS + GPU

## B.7 Programming Languages / Runtimes

| Dimension           | Python            | TypeScript        | Go                     | Rust                     |
|---------------------|-------------------|-------------------|------------------------|--------------------------|
| LLM SDK maturity    | Top-tier          | Good              | Good                   | Early                    |
| Data processing     | Excellent         | Mid               | Strong                 | Strong                   |
| Async concurrency   | asyncio (mid)     | Native, strong    | goroutines, strong     | tokio, strong            |
| Deployment size     | Large             | Mid               | Tiny (single binary)   | Tiny                     |
| Hiring              | Very easy         | Easy              | Mid                    | Hard                     |
| FDE recommended use | Data / Eval / PoC | Frontend / Lambda | API / high-concurrency | Critical path / embedded |

**FDE default:** Python + TypeScript dual stack covers the vast majority of scenarios.

## B.8 Decision Synthesis Table

Customer scenario → recommended stack

AWS mid-size insurer + document RAG

Model: Bedrock Claude 3.5 Sonnet  
Framework: Bedrock Knowledge Bases + custom Lambda  
Vector: OpenSearch Serverless  
Eval: LangFuse Cloud + Promptfoo CI  
Deploy: Lambda + API Gateway  
IaC: AWS CDK  
Monitor: CloudWatch + X-Ray  
Security: IAM Identity Center + Bedrock Guardrails

Existing PG database + Chinese finance + intranet

Model: DeepSeek V3 / Qwen 2.5 72B self-hosted (vLLM)  
Framework: Native SDK + state machine  
Vector: pgvector  
Eval: Phoenix (self-hosted) + custom judge  
Deploy: K8s  
IaC: Terraform  
Monitor: Prometheus + Grafana  
Security: Internal LDAP + custom audit

Multinational retail + multi-SaaS integration + Agent

Model: Claude 3.5 Sonnet

Framework: LangGraph + MCP servers

Vector: Pinecone

Eval: LangFuse + DeepEval

Deploy: ECS Fargate

IaC: Pulumi (multi-cloud)

Monitor: Datadog

Security: Okta + per-call OAuth

---

# Appendix C: Eval Set Design Templates

Eval templates for the four most common LLM task types: 1. RAG question answering 2. Agent task execution 3. Text classification 4. Structured information extraction

Each template includes a jsonl schema, field semantics, sampling strategy, evaluation method, and CI example.

---

## C.1 Common Design Principles

Four tiers of Eval sets

---

|             |              |                                                     |
|-------------|--------------|-----------------------------------------------------|
| Seed Set    | (5-20 cases) | handwritten, ultra-fast regression, never edited    |
| Golden Set  | (100-300)    | annotated by customer SMEs, the acceptance baseline |
| Adversarial | (50-150)     | counterexamples / edge cases / privilege escalation |
| Production  | (streaming)  | sampled from prod → re-injected weekly              |

Common schema fields:

```
{
  "id": "unique stable id",
  "category": "case category for slicing",
  "input": "the input to the system",
  "expected": "what success looks like",
  "metadata": {
    "source": "where this case came from",
    "added_at": "2026-05-22",
    "reviewer": "reviewer email",
    "difficulty": "easy / medium / hard",
    "is_adversarial": false
  }
}
```

Why jsonl rather than csv: nested fields, multi-line text, and long prompts are common

— csv escaping turns into a nightmare.

---

## C.2 Template 1: RAG QA Eval

### Schema

```
{
  "id": "rag-001",
  "category": "policy_lookup",
  "input": {
    "question": "Does the customer's health insurance include hospitalization allowance? Pol
001"
  },
  "expected": {
    "must_contain_keywords": ["hospitalization allowance", "200 RMB/day", "30 days"],
    "must_not_contain": ["excluded", "not covered"],
    "reference_doc_id": "POL-HC-002-v3.pdf",
    "reference_section": "Chapter 3, Clause 2",
    "reference_answer": "Includes hospitalization allowance at 200 RMB per day, up to 30 days"
  },
  "metadata": {
    "source": "Customer SME 2026-05",
    "difficulty": "medium"
  }
}
```

### Evaluation logic (Python pseudocode)

```
def evaluate_rag(case, system_output):
    score = {}

    # 1. Hard rules — keywords
    score['kw_match'] = all(
        kw in system_output['answer']
        for kw in case['expected']['must_contain_keywords']
    )
    score['no_forbidden'] = all(
        kw not in system_output['answer']
        for kw in case['expected']['must_not_contain']
    )

    # 2. Was the retrieval source correct? (the most common RAG failure)
    score['retrieval_correct'] = (
        case['expected']['reference_doc_id']
        in [d['doc_id'] for d in system_output['retrieved_docs']]
    )
```



```

)

# 3. Semantic similarity (embedding cosine)
score['semantic_sim'] = cosine(
    embed(case['expected']['reference_answer']),
    embed(system_output['answer'])
)

# 4. LLM-as-judge (most expensive — reserve for key cases)
if case['metadata']['difficulty'] == 'hard':
    score['llm_judge'] = call_judge(
        question=case['input']['question'],
        reference=case['expected']['reference_answer'],
        actual=system_output['answer']
    )

return score

```

## Sampling strategy

Seed Set:

- The 5 most frequent questions (the ones the business asks most)
- 5 explicit counterexamples (the ones we must never get wrong)

Golden Set 200 cases — split like this:

- 60% mainstream questions (high-frequency cases the SME named)
- 20% edge cases (cross-policy / multi-insured / historical policies)
- 10% long-tail rare questions
- 10% counterexamples (out-of-scope / PII / things we should refuse)

Adversarial 50 cases:

- Rephrasing / typos / synonym swaps
- Prompt injection
- Cross-domain (asking about another insurer)

## CI thresholds

```

# .github/workflows/rag-eval.yml
- name: Run RAG eval
  run: |
    python eval/run_rag.py --set golden --output result.json
    python eval/check.py \
      --result result.json \
      --thresholds \
        kw_match>=0.95 \
        retrieval_correct>=0.92 \

```

```
semantic_sim>=0.80 \
llm_judge>=0.85
```

---

## C.3 Template 2: Agent Task Execution Eval

### Schema

```
{
  "id": "agent-claim-001",
  "category": "auto_settle_simple",
  "input": {
    "claim_id": "CLM-2026-001",
    "user_message": "I want to claim last week's outpatient medication expenses, 580 RMB",
    "context": {
      "customer_id": "C123",
      "policy_id": "P-OPD-001"
    }
  },
  "expected": {
    "task_completed": true,
    "tool_calls": [
      {"tool": "get_policy", "params_must_include": ["P-OPD-001"]},
      {"tool": "validate_amount", "params_must_include": [580]},
      {"tool": "create_settlement", "must_appear": true}
    ],
    "tool_calls_must_not": ["transfer_funds"],
    "final_state": {
      "claim_status": "approved",
      "approved_amount": 580
    },
    "max_steps": 8,
    "max_cost_usd": 0.05
  },
  "metadata": {
    "difficulty": "easy"
  }
}
```

### 5-dimension evaluation

1. Task completion  
final\_state == expected.final\_state ?
2. Path soundness  
Do tool\_calls satisfy the "must / must-not" constraints?

Is the step count  $\leq$  max\_steps?

3. Tool-call correctness

Are the parameters of each tool call right?

4. Side effects

Did it call any tool on the must-not list?

Did it cause any external state pollution?

5. Cost

Total tokens / dollars  $\leq$  max\_cost\_usd?

## Python evaluator

```
def evaluate_agent(case, trace):
    """trace is the agent's full execution trajectory"""
    tool_calls = trace['tool_calls']

    # 1. Task completion
    task_done = trace['final_state'] == case['expected']['final_state']

    # 2. Tool sequence
    must_tools = case['expected']['tool_calls']
    must_present = all(
        any(
            tc['tool'] == m['tool']
            and all(p in tc['params'] for p in m.get('params_must_include', []))
            for tc in tool_calls
        )
        for m in must_tools
    )

    # 3. Forbidden tools
    forbidden_used = any(
        tc['tool'] in case['expected']['tool_calls_must_not']
        for tc in tool_calls
    )

    # 4. Steps / cost
    within_steps = len(tool_calls) <= case['expected']['max_steps']
    within_cost = trace['total_cost_usd'] <= case['expected']['max_cost_usd']

    return {
        'task_done': task_done,
        'must_tools_present': must_present,
        'no_forbidden': not forbidden_used,
        'within_steps': within_steps,
```

```

        'within_cost': within_cost,
        'overall_pass': all([
            task_done, must_present, not forbidden_used,
            within_steps, within_cost
        ])
    }

```

## Sampling strategy

Golden 100 cases — split like this:

- 40% standard cases (simple cases the agent should auto-complete)
- 30% HITL cases (where escalation is the correct outcome)
- 15% abnormal inputs (missing fields / ambiguous reference)
- 15% counterexamples (out-of-scope / injection / adversarial)

## C.4 Template 3: Classification / Routing Eval

### Schema

```

{
  "id": "intent-001",
  "category": "core_intents",
  "input": {
    "user_message": "My car was in an accident — what do I do?"
  },
  "expected": {
    "intent": "claim_intake",
    "confidence_min": 0.8
  },
  "metadata": {
    "difficulty": "easy"
  }
}

```

### Metrics

```

# Multi-class classification
from sklearn.metrics import classification_report, confusion_matrix

y_true = [c['expected']['intent'] for c in cases]
y_pred = [run_classifier(c['input']) for c in cases]

print(classification_report(y_true, y_pred))
print(confusion_matrix(y_true, y_pred))

```

```
# CI thresholds: macro F1 ≥ 0.85, per-class recall ≥ 0.80
```

## Key sampling

- At least 30 cases per intent (upsample if imbalanced)
  - Confusion classes with asymmetric business cost get tagged separately  
e.g. claim\_intake misclassified as general\_query → poor customer experience  
general\_query misclassified as claim\_intake → wasted human effort  
The first is more costly; test it heavily.
- 

## C.5 Template 4: Structured Extraction Eval

### Schema

```
{
  "id": "extract-invoice-001",
  "category": "ocr_invoice",
  "input": {
    "image_url": "s3://bucket/invoice-123.png",
    "ocr_text": "..."
  },
  "expected": {
    "invoice_no": "INV-2026-0312",
    "total_amount": 1280.50,
    "currency": "CNY",
    "issue_date": "2026-03-12",
    "vendor": {
      "name": "ABC Trading Co., Ltd.",
      "tax_id": "91110000XXXXXXXXX"
    }
  },
  "metadata": {
    "difficulty": "medium",
    "image_quality": "good"
  }
}
```

### Field-level evaluation

```
def field_level_eval(case, output):
    expected = case['expected']
    score = {}
    for field in flatten(expected).keys():
        e = get(expected, field)
```

```

    a = get(output, field)
    if isinstance(e, str):
        score[field] = (e == a)
    elif isinstance(e, (int, float)):
        score[field] = abs(e - a) / max(abs(e), 1) < 0.001
    elif isinstance(e, dict):
        # nested, recurse
        ...
    return score

# Total = weighted average across critical fields
weights = {
    'invoice_no': 1.0,
    'total_amount': 1.0,    # critical — numbers must be exact
    'currency': 0.5,
    'issue_date': 0.8,
    'vendor.name': 0.6,
    'vendor.tax_id': 1.0    # compliance-critical
}

```

---

## C.6 A Full Eval Project Layout

```

project/
├── eval/
│   ├── datasets/
│   │   ├── seed.jsonl           (10 cases, version-locked)
│   │   ├── golden_v1.jsonl      (200 cases, SME-labeled)
│   │   ├── golden_v2.jsonl      (250 cases, follow-on additions)
│   │   ├── adversarial.jsonl    (50 cases)
│   │   └── prod_sampled_2026w20.jsonl (weekly re-injection)
│   ├── runners/
│   │   ├── run_rag.py
│   │   ├── run_agent.py
│   │   └── run_classifier.py
│   ├── judges/
│   │   ├── llm_judge_prompts.yaml
│   │   └── rules.py
│   ├── reports/
│   │   ├── 2026-05-22_release_v1.2.html
│   │   └── ...
│   └── README.md
├── .github/workflows/
└── eval.yml

```

---

## C.7 Annotation Workflow — Getting Customer SMEs to Work Efficiently

Step 1: FDE prepares 100 seed questions  
(drawn from customer docs / historical tickets / support logs)

Step 2: SMEs annotate in Streamlit / Excel  
(UI shows the question + your draft answer + their revisions)

Step 3: Two-person review  
(one SME labels + a second one re-checks 20%)

Step 4: Commit + version-lock (golden\_v1)

Step 5: Monthly increments  
(production cases that went wrong → adversarial set)

**Key:** SMEs are not writing answers. They are **proofreading answers and codifying the judgment criteria.**

---

## C.8 Eval Set Lifecycle Management

An Eval set is not "build once and shelve it" — it's alive.

Rules:

- Add 20-50 new cases per month (from prod / bugs / new requirements)
  - Quarterly audit (any stale cases / wrong answers?)
  - Never delete the seed (unless that line of business is retired)
  - Major version upgrade → new set, keep old one as the baseline
- 

## C.9 An Eval Checklist

Pre-release, must-check:

- ☐ Every seed case passes
  - ☐ Golden set headline metric  $\geq$  threshold
  - ☐ Adversarial counterexamples refused at  $\geq 90\%$
  - ☐ Cost metric  $\leq$  budget
  - ☐ Latency metric  $\leq$  SLO
  - ☐ Slices (returning / new / high-value / low-value customers) all pass
  - ☐ Compared to previous version → no regressions
- 

[← Previous Appendix: Comparison Matrix](#) · [Next Appendix: Customer Kickoff Pack](#) →

# Appendix D: Customer Kickoff Pack

## — 12 Templates Ready for Week One

When you take on a new customer or new project, here are the 12 templates you'll need in week one. Each one comes with "when to use / how to use / template body."

Copy → adapt → use.

---

### Template 1: Discovery Outline (for interviews)

**When:** 1-7 days after kickoff **Audience:** 1-2 people each from customer business / IT / security

Discovery Interview — [Customer] / [Interviewee]

Date: 2026-XX-XX

Interviewer: [FDE name]

Interviewee: [Name / dept / title]

Duration: 45 minutes

———— Business background (10 min) ————

Q1. What does your team's daily core workflow look like?

Q2. What is the most painful point in this workflow today? How many people / how much time?

Q3. What change do you expect this project to deliver? How will you measure success?

———— Data (10 min) ————

Q4. What data is involved? Where does it live today?

Q5. Who owns the data? What's the process to obtain it?

Q6. Has anyone tried to do something similar with this data in the past? How did it go?

———— Technology (10 min) ————

Q7. What's your IT stack? (cloud / on-prem / hybrid)

Q8. What AI / ML / LLM efforts already exist?

Q9. Are there explicit technical constraints? (must be in-VPC / must be open-source / must support Chinese)



————— Compliance and decision-making (10 min) —————  
Q10. What compliance constraints are involved? (DJCP / data security law / sector regulator)  
Q11. Who is in the decision chain? Who signs off on the production launch?  
Q12. Any audits / inspections planned in the next six months?

————— Project expectations (5 min) —————  
Q13. What timeline do you expect?  
Q14. Who will operate this after the project ends?  
Q15. Anything we haven't asked but you want to mention?

#### FDE private-notes fields:

- Interviewee's enthusiasm for the project: high / medium / low
- Real decision-maker, or messenger?
- What "we cannot do this" red lines did they mention?
- Any "last attempt that failed" stories?

## Template 2: Discovery Summary Report

**When:** end of Discovery week **Audience:** customer leadership + project team

```
# [Customer] AI Project Discovery Report
## I. Business state today
- Current workflow diagram (one A4 page)
- Pain-point list (ranked, quantified loss)
- Prior attempts and lessons

## II. Problem boundaries
- We will solve: ...
- We will not solve: ... (state explicitly)

## III. Data state
- Data inventory (table + fields + scale + update frequency)
- Data availability assessment (red / yellow / green)
- Data governance risks

## IV. Technical constraints
- Deployment shape (cloud / private / offline)
- Must-use / cannot-use list
- Security and compliance requirements

## V. Recommended approaches (2-3)
- Option A: simple / fast / 6 weeks    covers 60% of cases
- Option B: medium / 12 weeks          covers 85% of cases
- Option C: complete / 20 weeks        covers 95% of cases

## VI. Dead ends to avoid
```

- Path X — reason
- Path Y — reason

## ## VII. Phase-1 SOW draft

- Scope
  - Acceptance
  - Milestones
  - Resource needs
- 

## Template 3: PoC SOW Skeleton (6-8 weeks)

# Statement of Work — [Project] PoC

### ## 1. Project purpose

[One sentence: validate whether X is feasible]

### ## 2. Scope — In-Scope

- Feature A
- Feature B
- Data source X (read-only)

### ## 3. Scope — Out-of-Scope (state explicitly)

- Feature C is excluded
- No SaaS Z integration
- No mobile app

### ## 4. Deliverables

- (1) PoC application (demo-ready)
- (2) 200-case Eval set + pass-rate report
- (3) Architecture diagram + deployment docs
- (4) Retrospective + follow-on roadmap

### ## 5. Acceptance criteria

- Eval headline metric  $\geq X\%$
- Single-response P95 latency  $\leq Y$  seconds
- Cost per call  $\leq Z$  RMB
- 5 main demo scenarios all pass

### ## 6. Timeline

W1: Discovery wrap-up + Eval set v0

W2-W3: Scaffold + first runnable version

W4-W5: Iterate + raise Eval scores

W6: Acceptance + Demo + summary

```
## 7. Responsibilities
- Customer: SMEs 8h/week + IT 4h/week + data preparation
- FDE: 1 person full-time

## 8. Assumptions and dependencies
- AWS account + quota ready by W1
- VPC / network paths opened by D-3
- Data sample (de-identified) available by W1

## 9. Change management
- Scope changes require written confirmation from both sides
- Changes trigger re-estimation of timeline and price

## 10. Pricing
[Per company pricing rules]
```

---

## Template 4: Eval Set v0 (jsonl starter)

```
{ "id": "E001", "category": "core", "input": {...}, "expected": { "must_contain": [...] }, "metadata": {} }
{ "id": "E002", "category": "core", "input": {...}, "expected": { "must_contain": [...] }, "metadata": {} }
{ "id": "E003", "category": "edge", "input": {...}, "expected": { "must_not_contain": [ "excluded" ] }, "metadata": {} }
{ "id": "E004", "category": "adversarial", "input": {...}, "expected": { "refuse": true }, "metadata": {} }
```

**The 5 things to ask for on day one:** 1. SME hand-writes 5-10 most-common questions + expected answers 2. 5 counterexamples ( “if the system gets these wrong, something blows up” ) 3. 5 edge cases 4. 5 adversarial cases 5. 1 written judgment-criteria spec ( “what counts as correct” )

---

## Template 5: Security / Compliance Questionnaire Response Template

**When:** customer security team sends an “AI Security Survey”

25 frequently-asked questions + skeleton answers

Q1. Does data leave the customer VPC?

A: It does not. Model inference goes through a VPC Endpoint; logs go to the customer's S3

Q2. Is customer data used to train models?

A: Bedrock / Claude API have data isolation statements (see Anthropic / AWS contracts).

Q3. How is PII protected from leakage?

A: (1) Bedrock Guardrails sensitive-info filter  
(2) Application-layer PII scanning  
(3) Output auditing

Q4. Who can access what?

A: IAM Identity Center + SCIM; role list in Doc 4.2.

Q5. How are issues traced?

A: End-to-end trace\_id, CloudTrail + Bedrock Logs + application logs retained 90 days (configurable up to 7 years, KMS-encrypted).

Q6. How is prompt injection defended?

A: Guardrails + input sanitization + tool dry\_run + HITL.

... (truncated)

---

## Template 6: Architecture Diagram (Mermaid starter)

Save as docs/architecture.md.

Generate with mermaid; export to PNG directly during customer audits.

flowchart TB

```
User[Business User] --> ALB[ALB / API Gateway]
ALB --> App[App Layer - Lambda/ECS]
App --> Guard[Bedrock Guardrails]
Guard --> Bedrock[Bedrock - Claude]
App --> KB[Bedrock Knowledge Base]
KB --> S3[S3 - Customer Docs]
KB --> OS[OpenSearch Serverless]
App --> DDB[DynamoDB - Session State]
App --> CW[CloudWatch + X-Ray]
App --> Audit[Audit Log → S3]
```

```
subgraph VPC[Customer VPC]
    ALB
    App
    DDB
end
```

```
subgraph Endpoints[VPC Endpoints]
    Bedrock
    S3
    DDB
end
```

---

## Template 7: Risk Register

Risk Register — [Project]

Updated: 2026-XX-XX

---

| ID | Risk                          | Impact | Probability | Mitigation + Owner  | Status      |
|----|-------------------------------|--------|-------------|---------------------|-------------|
| R1 | Data delivery delayed         | High   | Med         | W1 escalate to IT   | Open        |
| R2 | Bedrock quota insufficient    | Med    | High        |                     | Pre-request |
|    |                               | Closed |             |                     |             |
| R3 | SME bandwidth too low         | Med    | Med         | Lock 8h/week        | Open        |
| R4 | Guardrails false positives    | Med    | Med         | Eval + human review | Open        |
| R5 | No customer owner for handoff | High   | Med         | Train at T-3 weeks  | Open        |

---

## Template 8: Weekly Report Template

# Week N Report — [Project]

Date: 2026-XX-XX

## What we did this week

- [Milestone] Completed X
- [Eval] Headline metric improved from 70% to 78%
- [Code] Fixed 3 bugs

## Headline numbers

| Metric                      | Last week | This week | Target  |
|-----------------------------|-----------|-----------|---------|
| Golden Eval pass rate       | 70%       | 78%       | ≥85%    |
| P95 single-response latency | 4.2s      | 3.1s      | ≤3s     |
| Average cost per call       | \$0.08    | \$0.06    | ≤\$0.05 |

## Plan for next week

- Resolve Bedrock throttling (R2)
- Run 50 Adversarial cases
- Review with customer SMEs

## Blockers / customer support needed

- [Urgent] Data export for table X is not yet ready (Owner: customer IT, due Wed)
- [Standard] Awaiting security team's IAM-role review

## Risk changes

- R3 elevated to red (SME participated only 4h this week)
-

## Template 9: Runbook Skeleton (3 weeks before Handoff)

```
# Runbook — [Project]

## 1. System overview (one A4 page)
- Architecture diagram
- Main flow
- Key dependencies

## 2. Deploy / rollback
### 2.1 Normal deploy
```bash
./scripts/deploy.sh prod --version=v1.2.3
```

### 2.2 Rollback

```
./scripts/rollback.sh prod --target=v1.2.2
```

## 3. Top-10 incident SOPs

- SOP-001: Error rate spikes → ...
- SOP-002: Latency spikes → ...
- SOP-003: Cost anomaly → ...
- ...

## 4. How to run Evals

```
python eval/run.py --set golden --output report.html
```

Thresholds: kw\_match >= 0.95, semantic >= 0.80

## 5. Key configuration

- AppConfig: prompt\_template\_v3
- Model ID: us.anthropic.claude-3-5-sonnet-...
- KB ID: KB-XXXXX

## 6. Data / KB update SOP

1. Upload to S3: s3://docs/incoming/
2. Trigger sync: aws bedrock-agent start-ingestion-job ...
3. Run Eval to verify

## 7. Escalation

- L1 (customer ops): @ops-channel
- L2 (FDE on-call): +XX-XXXX-XXXX (24h)
- L3 (architect): emergency only

---

### ## Template 10: Acceptance Checklist (signed by both sides)

---

[Project] Acceptance Sheet Date: 2026-XX-XX

- ☐ 1. Functional acceptance — all 5 demo scenarios pass [✓] Scenario 1: ... [✓] Scenario 2: ... [✓] Scenario 3: ... [✓] Scenario 4: ... [✓] Scenario 5: ...
- ☐ 2. Metric acceptance [✓] Golden Eval pass rate  $\geq$  85% actual: 88% [✓] P95 latency  $\leq$  3s actual: 2.4s [✓] Cost per call  $\leq$  \$0.05 actual: \$0.04
- ☐ 3. Compliance acceptance [✓] PII check passed [✓] Audit logs complete [✓] Security team sign-off
- ☐ 4. Documentation acceptance [✓] Architecture diagram [✓] Runbook [✓] Eval report [✓] Retrospective + roadmap
- ☐ 5. Handoff [✓] Training complete (4h, 3 attendees) [✓] Shadow operations 5 days [✓] Customer owner can deploy / rollback independently

---

Customer signature: Date: FDE signature: Date:

---

### ## Template 11: Project Retrospective Template (write within 1 week)

```markdown

# [Project] Retrospective (internal)

Date: 2026-XX-XX

Author: [FDE]

#### ## I. What we did

[One paragraph — do not repeat the project plan]

#### ## II. Headline numbers

- Time: planned 12w, actual 13w
- Budget: \$XX, actual \$XX
- Eval: headline metric 88%
- Customer satisfaction: 4.5/5

#### ## III. What went right (3 things)

1. Built the Eval set in W1 → no idle spinning all the way through
2. Chose Bedrock KB over building our own → saved 4 weeks

3. Started Handoff at T-3 → 0 Pls after launch

#### ## IV. What went wrong (3 things)

1. OAuth flow change wasn't surfaced until W3 → 1-week slip
2. First-version Agent had 35 tools → accuracy chaos; trimmed to 12 by W7
3. Did not lock SME bandwidth → progress stalled in W4-5

#### ## V. Decision cards (3)

1. "When the customer asks X, answer Y" → ...
2. "When Eval is on the line, prioritize Y" → ...
3. "When signal Z appears, roll back immediately" → ...

#### ## VI. Reusable assets

- Code: insurance-rag-starter v3.2
- Docs: Insurance industry Discovery template
- Eval: 50-case golden template for insurance QA

#### ## VII. Advice for the next FDE

- For insurance customers, start from the [v3.2 template]
- Don't dance around DJCP-3 questions; face them in week one

---

## Template 12: Customer Stakeholder Map

### Stakeholder Map — [Customer]

---

#### Decide circle

- ★ CEO / VP — Sponsor — quarterly check-in — cares about ROI
- ★ CIO — Decider — monthly review — cares about stability + compliance

#### Drive circle

- Business owner — weekly meeting — cares about business outcomes
- Tech lead — weekly meeting — cares about architecture + delivery
- Security lead — monthly + key milestones — cares about compliance

#### Do circle

- SMEs (3 people) — annotate Evals / review answers
- Data engineers (2 people) — data delivery
- App engineers (2 people) — integrate customer internal systems

#### Informed circle

- Customer service director — cares about launch impact
- HR — cares about staffing / process changes

---

Key relationship signals:



- Tension between business owner ↔ tech lead? → high project risk
  - When does the security lead first review? → the earlier the better
  - Is the Sponsor actually engaged (meeting attendance rate)?
- 

## Usage Summary

|        |                               |                                                           |
|--------|-------------------------------|-----------------------------------------------------------|
| D-7    | Take on the customer          | → Template 1 (Discovery outline)                          |
| D-3    | Discovery wrap-up             | → Template 2 (Summary report)                             |
| D-1    | Pricing                       | → Template 3 (SOW)                                        |
| D+1    | First interviews + annotation | → Template 4 (Eval v0)                                    |
| W1     | Security review               | → Template 5 (Questionnaire response)                     |
| W1     | Architecture submission       | → Template 6 (Mermaid)                                    |
| W1     | Project kickoff               | → Template 7 (Risk register) + Template 12 (Stakeholders) |
| W2-W11 | Daily collaboration           | → Template 8 (Weekly report)                              |
| W9-W11 | Handoff prep                  | → Template 9 (Runbook)                                    |
| W12    | Acceptance                    | → Template 10 (Checklist)                                 |
| W12+1  | Retrospective                 | → Template 11 (Retro)                                     |

**The whole 12-template paperwork pack ≈ 8-10 hours to build once**, and you reuse it for the rest of your career.

---

# Glossary

Alphabetical order. Source pages in [bibliography.md](#).

---

## A

**A. Lawrence** — Author of Forward Deployed Engineer Rule Book (Oct 2025), the only standalone English book on FDE.

**Agent (AI Agent)** — An LLM application that plans, calls tools, and iterates autonomously. Subject of Part VI.

**Agent Toolset** — The set of tools (read/write/exec/network/...) an agent can call. Topic of Ch 14.

**Anthropic** — Maker of Claude.

**AWS GenAI Innovation Center** — Amazon’ s FDE-equivalent organization. Officially uses “Forward deployment engineering” ; published 45-day / 73% case figures.

## B

**Bob McGrew** — Former Chief Research Officer at OpenAI. Origin of “Sell the outcome, not the product.”

## C

**Conikeec** — Substack author of The FDE Playbook: A Practitioner’ s Field Manual.

**Confluence** — Atlassian’ s enterprise wiki. Almost always present in customer documentation; unavoidable in the Discovery phase.

## D

**Discovery** — The first FDE phase: observe the customer’ s workflow, read existing systems, find the real problem. Topic of Part II.

**Dual-Credential Training** — Lawrence’ s concept: an FDE must hold both engineering credibility and domain credibility.

## E

**ETL (Extract-Transform-Load)** — Classic data-pipeline operations. Topic of Ch 9.

**Eval / Eval Set / Eval-driven** — The evaluation set; the practice of writing the eval set before feature code. Spine of Ch 8.

**Embedded Problem Solver** — Lawrence’ s framing: an FDE is embedded inside the customer’ s problem.

## F

**FDE (Forward Deployed Engineer)** — An engineer who lives at the customer site and ships AI/software into production. Subject of this book.

**Fine-tuning** — Continuing LLM training on customer-private data. A choice point in Ch 7.

**Fix Forward** — Lawrence’ s principle: solve on-site; do not drag the problem back to HQ.

**Foundry** — Palantir’ s flagship platform — ontology + data integration + application layer. Reference frame for Ch 9.

**Forward Deployment Engineering** — AWS’ s official term; equivalent to FDE elsewhere.

## G

**GTM (Go-To-Market)** — How a product reaches customers.

## H

**Handoff** — The move at FDE departure: returning the project to the customer’ s internal team. Topic of Ch 16.

## I

**Immersion Before Judgment** — Lawrence’ s principle: live in the customer workflow before designing.

## J

**JD (Job Description)** — A job posting.

## L

**LLM (Large Language Model)** — Assumed to be familiar; the book assumes you can call APIs.

## M

**MCP (Model Context Protocol)** — Anthropic-led open protocol standardizing how agents connect to external tools. Topic of Ch 15.

**Morgan Stanley (MS)** — OpenAI’ s public case study; source of the 6-8+4 cycle reference.

## N

**Nabeel Qureshi** — Early Palantir; the most influential English-language interpreter of FDE.

## O

**Ontology** — Formal modeling of customer business concepts (entities, properties, relationships, actions). Palantir’ s core abstraction; primary battlefield of the field-delivery FDE track. Topic of Ch 9.

**Outcome (in “Sell the outcome” )** — The customer’ s business result (revenue, cost, risk), as opposed to a feature, tool, or demo.

## P

**Palantir** — Inventor of the FDE model.

**PoC (Proof of Concept)** — Concept verification. Ch 12 focuses on the PoC → production cut-over criteria.

**Private Deployment / VPC Deployment** — Customer-private / VPC deployment. Topic of Ch 10.

## R

**RAG (Retrieval-Augmented Generation)** — A choice point in Ch 7.

**Rule Book** — Shorthand for Lawrence’ s Forward Deployed Engineer Rule Book.

## S

**SCIM (System for Cross-domain Identity Management)** — Enterprise identity-sync standard. Topic of Ch 11.

**Sell the outcome, not the product** — Bob McGrew's first law. Topic of Ch 2.

**SOW (Statement of Work)** — Project scope document. Topic of Ch 5.

**SSO (Single Sign-On)** — Almost always required in customer compliance. Topic of Ch 11.

## T

**T-shaped Growth** — Engineering depth + domain depth. Topic of Ch 17.

**Trace / Tracing** — Distributed tracing. Topic of Ch 13.

## V

**VPC (Virtual Private Cloud)** — A common substrate for customer-private deployment.

## Other

**Anti-pattern** — A common but harmful practice. End-of-chapter checklist in this book.

**Action Checklist** — Copy-do items for the week. End-of-chapter checklist in this book.

---

# Bibliography

Full raw URL list and search-method log live in `../research/05-sources.md`.  
This document re-orders sources by importance and type, leading with the few sources cited repeatedly throughout this book.

---

## 1. Most-Cited Core Sources

### Books

| # | Author / Title                                                                              | Notes                                                                  |
|---|---------------------------------------------------------------------------------------------|------------------------------------------------------------------------|
| 1 | A. Lawrence, Forward Deployed Engineer Rule Book, AI Circuit, Oct 2025. ISBN 9781970328097. | The only standalone English FDE book. Primary reference for Chapter 6. |

### Primary Interviews & Originals

| # | Source                                   | Title / Content                                                                                                   |
|---|------------------------------------------|-------------------------------------------------------------------------------------------------------------------|
| 2 | Bob McGrew × Y Combinator (2025)         | YC interview — origin of “Sell the outcome, not the product”                                                      |
| 3 | AWS Generative AI Innovation Center site | Public 45-day / 73% / 22-partner figures                                                                          |
| 4 | Palantir AI FDE official documentation   | <a href="https://palantir.com/docs/foundry/ai-fde/overview">https://palantir.com/docs/foundry/ai-fde/overview</a> |

### Long-Form Analyses

| # | Author                 | Title                                                             |
|---|------------------------|-------------------------------------------------------------------|
| 5 | Conikeec (Substack)    | The FDE Playbook: A Practitioner' s Field Manual                  |
| 6 | Fresh HQ AI / brgr.one | The FDE Playbook for AI (transcribed from McGrew' s YC interview) |
| 7 | Nabeel Qureshi         | Reflections on Palantir (early-Palantir practice)                 |

## 2. The May 4, 2026 Inflection Event

| Source                           | Title                                                                                             |
|----------------------------------|---------------------------------------------------------------------------------------------------|
| TechCrunch (2026-05-04)          | Anthropic and OpenAI are both launching joint ventures for enterprise AI services                 |
| MarkTechPost (2026-05-20)        | What is a Forward Deployed Engineer: The AI Role OpenAI, Anthropic, and Google Are Hiring in 2026 |
| MindStudio                       | Palantir' s Forward Deployed Engineer Model Drove 640% Returns                                    |
| Revolution in AI                 | The Palantir Model That Anthropic and OpenAI Are Now Copying                                      |
| SuperML.dev                      | OpenAI and Anthropic Adopted the Palantir Playbook                                                |
| LinkedIn Pulse                   | The FDE Wars Are Here: Anthropic and OpenAI Bet \$5.5 Billion                                     |
| Forbes Tech Council (2025-12-29) | The Rise of the Forward Deployed Engineer                                                         |

## 3. Industry Blogs & Playbooks

| Source                                  | Title                                                 |
|-----------------------------------------|-------------------------------------------------------|
| Jediteck                                | What is a Forward Deployed Engineer?                  |
| Kiadev                                  | What Is a Forward Deployed Engineer (FDE)?            |
| Oflight (Japan)                         | FDE: From Palantir' s Original Playbook to 2026 Guide |
| LinkedIn — Zamir Akimbekov (2025-11-04) | The FDE Playbook                                      |
| fderole.com                             | Definitive Role Guide                                 |

## 4. Chinese-Language Community Sources (referenced for the China-edition Part VI)

| Platform                             | Title                                                                                                                        |
|--------------------------------------|------------------------------------------------------------------------------------------------------------------------------|
| Zhihu                                | What is Palantir’ s “Forward Deployed Engineer” ?                                                                            |
| Xueqiu                               | FDE — The Core Secret of the Palantir Model (Chinese rendition of Nabeel’ s view)                                            |
| Tencent Cloud Dev Community<br>51CTO | Palantir Deep Dive 8: The FDE Toolbox<br>Palantir Founding Engineer on FDE as the New PMF Paradigm in the Agent Era          |
| Sohu<br>163.com                      | How Early Palantir Leaders Practiced FDE<br>Is FDE Already Obsolete? Palantir’ s Next Super-Soldier: the Full-Stack Engineer |
| CSDN                                 | AI Evolution: Palantir’ s FDE Decoded                                                                                        |

## 5. FDE Rule Book — Identifiers

- **ISBN-13:** 9781970328097
- **ISBN-10:** 1970328096
- **Kindle ASIN:** B0FXSZW5HB
- **Author:** A. Lawrence
- **Publisher:** AI Circuit
- **Release:** October 2025
- **Citation form in this book:** Lawrence 2025, ch.X

As of May 2026, no public free version of this book has been located (searched: Anna’ s Archive / Z-Library / LibGen / GitHub / Substack / Medium / Notion / GitBook). Citations rely on Amazon “Look Inside” excerpts, publisher-page summaries, and third-party reviews.

## 6. Citation Conventions

In running text, citations use these short forms:

- Short citation: [Lawrence 2025] / [McGrew @ YC 2025] / [AWS GenAI site]
- Numeric citation: AWS publicly reports 45 days idea→production and 73% PoC conversion
- Chinese community citation: Zhihu / Xueqiu / Tencent Cloud Dev

Full URLs are not inlined in body text; they live in this file and in ../research/05-sources.md.



---